Cloud Computing

Comprehensive Study Notes

Complete Course Notes: Weeks 1 – 12 • Full Syllabus & Solved Problems

Course: NPTEL Cloud Computing (Prof. Soumya K. Ghosh, Department of Computer Science & Engineering, IIT Kharagpur)

Coverage: Complete 12-Week Curriculum: NIST cloud model, hypervisors and virtualization, XML and web services, SLAs and cloud economics, large-scale data management (GFS, BigTable, MapReduce), OpenStack, Microsoft Azure, Google Cloud Platform, cloud security, mobile cloud computing and cloudlets, sensor-cloud infrastructure, fog and edge computing, cloud federation, VM live migration and containers (Docker, Kubernetes), Dew computing, serverless FaaS, sustainable green clouds, 5G era networks and MEC, Cyber-Physical Systems (CPS), Spatial Cloud Computing (Traj-Cloud), and Internet of Health Things (IoHT).

Pedagogical Structure: In-depth lecture analyses, formal architectural TikZ diagrams, taxonomies, mathematical formulations, and a comprehensive collection of 122 solved NPTEL-style exam problems with complete derivations and explanations.

Publication-grade master reference manual compiled from official NPTEL lecture materials and curriculum standards (Weeks 1–12).

Contents

1 Introduction, NIST Model and Cloud Architecture Basics 8

1.1 Overview of Computing . . . 8
1.1.1 Course Roadmap . . . 8
1.1.2 Trends in Computing . . . 8
1.2 Distributed Computing . . . 9
1.2.1 Computers in a Distributed System . . . 9
1.2.2 Common Properties . . . 9
1.2.3 Why Distributed Computing? . . . 9
1.2.4 Distributed Applications: Client-Server vs. Peer-to-Peer . . . 10
1.3 Grid Computing . . . 10
1.3.1 Types of Grids . . . 11
1.4 Cluster Computing . . . 11
1.5 Utility Computing . . . 12
1.6 Cloud Computing: NIST Definition . . . 12
1.6.1 Five Essential Characteristics . . . 12
1.6.2 Common Characteristics . . . 13
1.7 Cloud Service Models . . . 13
1.7.1 Simplified Descriptions and Analogy . . . 14
1.8 Deployment Models . . . 14
1.9 Cloud and Virtualization . . . 15
1.10 Cloud-sourcing, Cloud Storage, Advantages and Disadvantages . . . 16
1.10.1 Advantages . . . 16
1.10.2 Disadvantages . . . 16
1.11 Business Drivers and Cloud Economics (Introductory) . . . 16
1.12 Cloud Computing Architecture . . . 17
1.12.1 Building Blocks . . . 17
1.12.2 XaaS . . . 18
1.12.3 Classical (Legacy) Model versus Evergreen IT . . . 18
1.12.4 Client-Server Model versus Cloud Model . . . 18
1.12.5 SaaS, PaaS, IaaS in Detail . . . 19
1.13 Role of Networking and Network Function Virtualization . . . 20
1.14 Week 1 Assignment Review . . . 20

2 Deployment Models, Virtualization, XML and Web Services 21

2.1 Deployment Models in Depth . . . 21
2.1.1 Public Cloud: Issues . . . 21
2.1.2 Private Cloud: On-site versus Outsourced . . . 22
2.1.3 Community and Hybrid Clouds . . . 22
2.2 IaaS: Provider-Subscriber View and Architecture . . . 22
2.2.1 Three-level Hierarchy . . . 23
2.3 Virtualization . . . 24
2.3.1 Hypervisor and Popek–Goldberg Requirements . . . 24
2.3.2 Approaches to Server Virtualization . . . 25
2.3.3 Host OS, Guest OS and Hypervisor . . . 26
2.4 Network Virtualization . . . 26
2.5 XML: eXtensible Markup Language . . . 26
2.5.1 Well-formedness, Entities and Parsers . . . 27
2.5.2 DTD, XML Schema and Namespaces . . . 28
2.5.3 XML Parser APIs and Processing . . . 28
2.6 Web Services and Service-Oriented Architecture . . . 29
2.6.1 SOA Roles and Operations . . . 29
2.6.2 The Web Service Stack . . . 30
2.6.3 Web Service Example: Web-based Purchase . . . 31
2.7 Week 2 Assignment Review . . . 31

3 SLAs, Cloud Economics, Data Management, MapReduce and OpenStack 32

3.1 Service Level Agreements (SLA) . . . 32
3.1.1 Availability and Downtime . . . 33
3.2 Cloud Economics . . . 33
3.2.1 Value of Common Infrastructure: Statistics of Scale . . . 34
3.2.2 Value of Location Independence . . . 34
3.2.3 Value of Utility Pricing . . . 35
3.2.4 Value of On-demand Services and Penalty Cost . . . 35
3.3 Managing Data in the Cloud . . . 36
3.3.1 Relational and Parallel Databases . . . 36
3.3.2 Google File System and HDFS . . . 37
3.3.3 BigTable . . . 38
3.3.4 Dynamo . . . 38
3.3.5 Datastore (App Engine) and SimpleDB . . . 39
3.4 Introduction to MapReduce . . . 39

3.4.1 Parallel Efficiency . . . 39
3.4.2 The Model . . . 40
3.4.3 MapReduce Efficiency . . . 40
3.5 OpenStack . . . 42
3.6 Week 3 Assignment Review . . . 43

4 Private Cloud with OpenStack, Microsoft Azure, and Google Cloud Platform 44
4.1 Introduction and Week Overview . . . 44
4.2 Private Cloud Implementation: Meghamala at IIT Kharagpur . . . 44
4.2.1 Architectural Blueprint and Infrastructure Topology . . . 44
4.2.2 Horizon Web Dashboard: Operational Modules . . . 45
4.2.3 Step-by-Step VM Provisioning and Access Workflow . . . 46
4.3 Microsoft Azure Enterprise Architecture . . . 47
4.3.1 Global Infrastructure: Regions and Availability Zones . . . 48
4.3.2 Azure Resource Management Hierarchy . . . 48
4.3.3 Azure App Service: Architecture and Deployment Topologies . . . 48
4.3.4 Azure SQL Database: Decoupled Storage and Hyperscale Tier . . . 50
4.3.5 Hands-On Deployment: Python Web App via Azure CLI . . . 50
4.4 Google Cloud Platform (GCP) Core Ecosystem . . . 51
4.4.1 Google Global Infrastructure and Software-Defined Networking . . . 51
4.4.2 Taxonomy of GCP Compute Services . . . 51
4.4.3 Google App Engine (GAE): Standard vs Flexible Environments . . . 52
4.4.4 GCP Storage and Database Services . . . 53
4.5 Cross-Cloud Platform Comparative Matrix . . . 54
4.6 Comprehensive Solved Assignment Problems (Week 4) . . . 54

5 Service Level Agreements, Cloud Economics, and Green Computing 58
5.1 Introduction and Week Overview . . . 58
5.2 Service Level Agreements (SLAs) in Cloud Computing . . . 58
5.2.1 Contractual Foundations and Definitions . . . 58
5.2.2 Availability Mathematics and “Nines” Notation . . . 59
5.2.3 Scheduled Operational Hours Downtime Formulation . . . 59
5.2.4 Service Credit Penalty Models . . . 60
5.3 Cloud Economics and Utility Pricing . . . 61
5.3.1 Economic Drivers of Cloud Adoption . . . 61
5.3.2 Formal Utility Premium Break-Even Analysis . . . 61
5.3.3 Workload Penalty Integrals for Static Provisioning . . . 62

5.4 Big Data Architecture: Hadoop and HDFS 63
5.4.1 Hadoop Distributed File System (HDFS) Architecture 63
5.4.2 Block Size and Input Split Mathematics 63
5.4.3 MapReduce Execution Pipeline: Partitioner vs Combiner 64
5.5 Green Cloud Computing and Server Energy Dynamics 64
5.5.1 Data Center Energy Footprint and Efficiency Metrics 64
5.5.2 Server Power Dynamics and Core Utilization 64
5.6 Comprehensive Solved Assignment Problems (Week 5) 65

6 Cloud Security, Privacy, and Trust Management 68
6.1 Introduction and Week Overview 68
6.2 Foundational Security Primitives: The CIA Triad 68
6.3 Structural Attack Taxonomy 69
6.3.1 Passive vs Active Attacks 70
6.3.2 The Multi-Phase Attack Lifecycle 70
6.4 Identity Management: Authentication vs Authorization 71
6.5 Virtualization Security and Multi-Tenancy Exploits 71
6.5.1 Hypervisor Isolation and Virtual Machine Escape 71
6.5.2 Cross-VM Side-Channel Attacks (Ristenpart et al.) 72
6.6 Gartner's Seven Cardinal Cloud Security Risks 72
6.7 Comprehensive Solved Assignment Problems (Week 6) 73

7 Mobile Cloud Computing, Cloudlets, and Geospatial Cloud 76
7.1 Introduction and Week Overview 76
7.2 Mobile Cloud Computing (MCC) Foundations 76
7.2.1 Motivation and Intrinsic Hardware Constraints 76
7.2.2 Workload Suitability for Offloading 77
7.3 Mathematical Formulation of Computation Offloading 77
7.3.1 System Model and Parameters 77
7.3.2 Energy Consumption Derivations 78
7.4 Cloudlets and Mobile Edge Architecture 80
7.4.1 The WAN Latency Bottleneck 80
7.4.2 Three-Tier Cloudlet Architecture 80
7.5 Geospatial Cloud Computing 80
7.5.1 Foundations of Geographic Information Systems (GIS) 80
7.5.2 Spatial Data Infrastructure (SDI) and OGC Web Standards 81
7.5.3 Cloud Scaling for Big Spatial Data 81
7.6 Comprehensive Solved Assignment Problems (Week 7) 81

8 Containers, Docker Virtualization, and Sensor-Cloud Infrastructure 85
8.1 Introduction and Week Overview . . . 85
8.2 Containers vs Virtual Machines: Architectural Divergence . . . 85
8.2.1 Structural Comparison . . . 85
8.2.2 Quantitative Tradeoff Analysis . . . 86
8.3 The Docker Ecosystem and Architecture . . . 86
8.3.1 Docker Engine Components . . . 86
8.3.2 Copy-on-Write (CoW) and Storage Layering . . . 87
8.3.3 Dockerfile Syntax and Construction . . . 87
8.3.4 Complete Docker Command Lifecycle . . . 88
8.4 Container Orchestration: Kubernetes and etcd . . . 88
8.5 Sensor-Cloud Infrastructure . . . 88
8.5.1 Limitations of Traditional Wireless Sensor Networks (WSNs) . . . 89
8.5.2 The Sensor-Cloud Paradigm . . . 89
8.5.3 Stakeholders and Economic Model . . . 89
8.5.4 Virtual Sensor Configurations and Mapping Topologies . . . 90
8.6 Comprehensive Solved Assignment Problems (Week 8) . . . 90

9 Hierarchical Edge-Fog-Cloud Computing and Cloud Federation 93
9.1 Introduction and Week Overview . . . 93
9.2 The Edge-Fog-Cloud Computing Hierarchy . . . 93
9.2.1 The Need for Decentralization . . . 93
9.2.2 Architectural Division of Responsibilities . . . 93
9.2.3 Empirical Simulation Analysis: The iFogSim Framework . . . 94
9.3 Resource Management and Scalability Models . . . 95
9.3.1 Resource Management Taxonomies . . . 95
9.3.2 Vertical vs Horizontal Scalability . . . 95
9.4 Cloud Federation and Inter-Cloud Governance . . . 95
9.4.1 Motivation for Cloud Federation . . . 95
9.4.2 Federation Architectural Topologies . . . 96
9.5 Comprehensive Solved Assignment Problems (Week 9) . . . 96

10 Virtual Machine Migration, Container Primitives, and Cloud Orchestration 100
10.1 Introduction and Week Overview . . . 100
10.2 Virtual Machine Migration Fundamentals . . . 100
10.2.1 Motivation and Operational Drivers . . . 100
10.2.2 The Migration State Tuple . . . 101

10.3 Cold vs Live Migration Taxonomy . . . 101
10.3.1 Cold Migration Transfer Duration Formulation . . . 101
10.4 Live Migration Paradigms: Pre-Copy vs Post-Copy . . . 102
10.4.1 Pre-Copy Iterative Live Migration . . . 102
10.4.2 Post-Copy Live Migration . . . 103
10.5 Linux Kernel Container Primitives . . . 103
10.5.1 Linux Namespaces: Isolation of System Views . . . 104
10.5.2 Control Groups (cgroups): Resource Allocation and Quotas . . . 104
10.5.3 Docker Storage Persistence: Named Volumes . . . 104
10.6 Comprehensive Solved Assignment Problems (Week 10) . . . 105

11 Emerging Paradigms: Dew Computing, Serverless Architecture, and Sustainable Cloud 108
11.1 Introduction and Week Overview . . . 108
11.2 Dew Computing Foundations . . . 108
11.2.1 Motivation: Overcoming the Internet Dependency Bottleneck . . . 108
11.2.2 Core Principles: Independence and Collaboration . . . 109
11.2.3 Dew Service Architecture Taxonomy . . . 109
11.3 Serverless Computing and Function-as-a-Service (FaaS) . . . 110
11.3.1 The Paradigm Shift from Servers to Applications . . . 111
11.3.2 Architectural Characteristics of FaaS . . . 111
11.3.3 The Serverless Cold Start Challenge . . . 111
11.3.4 Serverless Economic Billing Formulation . . . 112
11.4 Sustainable and Green Cloud Computing . . . 112
11.4.1 The Data Center Energy Dilemma . . . 112
11.4.2 Energy Breakdown Within a Cloud Data Center . . . 112
11.4.3 Energy Efficiency Metrics: PUE and DCiE . . . 113
11.4.4 Power Management and Conservation Approaches . . . 113
11.5 Comprehensive Solved NPTEL-Style Questions (Week 11) . . . 114

12 Advanced Paradigms and Case Studies: 5G Era, Cyber-Physical Systems, Spatial Cloud, and IoHT 118
12.1 Introduction and Week Overview . . . 118
12.2 Cloud Computing in the 5G Era . . . 118
12.2.1 Generational Evolution of Mobile Networks . . . 118
12.2.2 The Three Functional Pillars of 5G . . . 119
12.2.3 5G and Cloud Synergy: Softwarization and Edge Convergence . . . 119
12.2.4 Mobile Cloud Computing (MCC) vs. MEC . . . 120

12.3 Cyber-Physical Systems (CPS) and Cloud Integration 120
12.3.1 Foundations of Cyber-Physical Systems 120
12.3.2 CPS vs. Traditional Embedded Systems and IoT 120
12.3.3 Cyber-Physical Cloud Computing (CPCC) 121
12.4 Case Study I: Spatial Cloud Computing and Traj-Cloud 121
12.4.1 Geospatial Data Characteristics 121
12.4.2 Spatial Cloud Computing Principles 122
12.4.3 Traj-Cloud: Trajectory Cloud for Mobility Trace Mining 122
12.5 Case Study II: Internet of Health Things (IoHT) 122
12.5.1 The Healthcare Paradigm Problem 122
12.5.2 The Four-Tier Hierarchical Network Topology 123
12.5.3 Empirical Simulation Parameters (iFogSim Model) 123
12.5.4 Application Module Placement: Fog Model vs. Cloud Model 123
12.5.5 Comparative Performance Evaluation Metrics 124
12.5.6 Hardware Prototype Implementation 124
12.5.7 Dew Resilience Extension in Healthcare 125
12.6 Comprehensive Solved NPTEL-Style Questions 127

Introduction, NIST Model and Cloud Architecture Basics

1.1 Overview of Computing

The ACM Computing Curricula 2005 defines *computing* as "any goal-oriented activity requiring, benefiting from, or creating computers". It therefore covers designing and building hardware and software systems, processing and managing information, scientific studies using computers, making systems behave intelligently, creating and using communication and entertainment media, and finding and gathering relevant information.

1.1.1 Course Roadmap

1. **Introduction:** overview of computing, NIST model, properties, characteristics, disadvantages, role of open standards.
2. **Architecture:** cloud stack, service models (IaaS, PaaS, SaaS), deployment models.
3. **Service management:** Service Level Agreements (SLAs) and cloud economics.
4. **Resource management** in cloud computing.
5. **Data management:** scalability, databases and data stores, large-scale data processing.
6. **Cloud security:** infrastructure security, data security and storage, identity and access management, access control, trust, reputation, risk.
7. **Case studies:** open-source and commercial clouds, cloud simulators.
8. **Research trends:** fog computing and others.

1.1.2 Trends in Computing

The lecture traces the evolution *Distributed Computing* → *Grid Computing* → *Cluster Computing* → *Utility Computing* → *Cloud Computing*.



Figure 1.1: Evolution of computing paradigms leading to cloud computing.

1.2 Distributed Computing

Definition: Distributed System

A distributed system consists of several **autonomous computational entities**, each with its **own local memory**, that communicate with each other by **message passing** (Wikipedia). In *Operating System Concepts*, processors communicate through lines such as high-speed buses or telephone lines, and each processor has its own local memory. *Distributed computing* is the field that studies distributed systems and uses them to solve computational problems.

Early computing used a single processor (*uni-processor* or *centralized* computing). Examples of distributed systems are the Internet, ATM (bank) machines, intranets/workgroups, and the emerging landscape of ubiquitous network-connected devices.

1.2.1 Computers in a Distributed System

- **Workstations:** used by end users to perform computing.
- **Server systems:** provide resources and services.
- **Personal assistance devices:** handheld computers connected over wireless links.

1.2.2 Common Properties

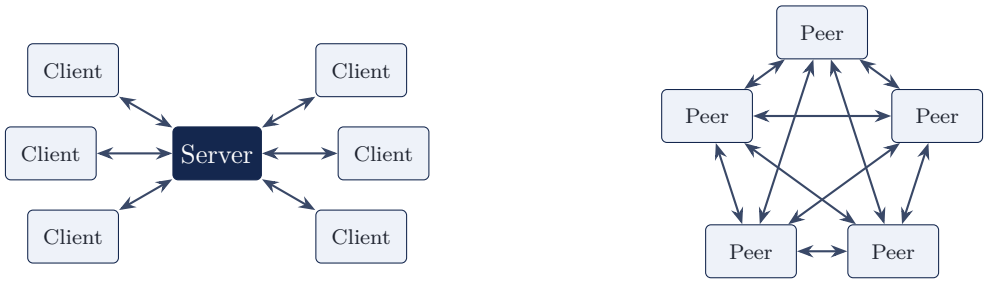
- **Fault tolerance:** when one or some nodes fail, the system still works (only performance degrades); the status of every node must be checked.
- **Partial role of each node:** each computer has a limited, incomplete view of the system and may know only one part of the input.
- **Resource sharing:** users share computing power and storage.
- **Load sharing:** dispatching tasks to different nodes balances system load.
- **Easy expansion:** adding nodes should take little or no time.
- **Performance:** parallel computing is considered a subset of distributed computing.

1.2.3 Why Distributed Computing?

- **Nature of application.**
- **Performance:** *computing-intensive* tasks (e.g. Monte Carlo estimation of π) and *data-intensive* tasks (e.g. Facebook, Large Hadron Collider data processing).
- **Robustness:** no *Single Point Of Failure* (SPOF); other nodes can execute the task of a failed node.

1.2.4 Distributed Applications: Client-Server vs. Peer-to-Peer

A distributed application is a set of processes distributed across a network of machines that work as an ensemble to solve a common problem. In the past most were **client-server** (resource management centralised at the server). **Peer-to-peer (P2P)** computing represents a movement toward "truly" distributed applications.



(a) Clients invoke servers (b) Peer processes

Figure 1.2: Client-server versus peer-to-peer distributed applications.

1.3 Grid Computing

Definition: Grid Computing

- **Pcwebopedia:** harnesses *unused processing cycles* of all computers in a network for problems too intensive for any stand-alone machine.
- **IBM:** enables the *virtualization* of distributed computing and data resources (processing, network bandwidth, storage) to create a *single system image*; a grid user sees one large virtual computer.
- **Sun Microsystems:** infrastructure providing *dependable, consistent, pervasive and inexpensive* access to computational capabilities.

Electrical power grid analogy. Users plug into a wall socket without caring where the electricity is generated; the power grid links many kinds of power plants. Likewise, grid users get processors, storage, data and applications as needed with little or no knowledge of where resources are or what hardware/OS lies beneath; "the Grid" links PCs, workstations, servers and storage elements and provides the mechanism to access them.

Features of grids: (1) share more than information – data, computing power and applications in a dynamic, multi-institutional *virtual organisation*; (2) efficient use of resources at many institutes; (3) join local communities; (4) interaction with the underlying layers must be *transparent and seamless.*

Need: today's science is based on computation, data analysis, visualization and collaboration; simulation and modelling are more cost effective than experiments; problems are increasingly complex and need accurate solutions quickly; data visualization is important; and under-utilised resources can be exploited.

1.3.1 Types of Grids

Computational grid	Secure access to a huge pool of shared processing power for high-throughput and computation-intensive applications.
Data grid	Infrastructure for storage, discovery, handling, publication and manipulation of large data volumes stored in heterogeneous databases and file systems.
Collaboration grid	Supports collaboration, e.g. people from different companies in a virtual enterprise working on components of a CAD project without disclosing proprietary technology.
Network grid	Fault-tolerant, high-performance communication; each node acts as a data router providing caching to speed up communication.
Utility grid	Ultimate form: not only data and cycles but software and special equipment are shared; applications run on one machine and users send data and receive results.

1.4 Cluster Computing

Definition: Cluster

A cluster is a type of parallel or distributed computer system consisting of a collection of **inter-connected stand-alone computers** working together as a **single integrated computing resource**. Key components: multiple standalone computers (PCs, workstations or SMPs), operating systems, high-performance interconnects, middleware, parallel programming environments and applications.

Clusters improve speed and/or reliability over a single computer and are more cost effective than a single computer of comparable speed or reliability. In a typical cluster the network is faster and closer than a LAN, uses low-latency protocols, and is *more loosely coupled than SMP*.

Types: High Availability (failover) clusters; Load-Balancing clusters; Parallel/Distributed processing clusters. **Components:** cluster nodes, cluster network, network characterisation.

Benefits: system availability (redundant hardware, OS and applications); hardware fault tolerance (e.g. disk RAID); OS and application reliability; scalability (add servers or clusters); high performance for cluster-enabled programs.

Note: Grid versus Cluster (Assignment 1, Q10)

A **grid** is a form of distributed computing in which a large number of geographically dispersed, often heterogeneous resources across multiple administrative domains are *loosely coupled* to work on a large-scale task. A **cluster** is tightly coupled and co-located; a node is a single machine; a hypervisor is virtualization software.

1.5 Utility Computing

Definition: Utility Computing

Utility computing is a *service provisioning model* in which a provider makes computing resources and infrastructure management available to the customer as needed and charges for **specific usage rather than a flat rate**. There is low or no initial cost; resources are essentially rented. The analogy is electricity: fluctuating customer demand, charged by usage (*pay-per-use*). Utility computing is purely a concept which cloud computing practically implements, usually with virtualization so that available storage/computing power is far larger than that of a single time-sharing computer.

Key elements: (a) pay-for-use pricing model, (b) data-centre virtualization and provisioning, (c) solves the resource-utilization problem, (d) outsourcing, (e) web services delivery, (f) automation.

Payment models: flat rate, tiered, subscription, metered, pay-as-you-go, standing charges; different customers get different models depending on scale, commitment and payment frequency; the price expresses the provider's cost plus a profit margin.

Risks: data backup, data security, partner competency, defining the SLA, getting value from charge-back.

1.6 Cloud Computing: NIST Definition

Definition: Cloud Computing (NIST)

Cloud computing is a model for enabling **ubiquitous, convenient, on-demand network access** to a **shared pool of configurable computing resources** (e.g. networks, servers, storage, applications and services) that can be **rapidly provisioned and released with minimal management effort or service provider interaction**.

1.6.1 Five Essential Characteristics

1. **On-demand self-service:** a consumer can unilaterally provision capabilities such as server time and network storage automatically, without human interaction with each provider.
2. **Broad network access:** capabilities are available over the network and accessed by standard mechanisms that promote use by heterogeneous thin or thick clients (mobile phones, tablets, laptops, workstations).
3. **Resource pooling:** the provider's resources are pooled to serve many consumers using a *multi-tenant model*, with physical and virtual resources dynamically assigned and reassigned according to demand.
4. **Rapid elasticity:** capabilities can be elastically provisioned and released, sometimes automatically, to scale outward and inward with demand; to the consumer they often appear unlimited.
5. **Measured service:** resource use is automatically controlled and optimised using a metering capability (storage, processing, bandwidth, active user accounts) and can be monitored, controlled and reported, giving transparency to provider and consumer.

1.6.2 Common Characteristics

Massive scale, resilient computing, homogeneity, geographic distribution, virtualization, service orientation, low-cost software, advanced security.

Solved Example: Identifying the characteristic

Given: Statement: "Computing resources are dynamically assigned to multiple customers using a shared infrastructure."
Formula: Match the statement to a NIST essential characteristic.
Step-by-step substitution:

1. Many customers share the same physical hardware ⇒ multi-tenancy.
2. Assignment and reassignment follows demand ⇒ dynamic.
3. These two properties together define *resource pooling*.

Final Answer: Resource pooling.

1.7 Cloud Service Models

Model	Capability provided to the consumer	Consumer controls
SaaS	Use provider's applications on cloud infrastructure via thin client (web browser) or program interface. Example: Google Spreadsheet.	Nothing underneath; at most limited user-specific configuration.
PaaS	Deploy consumer-created or acquired applications using languages, libraries, services and tools supported by the provider. Examples: Microsoft Azure, Google App Engine.	Deployed applications and possibly hosting-environment configuration; not network, servers, OS, storage.
IaaS	Provision processing, storage, networks and other fundamental resources; consumer deploys arbitrary software. Examples: Amazon Web Services, Flexiscale, EC2.	OS, storage, deployed applications and possibly some networking components.

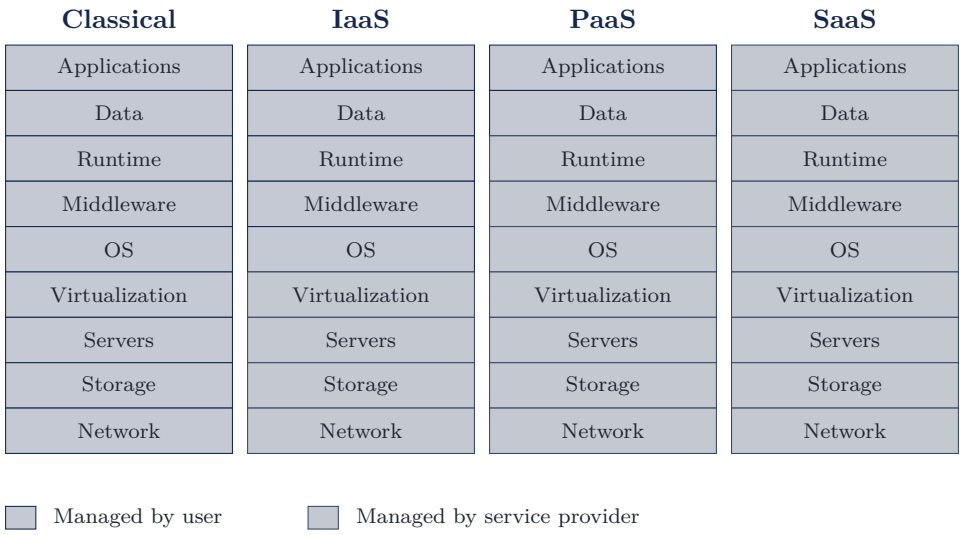


Figure 1.3: Responsibility split in the classical model, IaaS, PaaS and SaaS.

Solved Example: Classify the service model

Given: A startup rents Amazon EC2 virtual servers; the team installs and patches its own OS, configures the web server, manages runtime and applications. The provider supplies only virtualized compute, storage and networking.
Formula: Service model = determined by the lowest layer the provider manages and the first layer the consumer manages.
Step-by-step substitution:

1. Provider manages: virtualization, servers, storage, network.
2. Consumer manages: OS, middleware/runtime, applications, data.
3. This matches the right-most boundary of the IaaS column in the figure above.

Final Answer: **IaaS**. SaaS (no consumer control), PaaS (OS is provider-managed) and FaaS are rejected.

1.7.1 Simplified Descriptions and Analogy

SaaS applications are designed for end users and delivered over the web; PaaS is the set of tools and services that makes coding and deploying applications quick and efficient; IaaS is the hardware and software that powers it all – servers, storage, networks, operating systems. In the **transportation analogy**, roads are the infrastructure (IaaS), cars and trucks are the platform (PaaS) and the people and goods are software and information (SaaS).

1.8 Deployment Models

- **Private cloud:** operated solely for one organization (e.g. Windows Server Hyper-V).
- **Community cloud:** shared by several organizations supporting a specific community goal.
- **Public cloud:** made available to the general public (e.g. Google Docs, Spreadsheet).

- **Hybrid cloud:** composition of two or more clouds (private, community or public), e.g. *cloud bursting* for load balancing between clouds.

1.9 Cloud and Virtualization

Virtual workspaces are abstractions of an execution environment, dynamically available to authorised clients through well-defined protocols, with a resource quota (CPU, memory share) and a software configuration (OS). They are implemented on **virtual machines (VMs)**: an abstraction of a physical host machine. A **hypervisor** (VMM) intercepts and emulates VM instructions and manages VMs (VMware, Xen, KVM). Para-virtualization (e.g. Xen) approaches raw physical performance.

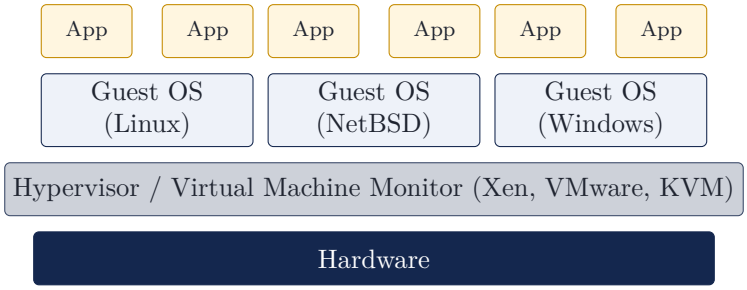


Figure 1.4: Virtualized stack: several guest operating systems share one physical machine.

Advantages of VMs: run OSs where physical hardware is unavailable; easy to create and back up machines; software testing on clean installs; emulate more machines than physically available; timeshare lightly loaded systems on one host; debug by suspend/resume; easy migration; run legacy systems.

Solved Example: Virtualization statements

Given: Four statements about VMs and hypervisors.
Formula: A VM provides a virtual machine with its own guest OS; Type-1 hypervisors run on bare metal, Type-2 run as applications on a host OS.
Step-by-step substitution:

1. "Each VM always shares the host kernel" describes *containers*, not VMs – false.
2. Multiple VMs can run different OSs simultaneously on one machine – **true**.
3. Virtualization pre-dates cloud, so "only possible in cloud" is false.
4. Overcommitment (CPU/memory) is a commonly used feature, not something virtualization prevents.
5. Type-2 (hosted) hypervisors (VirtualBox, VMware Workstation) run as an application on a host OS.

Final Answer: VMs of different OSs coexist (Q4: B); Type-2 hypervisor runs on top of a host OS (Q7: B).

1.10 Cloud-sourcing, Cloud Storage, Advantages and Disadvantages

Why cloud-sourcing matters: high-scale low-cost providers, any time/place access via browser, rapid scalability with incremental cost and load sharing, and no need to focus on local IT. **Concerns:** performance, reliability and SLAs; control of data and service parameters; application features and choices; interaction between providers; no standard API (mix of SOAP and REST); privacy, security, compliance, trust. **Cloud storage:** large web companies rent out spare storage (e.g. Amazon EC2 and S3).

1.10.1 Advantages

Lower computer costs; improved performance; reduced software costs; instant software updates; improved document-format compatibility; unlimited storage capacity (hundreds of petabytes versus a 1 TB drive); increased data reliability (e.g. Dropbox, SkyDrive); universal information access; latest-version availability; easier group collaboration; device independence.

1.10.2 Disadvantages

Requires a constant Internet connection; does not work well with low-speed connections; features may be limited (e.g. PowerPoint versus web presentation tools); can be slow; stored data might not be secure; stored data can be lost; HPC applications using MPI/OpenMP are difficult (co-location of VMs needed to minimise latency); general concerns – different protocols and APIs per cloud, proprietary database and workflow systems requiring application adaptation.

1.11 Business Drivers and Cloud Economics (Introductory)

Public clouds can significantly reduce IT cost. To the end user, the cloud gives the illusion of potentially infinite capacity with rapid scale and pay-per-use. In contrast, private data centres must be *provisioned for peak capacity*, giving a low average utilization of **5–20%**.

	In-house server	Cloud server
Purchase cost	$9600	0
Cost/hr (over 3 years)	$0.36	$0.68
Cost ratio Cloud/In-house	1.88	
Efficiency	40%	80%
Cost per effective hour	$0.90	$0.85
Power and cooling	$0.36	0
Management cost	$0.10	$0.01
Total cost/effective hour	**$1.36**	**$0.86**
Cost ratio In-house/Cloud	1.58	

The in-house server in the table is an x86 machine with three quad-core processors, 12 GB RAM and a 300 GB disk.

Solved Example: IaaS economics: why the dearer cloud server is cheaper per useful hour

Given: In-house: \$0.36/hr, efficiency 40%, power and cooling \$0.36, management \$0.10. Cloud: \$0.68/hr, efficiency 80%, power and cooling \$0, management \$0.01.

Formula: Cost per effective hour $= \frac{\text{cost per hour}}{\text{efficiency}}$; total = cost per effective hour + power and cooling + management.

Step-by-step substitution:

1. In-house: $0.36/0.40 = 0.90$; total $= 0.90 + 0.36 + 0.10 = 1.36$.
2. Cloud: $0.68/0.80 = 0.85$; total $= 0.85 + 0 + 0.01 = 0.86$.
3. Raw price ratio: $0.68/0.36 = 1.88$ (cloud dearer per hour).
4. Total ratio: $1.36/0.86 = 1.58$.

Final Answer: Total cost per effective hour: in-house \$1.36, cloud \$0.86, ratio ≈ 1.58 in favour of the cloud.

Benefits for end users of public cloud: high utilization and scalability; no hardware procurement; no power cost; no IT administration; user-friendly SLAs (about 99% availability with compensation on a miss); rent to prototype before investing; portability standards (Cloud Computing Interoperability Forum, Open Cloud Consortium; Amazon EC2 and Eucalyptus share the same API); startups avoid capital investment. **Benefits for SMBs (fewer than 250 employees)** follow the same lines. **Benefits of private cloud:** one 12-core 12 GB server costs far less than twelve 1-core 1 GB servers; data confidentiality preserved; VMs cheaper and faster to provision than physical machines.

Economics of PaaS versus IaaS: for a 24x7 web application with unpredictable volume, IaaS requires a minimum set of servers always provisioned, while deploying on PaaS costs nothing at rest, costs are incurred by usage, and the platform scales automatically. **PaaS benefits:** no scaling or load balancing by the user; web-based IDE; easier migration from development to production; no separate IDE licences. **SaaS benefits:** subscribe instead of buying and licensing software; use free tools such as Google Docs; enterprises use web-based CRM instead of installing servers.

Purposes of cloud in organizations: IT platform for multi-organization processes; backup; CRM, ERP, supply-chain; personal productivity and collaboration; development and test; storing and archiving large files; analysing customer/operations data; e-business and e-government sites; R&D analysis; handling demand spikes; batch processing (conversion, risk modelling, rendering); sharing information with regulators; consumer entertainment. **Top cloud applications:** mail and messaging, archiving, backup, storage, security, virtual servers, CRM, cross-enterprise collaboration, hosted PBX, video conferencing.

1.12 Cloud Computing Architecture

1.12.1 Building Blocks

- **Technical architecture:** structuring by the XaaS stack; adopting cloud paradigms; structuring services and components; relationships and external endpoints; middleware and communication; management and security.

- **Deployment/operation architecture:** geo-location check (legal issues, export control); operation and monitoring.

1.12.2 XaaS

Definition: XaaS

Combining Service-Oriented Infrastructure (SOI) with cloud computing realises **XaaS**: "anything" or "everything" as a Service – a growing number of services delivered over the Internet rather than locally. **Broad network access + resource pooling (cloud) + business-driven infrastructure on demand + service orientation (SOI) = XaaS**, which fulfils the CSP demands: increase productivity, end-user satisfaction, innovation and agility.

Other XaaS examples: Business Process as a Service (BPaaS), Storage as a Service, Security as a Service (SECaaS), Database as a Service (DaaS), Monitoring/Management as a Service (MaaS), Communications, Content and Computing as a Service (CaaS), Identity as a Service (IDaaS), Backup as a Service (BaaS), Desktop as a Service.

Note: Assignment 1, Q6

The combination of SOA and cloud computing leading to "almost any IT capability delivered as a service" is called **XaaS**. FaaS (Functions as a Service) is just one specific model.

1.12.3 Classical (Legacy) Model versus Evergreen IT

In the classical model all layers (hardware, OS, development tools, applications) are managed by users who bear the hardware, maintenance and technology cost; systems are custom build-to-order, deployed as tightly coupled vertical stacks of layers (no single part easily replaced), with manual provisioning – *Legacy IT*. Cloud simplifies the stack towards *Evergreen IT*: dedicated infrastructure, application and end-user devices form a simplified IT stack.

1.12.4 Client-Server Model versus Cloud Model

	Client-server model	Cloud computing model
Service model	Simple: server services client requests	Variety of complex models (IaaS, PaaS, SaaS)
Load balancing	May or may not be load balanced	Load balanced
Scalability	Scalable to some extent in a cluster	Theoretically infinitely scalable
Virtualization	No concept of virtualization	Virtualization is the core concept

A three-tier client-server architecture has presentation, application and data tiers; the client is often a thin client, and the server may be a load-balanced cluster or a stand-alone machine.

1.12.5 SaaS, PaaS, IaaS in Detail

	SaaS	PaaS	IaaS
Definition	Software deployed over the Internet, licensed on demand, by subscription, pay-as-you-go, or free (ad/user-list revenue).	Platform for creating web applications quickly without buying and maintaining underlying software and infrastructure.	Servers, storage, network and OS delivered as an on-demand, outsourced service.
Characteristics	Web access to commercial software; managed from a central location; one-to-many delivery; no upgrades or patches by users; APIs for integration.	Integrated development, test, deploy, host and maintain services; web-based UI tools; multi-tenant; built-in scalability, load balancing and failover; web-service/database integration; team collaboration; billing and subscription tools.	Resources distributed as a service; dynamic scaling; variable utility pricing; multiple users per hardware.
Good fit	Interplay with the outside world (e.g. email campaigns); web/mobile access; short-term use; demand spikes (tax/billing); e.g. Salesforce CRM.	Multiple developers or external parties; automated test and deployment; agile development; e.g. Azure, Google App Engine.	Volatile demand; new organizations without capital; rapid growth; limiting CAPEX in favour of OPEX; trial or temporary needs.
Poor fit	Extremely fast real-time processing; regulations forbidding external hosting; existing on-premise solution suffices.	Highly portable applications; proprietary languages causing lock-in; need to customise underlying hardware/software.	Regulatory limits on offshoring/outsourcing data; highest performance needs met by on-premise hardware.

Solved Example: Advantages of PaaS

Given: Four claimed advantages: (i) developers focus on application development, (ii) automatic software updates of the underlying platform, (iii) complete control over physical server hardware, (iv) faster application deployment.
Formula: PaaS abstracts infrastructure management and offers managed runtime environments.
Step-by-step substitution:

1. (i) true – infrastructure is abstracted. (ii) true – the provider updates the platform.
2. (iii) false – physical hardware is the provider's responsibility.
3. (iv) true – ready-made runtime and IDE speed deployment.

Final Answer: (i), (ii) and (iv)

1.13 Role of Networking and Network Function Virtualization

Network resources can be provisioned dynamically in the cloud. Core concepts: Virtual LANs, Virtual Private Networks and protocol layers; tools such as OpenSSH and OpenVPN set up topologies and configurations.

Definition: Network Functions Virtualisation (ETSI NFV)

NFV aims to transform how operators architect networks by evolving standard IT virtualisation technology to consolidate many network-equipment types onto industry-standard high-volume servers, switches and storage in data centres, network nodes and end-user premises. Network functions are implemented in software that can be moved to, or instantiated in, various locations as required, without installing new equipment.

1.14 Week 1 Assignment Review

Q	Concept tested	Answer
1	Resource pooling (multi-tenant shared resources)	B
2	EC2 rented VM with user-managed OS ⇒ IaaS	C
3	Private cloud: exclusive use, on-premises or third-party hosted	D
4	Different OS VMs simultaneously on one machine	B
5	Distributed systems: scalability and fault tolerance, *not* guaranteed lower latency	A
6	SOA + cloud ⇒ XaaS	C
7	Type-2 hypervisor runs as an application on a host OS	B
8	PaaS advantages (i), (ii), (iv)	A
9	Amazon EC2 is IaaS	B
10	Grid: geographically dispersed, heterogeneous, loosely coupled	B

Solved Example: Distributed system statements

Given: (i) adding nodes improves scalability; (ii) replication increases fault tolerance; (iii) distribution guarantees lower latency for every operation than a centralized system.
Formula: Distribution adds network communication and coordination overhead.
Step-by-step substitution:

1. (i) True: horizontal scaling.
2. (ii) True: replication of data and services.
3. (iii) False: remote calls and coordination can make some operations slower.

Final Answer: (i) and (ii) only

Deployment Models, Virtualization, XML and Web Services

2.1 Deployment Models in Depth

Model	Definition	Examples
Public	Infrastructure provisioned for open use by the general public; owned, managed and operated by a business, academic or government organization; exists on the provider's premises.	Google App Engine, Microsoft Windows Azure, IBM SmartCloud, Amazon EC2
Private	Provisioned for exclusive use by a single organization with multiple consumers (business units); owned and operated by the organization, a third party or both; on or off premises.	Eucalyptus, Ubuntu Enterprise Cloud, Amazon VPC, VMware Cloud Infrastructure Suite
Community	Provisioned for a specific community with shared concerns (mission, security, policy, compliance); owned and operated by one or more members, a third party, or both.	Google Apps for Government, Microsoft Government Community Cloud
Hybrid	Composition of two or more distinct clouds (private, community, public) that remain unique entities but are bound by standardized or proprietary technology enabling data and application portability.	Windows Azure (hybrid capable), VMware vCloud

2.1.1 Public Cloud: Issues

- **Hidden workload location:** the provider may migrate workloads anywhere at any time (e.g. to low-cost data centres) unless location-restriction policies are offered.
- **Multi-tenancy risk:** a machine may host workloads of competitors or adversaries (reliability and security risk).
- **Network dependency:** access over the public Internet depends on DNS servers, router infrastructure and inter-router links.
- **Limited visibility and control:** provider operations and software are proprietary; a subscriber cannot verify complete deletion of data.

- **Elasticity:** illusion of unlimited resources; high flexibility in moving workloads.
- **Low up-front cost**, but **restrictive default SLAs**.

2.1.2 Private Cloud: On-site versus Outsourced

Aspect	On-site private	Outsourced private
Security perimeter	One perimeter around subscriber resources and private cloud	Two perimeters (subscriber and provider) joined by a protected link; security depends on both and the link
Up-front cost	Significant to high (install and manage cloud software even if free)	Modest to significant (negotiating SLA, upgrading network, porting, training)
Resources	Fixed capacity sized for anticipated workloads	Extensive; rent any quantity offered
Data import/export	Limited by on-site network capacity	Limited by provider-subscriber link, but adjustable by provisioning high-performance links
IT skills	Traditional plus cloud IT skills required	Less in-house operation
Multi-tenancy	Workloads of different clients separated only by access policies; a software or policy flaw can expose workloads	Same implications

2.1.3 Community and Hybrid Clouds

In a community cloud, each participant organization may provide services, consume services, or both (at least one must provide) and each implements its own security perimeter; perimeters are linked by boundary controllers. Access policy can be complex: with N members a sharing decision must be made between each pair, expressed with role-based (RBAC) or attribute-based access control. Costs for a participant vary widely depending on whether it consumes only or also provides. An *outsourced community cloud* resembles an outsourced private cloud but with multiple protected links. A hybrid cloud can be extremely complex and may change over time as constituent clouds join and leave.

2.2 IaaS: Provider-Subscriber View and Architecture

A subscriber gets access to virtual computers, network-accessible storage, network components such as firewalls, and configuration services. Fees are typically per CPU-hour, GB stored per hour, bandwidth, network infrastructure used (e.g. IP addresses per hour) and value-added services (monitoring, auto-scaling). The provider keeps spare VMs (e.g. clients A, B, C hold vm1–vm6 while the provider retains vm7–vmN).

Scope of control: the IaaS stack comprises hardware, OS (split into a privileged hypervisor/VMM layer and a guest OS layer), middleware and applications. The provider keeps total control of hardware and administrative control of the hypervisor; the subscriber controls the

guest OS, middleware and applications and may load any supported OS image. A hypervisor synthesizes VMs, each *"an efficient, isolated duplicate of a real machine"*.

2.2.1 Three-level Hierarchy

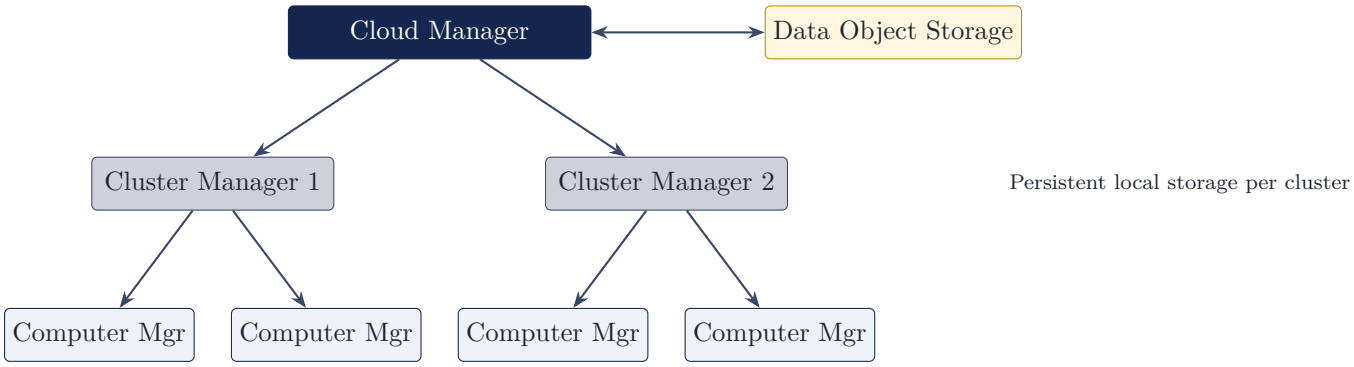


Figure 2.1: Logical view of an IaaS cloud: top, middle and bottom levels.

- **Cloud Manager:** public access point where subscribers sign up, manage rented resources and access data; authenticates subscribers, generates/validates access credentials, performs top-level resource management, and decides if the cloud has enough free resources.
- **Data Object Storage (DOS):** stores subscriber metadata such as credentials and OS images; usually one service per cloud.
- **Cluster Manager:** manages a collection of computers on a high-speed LAN (clusters may be geographically distributed); receives allocation commands from the Cloud Manager, queries Computer Managers for availability, instructs allocation, and reconfigures virtual networking; attached to *Persistent Local Storage (PLS)* giving disk-like storage to VMs.
- **Computer Manager:** runs on each host; uses virtualization to provide VMs; tracks how many VMs run and how many can start; uses the hypervisor command interface to start, stop, suspend and reconfigure VMs.

Solved Example: Capacity check by the hierarchy

Given: Two clusters, each with 10 computers; each computer can host at most 8 VMs; 60 VMs already running; a subscriber requests 25 more VMs.
Formula: Free slots = (clusters × computers × slots) − running VMs.
Step-by-step substitution:

1. Total capacity $= 2 \times 10 \times 8 = 160$ VM slots.
2. Free slots $= 160 - 60 = 100$.
3. The Cluster Managers report availability upward; $25 \leq 100$, so the Cloud Manager can satisfy the request.

Final Answer: Request accepted; 75 slots remain free after allocation.

2.3 Virtualization

Virtualization lets one computer do the job of several by sharing a single hardware platform among multiple isolated environments; the focus here is *platform virtualization*. Unlike dual-boot, both OSes run *simultaneously* and are completely isolated.

2.3.1 Hypervisor and Popek–Goldberg Requirements

A hypervisor runs the guest OS directly on the CPU in user mode, so privileged instructions must be intercepted. Popek and Goldberg (CACM 1974) give three goals:

- **Equivalence:** VM indistinguishable from the real hardware.
- **Resource control:** VMM fully controls virtualized resources.
- **Efficiency:** most instructions execute directly on the CPU without the hypervisor.

Instructions are *privileged* (trap in user mode) or *sensitive* (change underlying resources – *control sensitive*, or reveal the privilege level – *behavior sensitive*).

Key Result: Popek–Goldberg theorem

For any conventional third-generation computer, a VMM may be constructed if the set of sensitive instructions is a **subset** of the set of privileged instructions:

$$\text{Sensitive} \subseteq \text{Privileged}.$$

A machine is *recursively virtualizable* if it is virtualizable and a VMM without timing dependencies can be built for it.

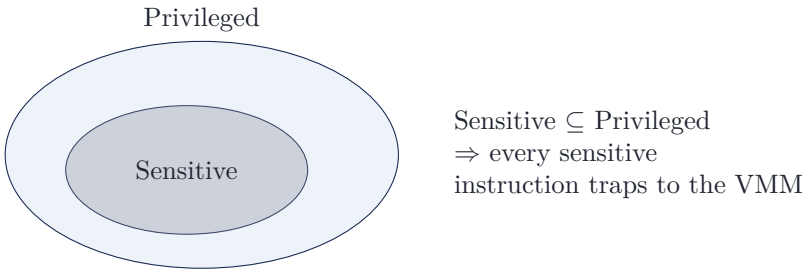


Figure 2.2: Virtualizable architecture: sensitive instructions lie inside the privileged set.

Solved Example: Is the instruction set virtualizable?

Given: Architecture A: privileged = {`HLT`, `LGDT`, `OUT`, `IN`, `LIDT`}, sensitive = {`LGDT`, `OUT`, `IN`}. Architecture B: same privileged set, sensitive = {`LGDT`, `OUT`, `POPF`} where `POPF` silently ignores flag changes in user mode.
Formula: VMM constructible iff Sensitive ⊆ Privileged.
Step-by-step substitution:

1. A: all three sensitive instructions appear in the privileged set, so subset holds.
2. B: `POPF` is sensitive but not privileged (does not trap), so the subset fails.

Final Answer: A is classically virtualizable; B is not (needs binary translation or hardware

assistance).

2.3.2 Approaches to Server Virtualization

	Full virtualization	Para-virtualization	Hardware-assisted
Generation	1st (binary rewriting, software based)	2nd (cooperative)	3rd (silicon based)
Guest OS	Unmodified; unaware of emulation	**Modified**; uses hypercalls/specialized API	Unmodified
Mechanism	Dynamic binary translation; all hardware including CPU emulated (QEMU, Bochs)	Guest runs kernel operations at ring 1; no privileged-instruction translation by VMM	CPU extensions (Intel VT-x, AMD-V) intercept privileged operations; guest at ring 0, VMM in a more privileged "ring −1"
Pros	Isolation; total VM portability across dissimilar hosts	Low overhead, close to native	Runs legacy unmodified OSs
Cons	Emulation performance cost; guest forced to ring 1/3 so VMM must trap and emulate	Requires modified OS or para-virtual drivers (recompiling kernel or installing drivers)	Speed/flexibility: unmodified OS cannot exploit virtualization features (partly solved by para-virtual drivers)

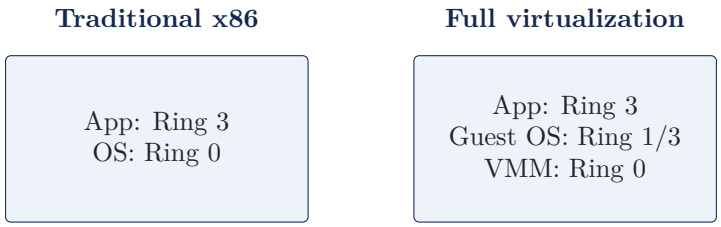


Figure 2.3: Privilege rings: full virtualization forces the guest kernel out of ring 0.

Solved Example: Para-virtualization recognition (Assignment 2, Q1, Q2)

Given: A guest OS must issue *hypercalls* instead of executing certain privileged instructions.
Formula: Hypercalls ⇔ modified guest OS ⇔ para-virtualization.
Step-by-step substitution:

1. Full / hardware-assisted virtualization runs an *unmodified* guest; privileged instructions handled by hypervisor and VT-x/AMD-V.

2. Hypercalls require OS modification, reducing overhead.

Final Answer: Para-virtualization (Q1: C). Full virtualization is characterised by an unmodified guest with CPU extensions and hypervisor (Q2: B).

2.3.3 Host OS, Guest OS and Hypervisor

The hypervisor creates and manages VMs; each guest OS runs inside a VM; for a Type-2 hypervisor the host OS is the OS on which the hypervisor runs (Assignment 2, Q5: B).

2.4 Network Virtualization

Network virtualization makes one physical network appear as multiple logical networks. **Motivation:** the Internet is "paralyzed" by makeshift fixes (overlays); a one-size-fits-all architecture is hard, so build an *all-sizes-fit-into-one* open, expandable architecture and a testbed for future protocols. **Related concepts:** VPNs (not customizable enough), active and programmable networks, overlay networks (not flexible enough).

- **Business model roles:** Infrastructure Providers (InPs) manage physical networks; Service Providers (SPs) create virtual networks and deploy end-to-end services; End Users buy services; Brokers mediate.
- **Design principles:** concurrence of multiple heterogeneous virtual networks; recursion; inheritance of architectural attributes; revisitation of virtual nodes.
- **Design goals:** flexibility; manageability (policy separated from mechanism); scalability; security, privacy and isolation; programmability; heterogeneity; experimental and deployment facility (PlanetLab, GENI, VINI); legacy support.

Definition: Network Virtualization

A networking environment that allows multiple service providers to dynamically compose multiple heterogeneous virtual networks that coexist in isolation, to deploy customized end-to-end services on the fly and manage them for end users, by effectively sharing underlying network resources leased from multiple infrastructure providers.

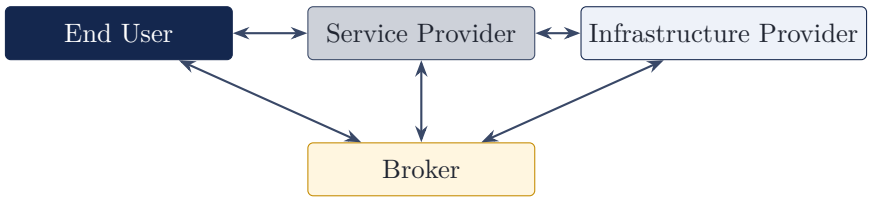


Figure 2.4: Business model of network virtualization.

2.5 XML: eXtensible Markup Language

XML is a syntax for encoding text-based data using printable Unicode characters. It is **extensible** (you define your own elements within the syntax rules) and a **universal format**: syntax rules ensure all processing software handles a given document identically. The data model is a **tree** of elements, attributes and text; it is self-describing, designed for distributed environments and can mix vocabularies.

```
<?xml version="1.0" encoding="utf-8"?>
<partorders>
  <order ref="x23-2112-2342" date="25aug1999-12:34:23h">
```

```
    <desc>Gold sprockel grommets</desc>
    <part number="23-23221-a12" />
    <quantity units="gross">12</quantity>
    <deliveryDate date="27aug1999-12:00h" />
  </order>
</partorders>
```

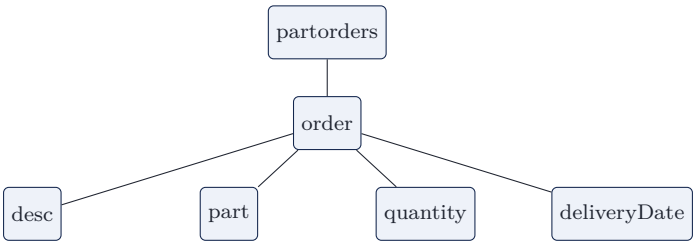


Figure 2.5: XML data model: a hierarchical tree.

2.5.1 Well-formedness, Entities and Parsers

- **Well-formed:** syntactically correct; the minimal requirement "to be" XML. A parser *must stop* (raise an exception) if data is not well-formed.
- **Internal entities** (declared in the DTD, e.g. `<!ENTITY messageHeader "...">`, referenced as `&messageHeader;`) and **external entities** (`SYSTEM "URL"`, shared among documents). References are replaced recursively.
- **Validating parser:** must retrieve all entities and process all DTD content, stopping on failure. **Non-validating parser:** tries to retrieve entities but stops DTD processing at the first missing one without error.
- **Characters:** use character references such as `♠` for characters outside the encoding; binary data must be encoded as printable characters.

Solved Example: Well-formed or not?

Given: Documents: (1) `<a><b>x</b></a>`; (2) `<a><b>x</a></b>`; (3) `<a x=1/>`; (4) `<a x="1"/>`.
Formula: Well-formed: single root, properly nested tags, all tags closed, attribute values quoted.
Step-by-step substitution:

1. (1) properly nested – well-formed.
2. (2) tags overlap – not well-formed.
3. (3) unquoted attribute value – not well-formed.
4. (4) quoted attribute, empty element closed – well-formed.

Final Answer: (1) and (4) are well-formed; (2) and (3) force a parser to halt.

2.5.2 DTD, XML Schema and Namespaces

Well-formed means syntactically correct; **valid** means well-formed and consistent with a DTD or Schema. A DTD specifies allowed element and attribute names, nesting rules and content restrictions.

```
<!DOCTYPE transfers [
  <!ELEMENT transfers (fundsTransfer)+>
  <!ELEMENT fundsTransfer (from, to)>
  <!ATTLIST fundsTransfer date CDATA #REQUIRED>
  <!ELEMENT from (amount, transitID?, accountID, acknowledgeReceipt)>
  <!ATTLIST from type (intrabank|internal|other) #REQUIRED>
  <!ELEMENT amount (#PCDATA)>
  <!ELEMENT to EMPTY>
  <!ATTLIST to account CDATA #REQUIRED>
]>
```

	DTD	XML Schema (2001)
Syntax	Special DTD grammar (part of core XML)	Pure XML
Types	Limited (CDATA, PCDATA)	Integers, dates, real numbers in a range, enumerations
Entities	Can declare entities	Cannot declare entities
Use	Basic structure validation	Type validation; relating database schemas to XML models

Namespaces identify different "spaces" for element names via the `xmlns` attribute (a URL string that need not point to anything), so `<key/>` as a security key differs from `<key/>` as a database key. They allow mixing dialects, e.g. XHTML with a `mt:` prefix for MathML.

2.5.3 XML Parser APIs and Processing

API	Model	Suitable when
SAX	Event-based, push, forward-only	Large documents, streaming, low memory
StAX	Event-based, pull, forward-only	Streaming with application control
DOM	Object/tree in memory; random access and modification	Moderately sized documents needing navigation and editing
JDOM / dom4j	Java-specific tree models	Java-friendly tree processing

XSLT (eXtensible Stylesheet Language Transformations) transforms XML into other XML, HTML or text using a stylesheet written in XML. **XML messaging** uses XML as the format for messages between systems, commonly over HTTP, with standard formats that "wrap" remote-procedure-call style data.

Solved Example: Choosing a parser

Given: An application repeatedly navigates, randomly accesses and modifies elements of a moderately sized XML document held in memory.
Formula: Random access plus modification ⇒ in-memory tree.
Step-by-step substitution:

1. SAX and StAX are forward-only streams: no random access or modification.
2. DOM builds the whole tree in memory: supports both.

Final Answer: **DOM**.

2.6 Web Services and Service-Oriented Architecture

Definition: Web Service

A software application identified by a URI whose interfaces and bindings can be defined, described and discovered as XML artifacts, supporting direct interactions with other applications using XML-based messages over Internet protocols.

History: structured programming → object-oriented programming → distributed computing (RPC, CORBA, DCOM, RMI) and Electronic Data Interchange (EDI) → web services. Web services build on OOP principles, provide EDI-like capability over the Internet and solve three problems: *interoperability*, *firewall traversal* (standard HTTP/HTTPS) and *complexity*.

2.6.1 SOA Roles and Operations

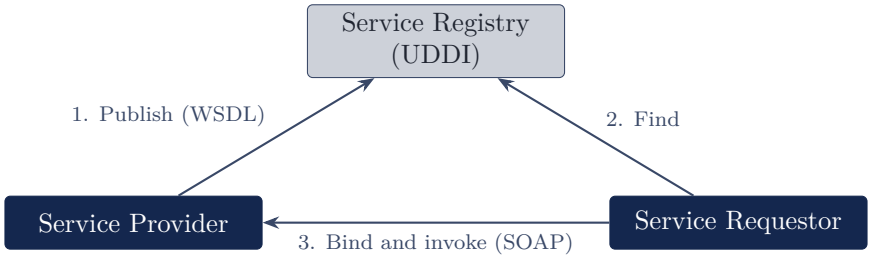


Figure 2.6: The publish–find–bind triangle of SOA.

Solved Example: SOA interactions

Given: Roles: service provider, registry, requestor.
Formula: Publish: provider → registry; Find: requestor → registry; Bind: requestor → provider.
Step-by-step substitution:

1. The provider describes the service in WSDL and publishes it to the registry.
2. The requestor queries the registry and finds a suitable service.
3. The requestor binds directly to the provider and invokes operations.

Final Answer: Publish, Find, Bind as stated.

2.6.2 The Web Service Stack

Layer	Role
XML	Uniform data format (derived from SGML); tags are user-defined; describes data, whereas HTML describes display
SOAP	Simple Object Access Protocol: message format over Internet protocols
WSDL	Web Service Description Language: operations, messages, binding
UDDI	Universal Description, Discovery and Integration: registry

SOAP building blocks: Envelope (required; identifies the document as SOAP), Header (optional), Body (required; call and response data), Fault (optional). Messages are request/response over HTTP (POST). SOAP is extensible, neutral (any transport) and independent (any programming model). Usage models: RPC-like exchange and document-style messaging. SOAP uses HTTP and can therefore use HTTPS for security; the WS-Security family adds message-level security.

```
POST /InStock HTTP/1.1
Host: www.stock.org
Content-Type: application/soap+xml; charset=utf-8

<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <m:GetStockPrice xmlns:m="http://www.stock.org/stock">
      <m:StockName>IBM</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

WSDL document structure: `types`, `message` (with `parts`), `portType` (operations), `binding` and `service`. Example fragment: a message `getTermRequest` with part `term` of type `xs:string`, and a response message `getTermResponse`.

UDDI: a framework to describe services, discover businesses and integrate business services; provides a service registry supporting publish and find. Benefits: discovering the right business from millions online, defining how services interact, and describing services and business processes programmatically. The WS-* family (WS-Security, WS-Policy, WS-Trust, WS-Privacy, WS-SecureConversation, WS-Federation) extends SOAP.

Solved Example: What do traditional web services solve? (Assignment 2, Q7–Q9)

Given: Candidate capabilities: HTTP/HTTPS firewall passage, cross-language interoperability, ultra-low-latency real-time communication, UDDI discovery.
Formula: XML/SOAP overhead ⇒ no ultra-low latency; WSDL names the interface; UDDI discovers.
Step-by-step substitution:

1. HTTP/HTTPS: addressed. XML/SOAP/WSDL interoperability: addressed. UDDI discovery: addressed.

2. XML parsing and SOAP overhead add delay, so microsecond-level real-time messaging is *not* addressed.

3. The interface (operations, messages, binding) is described by **WSDL**; UDDI only registers and discovers.

Final Answer: Items (i), (ii), (iv) addressed, (iii) not (Q9: B); WSDL describes the interface (Q7: B); SOAP provides (i), (ii), (iii) (Q8: A).

2.6.3 Web Service Example: Web-based Purchase

A purchase order service calls invoice and consolidation services; the workflow is composed through SOAP messages discovered via a registry, illustrating loose coupling and platform independence.

2.7 Week 2 Assignment Review

Q	Concept tested	Answer
1	Hypercalls ⇒ para-virtualization	C
2	Full (hardware-assisted): unmodified guest, CPU extensions	B
3	Public cloud: open use, owned by provider	B
4	DOM for random access and modification	C
5	Hypervisor / guest OS / host OS roles	B
6	Hosted private cloud: SLA, compliance, latency all matter	D
7	WSDL describes the web service interface	B
8	SOAP: platform independence, XML, UDDI discovery	A
9	Web services do not address ultra-low latency	B
10	SOA publish–find–bind	A

SLAs, Cloud Economics, Data Management, MapReduce and OpenStack

3.1 Service Level Agreements (SLA)

Definition: Service Level Agreement

A formal contract between a service provider and a consumer that specifies the services to be delivered, the measurable level of service (Service Level Objectives), how it is monitored, and the remedies or penalties when objectives are missed.

SLA contents: the set of services the provider will deliver; a complete, specific definition of each service; responsibilities of provider and consumer; the efficiency/quality metrics; how performance is monitored, audited and reported; the procedure for dispute resolution; and the consequences (credits, penalties) of non-compliance, including how the SLA is changed or terminated.

Web-service SLA: *WS-Agreement* is an XML-based language and protocol for negotiating, establishing and managing agreements. **Cloud SLA versus web-service SLA:** traditional QoS parameters (response time, throughput) are supplemented in the cloud by availability, resource capacity, scalability, elasticity and provisioning-time parameters.

Types of SLA: *off-the-shelf* (non-negotiable, standard terms of the provider) and *negotiated* (customised). **Multilevel SLA:** corporate level (applies to all customers), customer level (specific to a customer) and service level (specific service).

Service Level Objectives (SLOs) are objectively measurable conditions for the service (for example availability $\geq$ 99.9%, response time below 200 ms). **Service-level management** means monitoring and measuring service performance against SLOs.

Considerations for an SLA: business-level objectives; responsibilities of provider and consumer; business continuity and disaster recovery; maintenance; data location and portability; security controls; regulatory compliance; and exit (termination) terms.

Key Performance Indicators (KPIs): low-level resource metrics (CPU, memory, bandwidth) are combined into higher-level indicators; the questions are who should monitor (consumer, provider or third party) and how often. **Metrics for monitoring and auditing:** throughput (how quickly the service responds), reliability, load balancing, durability, elasticity, linearity, agility (how quickly the provider responds to change) and automation, performance and cost metrics.

Expected SLA parameters by model: *IaaS* – CPU capacity, memory, storage, network, availability, provisioning time; *PaaS* – integration, scalability, pay-as-you-go billing, environment; *SaaS* – reliability, usability, scalability, availability, customizability.

Limitations: service measurement is restricted to uptime percentage; measurement is often the customer's burden; there is a gap between QoS hype and the SLA offerings in reality; credits are small compared with business loss.

3.1.1 Availability and Downtime

Key Result: Allowed downtime

$$\text{Allowed downtime} = (1 - A) \times T_{\text{service}}, \qquad A_{\text{achieved}} = \frac{T_{\text{service}} - T_{\text{outage}}}{T_{\text{service}}}.$$

T_{service} is the agreed *scheduled* service time, which may be less than 24 hours a day.

Solved Example: 99.9% availability per 30-day month

Given: $A = 99.9\%$, month $= 30 \times 24 = 720$ hours.
Formula: Allowed downtime $= (1 - A) \times T$.
Step-by-step substitution:

1. $1 - 0.999 = 0.001$.
2. $0.001 \times 720 = 0.72$ hours.
3. $0.72 \times 60 = 43.2$ minutes.

Final Answer: About **0.72 hours** (≈ 43 minutes).

Solved Example: Multilevel SLA conflict (Assignment 3, Q2, Q3)

Given: Corporate level 95%, customer level 98%, service level 99.9% uptime.
Formula: The most specific, most granular level governs accountability for the actual service.
Step-by-step substitution:

1. A multilevel SLA consists of all three levels: corporate, customer and service.
2. In a conflict, the service-level SLA (99.9%) is the binding, most specific commitment.

Final Answer: Service-level SLA (Q3: C); a multilevel SLA includes (i), (ii) and (iii) (Q2: D).

Note: What belongs in an SLA? (Assignment 3, Q1, Q4)

RTO, MTTR, penalty/credit calculation and availability percentage are SLA items. The exact encryption algorithm and key-storage procedure belong to security documents, not the SLA. Reliability and recoverability are quantified by **availability** and **Mean Time To Recovery (MTTR)**.

3.2 Cloud Economics

Economic properties of the cloud:

- **Common infrastructure:** pooled standardized resources with benefits from statistical multiplexing.
- **Location independence:** ubiquitous availability with latency reduction and better user experience.
- **Online connectivity:** the enabler of other attributes.
- **Utility pricing:** pay-per-use, beneficial where demand varies.
- **On-demand resources:** elastic provisioning without delay or change cost.

3.2.1 Value of Common Infrastructure: Statistics of Scale

Multiplexing demand raises utilization and lowers cost per delivered resource. Smoothness is measured by the **coefficient of variation**

$$C_v = \frac{\sigma}{|\mu|}.$$

Key Result: Aggregating independent demands

For n independent demands $X_1, \ldots, X_n$ with identical mean μ and standard deviation σ: mean $= n\mu$, variance $= n\sigma^2$, so

$$C_v^{\text{agg}} = \frac{\sqrt{n}\,\sigma}{n\mu} = \frac{C_v}{\sqrt{n}}.$$

Perfectly correlated demands (nX) leave C_v unchanged; negatively correlated demands (X and $1 - X$) smooth the sum to a constant.

Solved Example: Smoothing by aggregation

Given: 100 independent workloads, each with $\mu = 40$ units and $\sigma = 20$ units.
Formula: $C_v = \sigma/\mu$; $C_v^{\text{agg}} = C_v/\sqrt{n}$.
Step-by-step substitution:

1. Single workload: $C_v = 20/40 = 0.5$.
2. Aggregate: $C_v^{\text{agg}} = 0.5/\sqrt{100} = 0.5/10 = 0.05$.
3. Check: mean $= 4000$, std $= \sqrt{100} \times 20 = 200$, $200/4000 = 0.05$.

Final Answer: C_v falls from 0.5 to 0.05: aggregating 100 workloads reduces the penalty to 10%.

3.2.2 Value of Location Independence

Latency depends strongly on distance and (second order) on routing; light in fibre covers only about 124 miles per millisecond; human response latency is tens to hundreds of ms; VoIP with latency above about 200 ms degrades. A global user base needs a dispersed architecture, raising coordination, consistency, availability and partition-tolerance issues.

3.2.3 Value of Utility Pricing

Cloud services need not be cheaper per unit to be economical: renting a car at INR 45,000 per day versus owning at INR 10,000 per day is cheaper if you need it for only two days. Let $D(t)$ be demand for $0 < t < T$, $P = \max D(t)$ the peak, $A = \operatorname{avg} D(t)$ the average, B the baseline (owned) unit cost, C the cloud unit cost, and $U = C/B$ the *utility premium*.

$$C_T = U\,B \int_0^T D(t)\,dt = A\,U\,B\,T, \qquad B_T = P\,B\,T \quad \text{(baseline must handle peak)}.$$

Key Result: When is the cloud cheaper?

$$C_T < B_T \iff U < \frac{P}{A}.$$

The cloud is cheaper when the utility premium is less than the ratio of peak to average demand.

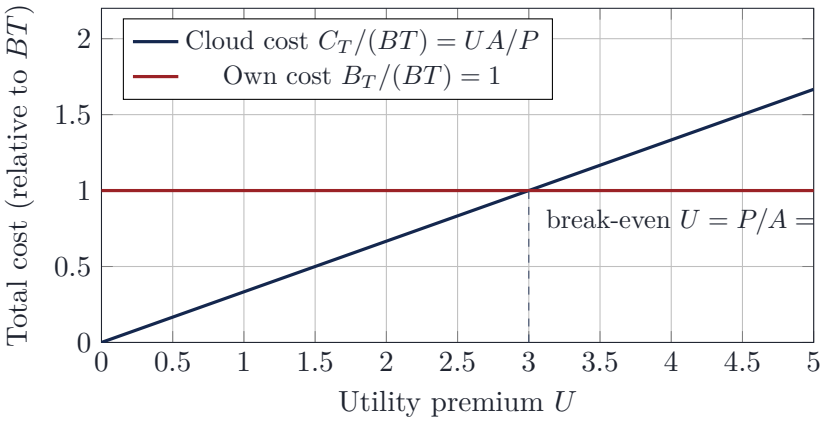


Figure 3.1: Cloud versus owned cost for $P/A = 3$: break-even at $U = 3$.

Solved Example: Own or rent?

Given: Average demand $A = 75$, peak demand $P = 225$, utility premium $U = 2.8$.
Formula: Cloud cheaper iff $U < P/A$.
Step-by-step substitution:

1. $P/A = 225/75 = 3$.

2. $U = 2.8 < 3$.

Final Answer: Cloud cost < owning cost.

Real-world note: demand is often spiky (promotions, tax season, flash crowds); a hybrid model is often best; network cost, interoperability overhead, reliability and accessibility must also be considered.

3.2.4 Value of On-demand Services and Penalty Cost

Owned resources incur a penalty whenever resources $R(t)$ do not match demand $D(t)$: Penalty $\propto \int |D(t) - R(t)|\,dt$. If demand is flat, the penalty is zero; for linear demand periodic provisioning

is acceptable; for exponential demand $D(t) = e^t$ with provisioning delay t_p, $R(t) = e^{t-t_p}$ and $D - R = e^t(1 - e^{-t_p}) = k_1 e^t$, so the penalty grows exponentially.

Solved Example: Penalty with provisioning delay (tutorial Assignment 1)

Given: Peak demand 120 units; $D(t) = 50 \sin t$ for $0 \le t < \pi/2$ and $20 \sin t$ for $\pi/2 \le t < \pi$; $R(t) = D(t) + \delta \dfrac{dD}{dt}$ with $\delta = \pi/12$; unit cost 0.9.
Formula: Penalty $= 0.9 \int |D(t) - R(t)|\, dt = 0.9\, \delta \int |D'(t)|\, dt$.
Step-by-step substitution:

1. $D - R = -\delta D'(t)$, hence $|D - R| = \delta |D'(t)|$.
2. On $[0, \pi/2)$: $D' = 50 \cos t$, $\int_0^{\pi/2} 50 \cos t\, dt = 50$.
3. On $[\pi/2, \pi)$: $|D'| = 20|\cos t|$, $\int_{\pi/2}^{\pi} 20|\cos t|\, dt = 20$.
4. Total $\int |D'|\, dt = 70$ (the jump at $t = \pi/2$ is treated as a single instant and ignored).
5. Penalty $= 0.9 \times \dfrac{\pi}{12} \times 70 = 5.25\pi$.

Final Answer: Penalty ≈ 16.5 cost units; the provisioning delay makes the cloud under-supply on the rising part of demand and over-supply on the falling part.

Solved Example: Cloud versus owned for decaying demand (tutorial Assignment 2)

Given: $D(t) = 50(1 + e^{-t})$, peak $P = 100$ (at $t = 0$), baseline unit cost $B = 120$, cloud unit cost $C = 200$, period $T = 100$.
Formula: $B_T = P\, B\, T$; $C_T = C \int_0^T D(t)\, dt$.
Step-by-step substitution:

1. $B_T = 100 \times 120 \times 100 = 1{,}200{,}000$.
2. $\int_0^{100} 50(1 + e^{-t})\, dt = 50\,[\,100 + (1 - e^{-100})\,] \approx 5050$.
3. $C_T = 200 \times 5050 = 1{,}010{,}000$.
4. Cross-check: $A \approx 50.5$, $U = 200/120 = 1.67 < P/A = 100/50.5 = 1.98$.

Final Answer: Yes, cloud cost 1.01×10^6 < own cost 1.2×10^6.

3.3 Managing Data in the Cloud

3.3.1 Relational and Parallel Databases

Relational databases are the default storage and retrieval mechanism, accessed through SQL by applications; the database file-system layer provides indexing and transaction support. **Row-oriented storage** is optimal for write-intensive transactional workloads; **column-oriented storage** is optimal for read-heavy analytics (only needed columns are read, and similar values compress well).

Architecture	Description	Suitable for
Shared memory	CPUs share one address space managed by an SMP operating system	Servers with multiple CPUs
Shared disk	Independent servers share storage over a high-speed network (NAS or SAN; Ethernet, Fibre Channel or Infiniband)	Hybrid, moderate scale
Shared nothing	Cluster of independent servers each with own disk, connected by a network; tables partitioned across nodes	Massively parallel, large scale

Advantages of a parallel DB over a relational DB: efficient SQL execution on multiple processors; partitioned tables with a distributed-join optimizer; distributed two-phase commit locking; fault tolerance via stand-by systems. Examples: Oracle, DB2, SQL Server (transaction processing); Netezza, Vertica, Teradata (data warehousing). (Assignment 3, Q6: parallel DBs speed up analytics by *distributed query execution across nodes.*)

3.3.2 Google File System and HDFS

GFS manages large files on a very large cluster of commodity servers. Failures must be tolerated because for N components $p(\text{system failure}) = 1 - (1-p)^N \to 1$ for large N.

- A single **Master** (GFS) / **NameNode** (HDFS) controls the namespace and metadata.
- Files are split into **chunks** (GFS) / **blocks** (HDFS) of typically 64 MB, stored on **chunk servers** / **data nodes**, replicated three times on different racks and network segments.
- **Read:** client sends path and offset to the master, gets replica metadata, caches it and reads from a chunk server.
- **Write/append:** master returns metadata for all replicas; client pushes data to all; the master designates a *primary*, which chooses the offset, applies the append and tells replicas; on any failure the primary retries.
- **Fault tolerance:** heartbeat messages; metadata updated on failure; a new primary is assigned if the primary fails.

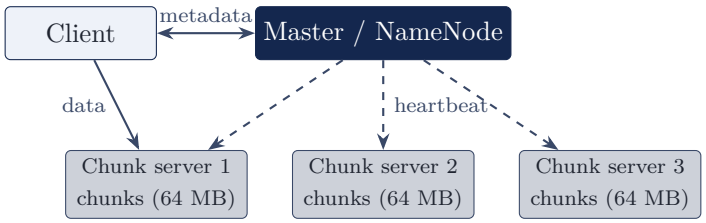


Figure 3.2: GFS/HDFS architecture: control path through the master, data path directly to chunk servers.

Solved Example: Chunks and failure probability

Given: A 1 GB file, chunk size 64 MB, replication factor 3; component failure probability $p = 0.001$ in a system of $N = 1000$ components.
Formula: Chunks $= \lceil \text{size/chunk size} \rceil$; $p_{\text{sys}} = 1 - (1-p)^N$.
Step-by-step substitution:

1. $1024/64 = 16$ chunks; replicas $= 16 \times 3 = 48$ chunk copies.

2. $p_{\text{sys}} = 1 - 0.999^{1000} = 1 - 0.3677 = 0.632$.

Final Answer: 16 chunks (48 copies stored); with 1000 components some failure is likely ($\approx 63\%$), so fault tolerance is mandatory.

3.3.3 BigTable

A distributed structured storage system built on GFS: a **sparse, persistent, multi-dimensional sorted map** indexed by *row key, column key and timestamp*. Columns are grouped into *column families* (fixed at table creation, with labels created dynamically as `family:label`); each cell keeps multiple versions in decreasing timestamp order. Tables split into row ranges called **tablets**, each managed by a **tablet server**, stored as *SSTable* files per column family; a metadata table (with a root tablet) locates tablets. Supports large parallel reads and inserts; inserts are sorted and need more work than a simple append.

3.3.4 Dynamo

Developed by Amazon for large volumes of concurrent small updates (key–value model, no dependence on GFS/HDFS), suited to e-commerce.

- Objects are ⟨key, value⟩ pairs; **MD5** yields a 128-bit hash whose range maps to *virtual nodes on a ring*; each physical node holds several virtual nodes. Objects replicate to N nodes.
- Writes execute at a virtual node and are forwarded to replicas; load balancing handles transient failures and partitions.
- **Versioning with vector timestamps**: superseded versions are discarded; concurrent writes keep all versions, returned to readers for conflict resolution.
- **Quorum:** a read touches R replicas, a write touches W; if $R+W > N$ the system is quorum consistent.

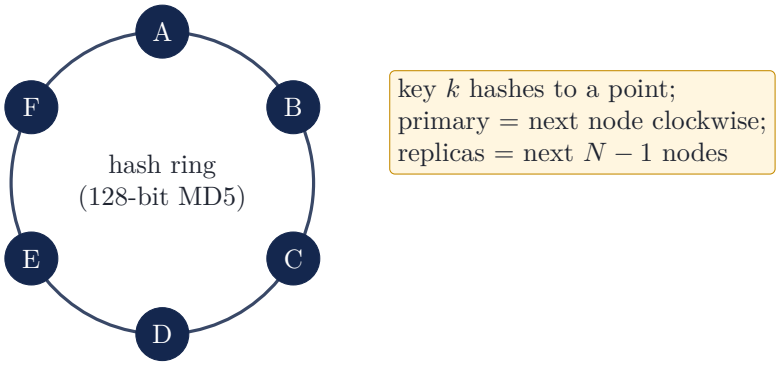


Figure 3.3: Dynamo consistent-hashing ring with virtual nodes.

Solved Example: Quorum consistency

Given: $N = 3$ replicas. Configurations: (a) $R = 1, W = 1$; (b) $R = 2, W = 2$; (c) $R = 1, W = 3$.
Formula: Quorum consistent iff $R + W > N$.
Step-by-step substitution:

1. (a) $1 + 1 = 2 \not> 3$: not consistent.
2. (b) $2 + 2 = 4 > 3$: consistent.
3. (c) $1 + 3 = 4 > 3$: consistent; reads are fast, writes slow.

Final Answer: (b) and (c). Raising W favours reads; raising R favours writes.

3.3.5 Datastore (App Engine) and SimpleDB

All entities reside in one BigTable table (single column family, no column-oriented storage), sharded and sorted lexicographically by key. Entities are grouped by ancestry for transactions; *prefix and range queries* are efficient. Index tables support queries: automatically created **single-property** and **kind** indexes, and configurable **composite** indexes; the index with highest selectivity is chosen.

3.4 Introduction to MapReduce

MapReduce is the programming model developed at Google for large-scale text processing and search on data in BigTable/GFS; it is fault tolerant and runs on tens of thousands of processors. *Hadoop* (Yahoo!) is the open-source implementation.

3.4.1 Parallel Efficiency

$$\varepsilon = \frac{T_1}{p \cdot T_p}$$

where T_1 is the uniprocessor time and T_p the time on p processors. Inefficiencies arise from synchronization, message overhead and load imbalance. An implementation is *scalable* if efficiency stays constant when data and processors grow proportionally.

Illustration (word frequencies). With n documents, m distinct words, each word occurring f times per document on average, read cost c per word:

- Single processor: $n\,m\,f\,c$.
- *Approach 1* (each processor handles m/p words): every processor reads all words of all documents, $n\,(m/p)\,f$ useful reads but scans everything $\Rightarrow$ efficiency falls with p (not scalable).
- *Approach 2* (each processor handles n/p documents for all m words): reads $(n/p)\,m\,f\,c$, writes partial vector $c\,m$, merges $p\,(m/p)\,c$; every read is useful, efficiency tends to 1 as n grows ($p \ll nf$).

3.4.2 The Model

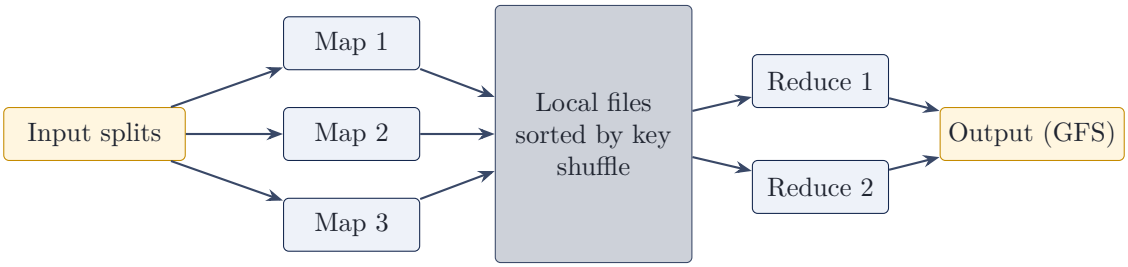


Figure 3.4: MapReduce data flow with 3 mappers and 2 reducers.

Map phase: each of M mappers reads about $1/M$ of the input; the map function transforms key-value pairs $(k_1, v_1) \to [(k_2, v_2)]$; each mapper writes one file per reducer, sorted by key, to its *local* file system, and the master tracks locations. **Reduce phase:** the master tells reducers where the partial files are; reducers fetch them by remote procedure call, group values by key, apply the reduce function $(k_2, [v_2]) \to$ output, and write results to GFS. **Fault tolerance:** heartbeats; tasks with no progress are duplicated; a failed *mapper* must be re-executed because its output is on local disk; a failed *reducer* needs only its unfinished tasks re-run since completed output is already in GFS.

```
map(String docId, String text):
    for each word w in text:
        EmitIntermediate(w, 1)

reduce(String word, Iterator counts):
    total = 0
    for each c in counts:
        total += c
    Emit(word, total)
```

3.4.3 MapReduce Efficiency

For a task on data volume D with per-word cost c: uniprocessor time $wD = cD + c\,m\,D + c\,r\,\sigma D + c\,\sigma\mu D$ (read, map, reduce, write). With P processors the extra overhead is each mapper writing $\sigma D/P$ locally and each reducer reading it remotely, so the overhead is $\dfrac{2c\sigma D}{P}$ in addition to $\dfrac{wD}{P}$.

$$\varepsilon_{MR} = \frac{wD}{P\left(\dfrac{wD}{P} + \dfrac{2c\sigma D}{P}\right)} = \frac{1}{1 + \dfrac{2c\sigma}{w}}.$$

Solved Example: MapReduce parallel efficiency

Given: $w = c(1 + m + r\sigma + \sigma\mu)$ with $m = 1$, $r = 1$, $\sigma = 1$, $\mu = 0.1$ (so $w = 3.1c$).
Formula: $\varepsilon_{MR} = 1/(1 + 2c\sigma/w)$.
Step-by-step substitution:

1. $2c\sigma/w = 2c/(3.1c) = 0.645$.

2. $\varepsilon_{MR} = 1/1.645 = 0.608$.

Final Answer: About 61% efficiency; efficiency is independent of P and D, so the implementation is scalable.

Applications: indexing a document collection (map emits (w_i, d_k); reduce builds $(w_i, [d_{i1}, \ldots, d_{im}])$); relational operations (joins, group-by) at large scale with fault tolerance; word count.

Solved Example: Combiner and shuffle volume (Assignment 3, Q9; Assignment 5, Q4, Q6, Q7)

Given: Word count of 2 TB with mapper output 1.5 TB, reducers run out of disk; HDFS block size 128 MB, input file 700 MB.
Formula: Number of map tasks = $\lceil$file size/block size$\rceil$.
Step-by-step substitution:

1. A **combiner** pre-aggregates mapper output locally (summing counts per word), shrinking 1.5 TB drastically before the shuffle; without it everything reaches the reducers and the merge exhausts disk space. Fix: configure the combiner.
2. $\lceil 700/128 \rceil = \lceil 5.47 \rceil = 6$ blocks $\Rightarrow$ 6 map tasks.
3. The **Partitioner** decides which reducer receives a key (hash-based by default).
4. The Combiner is optional and may run zero, one or many times, so the algorithm must stay correct without it.

Final Answer: Combiner (B); 6 map tasks (B); Partitioner (B); Combiner (C).

Solved Example: Blocks, average and grouped salary (tutorial problems)

Given: Block size 64 MB; files of 64 KB, 65 MB, 127 MB. Set $A = (10, 20, 30, 40, 50)$.
Formula: Blocks per file = $\lceil$size/64 MB$\rceil$. Average = $\sum x/n$ computed with pairs (sum, count).
Step-by-step substitution:

1. 64 KB $\rightarrow$ 1 block; 65 MB $\rightarrow \lceil 1.02 \rceil = 2$; 127 MB $\rightarrow \lceil 1.98 \rceil = 2$. Total = 5 blocks.
2. Mapper emits ("avg", $(x, 1)$) for each integer; combiner/reducer adds component-wise: sum = 150, count = 5.
3. Reducer outputs $150/5 = 30$.

Final Answer: 5 blocks; average = 30. (Note: averaging the *averages* of partial results is incorrect; carry (sum, count).)

```
// Salary statistics grouped by gender
map(line):                          // "Name, Gender, Salary"
    (name, g, s) = parse(line)
    Emit(g, (s, 1))
reduce(gender, values):
    total = 0; n = 0
    for (s, c) in values: total += s; n += c
    Emit(gender, (total, total / n))

// Word length categorisation
map(word):
    L = length(word)
    if L <= 2: Emit("tiny", 1)
    else if L <= 5: Emit("small", 1)
    else if L <= 9: Emit("medium", 1)
```

```
    else: Emit("big", 1)
reduce(category, counts): Emit(category, sum(counts))
```

3.5 OpenStack

OpenStack is a cloud operating system controlling large pools of compute, storage and networking resources in a data centre, managed through a dashboard. Origin: a collaboration between NASA and Rackspace. It offers IaaS (VMs on demand, provisioning, snapshotting, networking).

Service	Project	Function
Compute	Nova	Manages VM lifecycle on hypervisors (scheduling, spawning)
Networking	Neutron	Virtual networks, routers, DHCP, security groups, floating IPs; routes tenant traffic
Object storage	Swift	Scalable, redundant object (blob) store
Block storage	Cinder	Persistent block volumes independent of VM lifecycle
Identity	Keystone	Authentication, authorization, service catalog, tokens
Image service	Glance	Stores and retrieves VM disk images
Telemetry	Ceilometer	Metering and monitoring for billing
Dashboard	Horizon	Web UI for users and administrators

Provisioning flow: (1) the user logs in via Horizon and specifies VM parameters (name, flavor, keys, image); (2) Keystone authenticates and returns an auth token reused for later API calls; (3) the Nova API publishes a request to the scheduler (rpc cast); the *Nova scheduler* filters and weighs hosts; (4) Nova compute driver talks to the hypervisor; Glance supplies the image, Neutron allocates network ports and Cinder attaches volumes. **Ephemeral storage** persists only until the VM terminates; Cinder volumes persist beyond it.

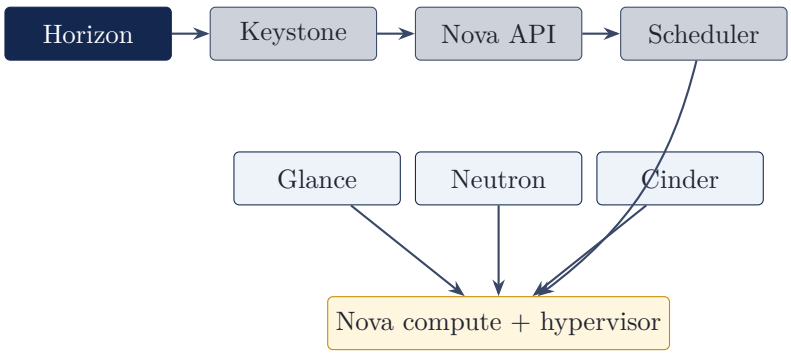


Figure 3.5: Simplified OpenStack VM provisioning flow.

Solved Example: Over-subscription

Given: A compute node with 640 physical vCPUs and a CPU allocation ratio of 5:1.
Formula: Allocatable vCPUs = physical vCPUs × allocation ratio.
Step-by-step substitution:

1. $640 \times 5 = 3200$.

Final Answer: 3200 virtual CPUs.

3.6 Week 3 Assignment Review

Q	Concept tested	Answer
1	Items not in an SLA: encryption algorithm details	B
2	Multilevel SLA: corporate, customer, service	D
3	Service-level SLA binds in conflict	C
4	Availability and MTTR	A
5	99.9% of 720 h $\Rightarrow$ 0.72 h	A
6	Parallel DB: distributed query execution	B
7	Sub-second fraud detection: nightly batch is least suitable	C
8	Petabyte imagery for Spark: object storage	B
9	Missing combiner exhausts reducer disk	B
10	$640 \times 5 = 3200$ vCPUs	B

Private Cloud with OpenStack, Microsoft Azure, and Google Cloud Platform

4.1 Introduction and Week Overview

Week 4 bridges the theoretical foundations of virtualization and service-oriented architectures into concrete, production-grade cloud implementations. The curriculum covers three distinct cloud ecosystems:

1. **Private Cloud Implementation via OpenStack:** An empirical examination of IIT Kharagpur's on-premises cloud infrastructure (*Meghamala*), exploring control plane components, hypervisor virtualization, and virtual machine lifecycle orchestration.
2. **Microsoft Azure Ecosystem:** Enterprise infrastructure components, geographic resilience architectures (Regions and Availability Zones), Azure App Service deployment topologies, deployment slot mechanics, and scalable database decoupling (Azure SQL Database Hyperscale).
3. **Google Cloud Platform (GCP):** Google's global network fabric, Platform-as-a-Service orchestration with Google App Engine (GAE), managed storage services, globally scalable relational consistency via Cloud Spanner, and real-time big data analytical pipelines.

4.2 Private Cloud Implementation: Meghamala at IIT Kharagpur

4.2.1 Architectural Blueprint and Infrastructure Topology

Meghamala (meaning "Garland of Clouds" in Sanskrit) is the dedicated private cloud computing facility at the Department of Computer Science and Engineering, IIT Kharagpur. Built on the open-source OpenStack platform, Meghamala serves institutional researchers, laboratory computing classes, and faculty development clusters.

The underlying infrastructure consists of high-density commodity server racks partitioned into:

- **Controller Nodes:** Running the OpenStack API daemons, message queue brokers (RabbitMQ), relational database instances (MariaDB/MySQL), and state coordination engines.
- **Network Nodes:** Executing OpenStack Neutron agents, virtual software switches (Open vSwitch / Linux Bridge), L3 routing daemons, DHCP agents, and metadata proxies.

- **Compute Nodes:** Running standard hypervisors (predominantly Linux KVM managed by `libvirt`) orchestrated by the `nova-compute` daemon, executing multi-tenant guest operating systems.
- **Storage Nodes:** Dedicated storage appliances hosting block storage pools (Cinder volumes backed by LVM, Ceph, or iSCSI SAN) and object storage clusters (Swift).

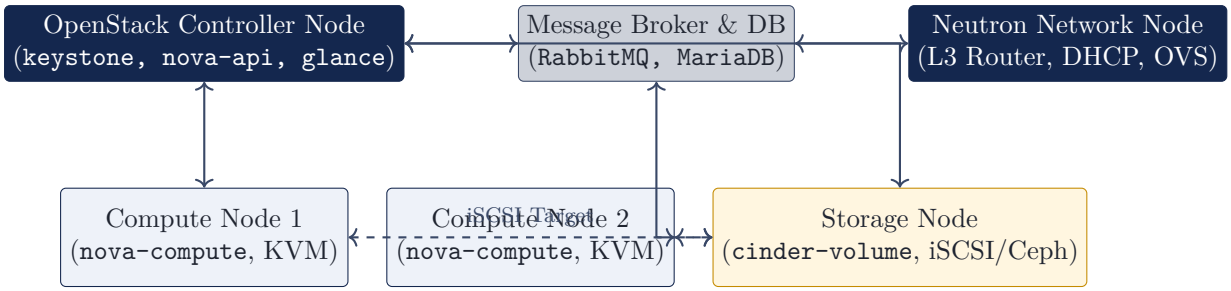


Figure 4.1: Meghamala OpenStack Physical and Logical Architecture.

4.2.2 Horizon Web Dashboard: Operational Modules

The primary administrative interface demonstrated in the lecture is OpenStack **Horizon**. The dashboard organizes cloud administrative actions into modular panels:

Horizon Module	Corresponding Service	Demonstrated Operational Capabilities
Project Overview	Horizon / Ceilometer	Graphical quota visualization (vCPUs allocated, RAM utilized, Floating IPs reserved, Security Group quotas).
Instances	Nova	Tabular listing of active, paused, or suspended VMs; display of hypervisor host, power state, task status, fixed IP (e.g., `10.4.0.x`), and public floating IP (e.g., `192.164.0.x`).
Flavors	Nova	Hardware profile templates defining virtual machine dimensions (e.g., `m1.tiny`: 1 vCPU, 512 MB RAM, 1 GB disk; `m1.large`: 4 vCPU, 8 GB RAM, 80 GB disk).
Images	Glance	Catalog of curated OS disk templates (Ubuntu Server, CentOS, Debian, CirrOS) uploaded in QCOW2 or RAW formats.
Volumes	Cinder	Lifecycle management of persistent block storage volumes; snapshot creation, volume expansion, and volume attachment to active Nova instances.
Networks & Routers	Neutron	Software-defined virtual tenant networks (VXLAN/GRE overlays), internal subnet allocation, default gateway assignment, external network routing, and floating IP allocation pools.
Security Groups	Neutron	Stateful virtual firewalls operating at the port level; protocol, port range, and ingress/egress CIDR rule definition.
Access & Security	Keystone / Nova	SSH key-pair generation, API access endpoints, and user credential tokens.

4.2.3 Step-by-Step VM Provisioning and Access Workflow

The demonstration shows the complete lifecycle of provisioning a virtual compute instance (named `cloud-nptel`) and establishing secure remote connectivity:

1. **Authentication and Scoping:** The user accesses Horizon via Web browser. Credentials (username and password) are sent to **Keystone**, which validates the user against the SQL identity backend and returns an unscoped token, followed by a scoped token associated with the specific tenant/project.
2. **Instance Parameter Specification:**
   - *Instance Name:* `cloud-nptel`.
   - *Source:* Boot from Glance image (e.g., Ubuntu Linux LTS `.qcow2`).
   - *Flavor Selection:* Selection of computing capacity matching computational requirements.

- *Network Attachment:* Binding the virtual network interface card (vNIC) to the tenant's private subnet (`10.4.0.0/24`).
- *Security Group Assignment:* Enforcing ingress security rules: Port 22 (SSH) and custom remote desktop ports.
- *Key Pair Association:* Injecting the user's public SSH key into the VM's `authorized_keys` file via `cloud-init` during boot.

3. **Compute Node Scheduling:** The `nova-scheduler` applies filtering algorithms (checking available vCPUs, memory, and hypervisor architecture) and weighting algorithms to select the optimal physical compute node.

4. **Image Caching and Hypervisor Execution:** The local `nova-compute` daemon fetches the base image from Glance, caches it locally, constructs an ephemeral copy-on-write overlay, writes the domain XML definition for `libvirt`, and boots the KVM guest.

5. **Floating IP Association:** The instance initially receives an internal, non-routable private IP (`10.4.0.4`). To make the instance accessible from campus desktop workstations, a public floating IP (`192.164.0.8`) is allocated from the external pool and mapped via 1:1 NAT in the Neutron virtual router.

6. **Remote Client Connectivity:**

   - *Command Line Access:* Direct terminal connection via secure shell:

     ```
     ssh -i ~/.ssh/meghamala_key.pem ubuntu@192.164.0.8
     ```

   - *Graphical Desktop Access via X2Go:* X2Go is an open-source remote desktop client based on NX technology, allowing high-performance graphical desktop access over low-bandwidth wide-area connections. The user configures an X2Go session targeting host `192.164.0.8`, specifying user `ubuntu`, RSA private key authentication, and session environment (e.g., XFCE or MATE).

7. **Instance Termination and Resource Reclamation:** When work completes, the user terminates the instance via Horizon. Nova signals `libvirt` to shut down the KVM process, deletes the ephemeral root disk, reclaims allocated RAM and vCPUs, and releases the private IP back to the Neutron DHCP pool. Associated Cinder volumes remain intact unless explicitly flagged for deletion.

Definition: Floating IP vs Fixed Private IP in OpenStack

Fixed IP: An internal, tenant-private IP address automatically assigned by Neutron DHCP upon VM creation (e.g., `10.4.0.x`). It enables communication between instances within the same virtual network but is unreachable from outside the cloud.
Floating IP: A public, globally routable IP address (e.g., `192.164.0.x`) drawn from an external network allocation pool. It is dynamically bound to an instance's virtual interface via Network Address Translation (1:1 NAT) on the virtual router, enabling direct inbound and outbound WAN communication.

4.3 Microsoft Azure Enterprise Architecture

4.3.1 Global Infrastructure: Regions and Availability Zones

Microsoft Azure organizes physical infrastructure hierarchically to provide enterprise-grade reliability and legal compliance:

- **Data Center:** A physical building housing server racks, uninterruptible power supplies (UPS), cooling infrastructure, and network switches.
- **Availability Zone (AZ):** A physically separated location within a single Azure region. Each Availability Zone comprises one or more data centers equipped with independent power, cooling, and networking infrastructure. AZs are connected through high-speed, ultra-low-latency private fiber networks (< 2 ms round-trip latency).
- **Region:** A defined geographical perimeter containing multiple Availability Zones (typically three or more). Examples include *Central India*, *East US*, and *West Europe*.
- **Region Pairs:** Each Azure region is paired with another region within the same geography (at least 300 miles apart) to ensure business continuity during catastrophic disasters (e.g., hurricanes or regional power outages). Planned Azure platform updates are rolled out to paired regions sequentially to prevent concurrent platform downtime.

Key Result: Availability Zone Fault Isolation Principle

If two Virtual Machines or database instances are deployed across separate Availability Zones within the same Azure Region:

$$P(\text{Simultaneous Hardware Failure}) = P(\text{AZ}_1 \text{ Failure}) \times P(\text{AZ}_2 \text{ Failure}) \approx 0 \tag{4.1}$$

A catastrophic physical event (e.g., total utility power grid failure or flooding) disabling AZ_1 leaves AZ_2 entirely unaffected because power feeds, generator fuel supplies, chiller plants, and network carriers are physically independent.

4.3.2 Azure Resource Management Hierarchy

Azure employs a strict hierarchical management model enforced through **Azure Resource Manager (ARM)**:

1. **Management Groups:** Containers that help manage access, policy, and compliance across multiple subscriptions.
2. **Subscriptions:** Agreements with Microsoft that associate resource usage with billing accounts and govern resource quotas.
3. **Resource Groups:** Logical containers into which Azure resources (web apps, virtual networks, storage accounts, databases) are deployed, managed, and monitored as a unified lifecycle unit. Deleting a resource group deletes all resources contained within it.
4. **Resources:** Individual instances of cloud services (e.g., a virtual machine, virtual network, or database).

4.3.3 Azure App Service: Architecture and Deployment Topologies

Azure App Service is a fully managed Platform-as-a-Service (PaaS) engine designed for hosting web applications, RESTful APIs, and mobile backends.

App Service Plans

An **App Service Plan** represents the set of compute resources allocated to run web apps. Every web application running in App Service must be associated with an active App Service Plan. Multiple applications can share a single App Service Plan to optimize cost.

Pricing Tier	Scale Limits	Deployment Slots	Intended Enterprise Use Case
Free / Shared	Shared CPU	None (0)	Prototyping, testing, and student projects.
Basic	Up to 3 dedicated VMs	None (0)	Low-traffic production workloads without automated scaling.
Standard	Up to 10 dedicated VMs	Up to 5 slots	Production workloads requiring auto-scaling, daily backups, and staging slots.
Premium (P1v3–P3v3)	Up to 30 dedicated VMs	Up to 20 slots	High-performance enterprise workloads, intense traffic, VNet integration, and maximum deployment slots.
Isolated	Up to 100 dedicated VMs	Up to 20 slots	Mission-critical workloads requiring dedicated hardware inside a private virtual network (App Service Environment).

Deployment Slots Mechanics and Zero-Downtime Swaps

Deployment slots are live web applications with their own distinct hostnames (e.g., `app-staging.azurewebsites.net` alongside `app.azurewebsites.net`). When a new build is deployed:

1. The application code is deployed to the *staging* slot.
2. Validation, warm-up requests, and automated regression tests run against staging without impacting live production traffic.
3. A **Slot Swap** is triggered. The Azure routing subsystem swaps the virtual IP addresses and DNS bindings of the two slots. Traffic seamlessly flows to the newly warmed-up code with zero downtime.
4. If an unforeseen bug emerges in production, an instantaneous reverse swap rolls back the environment immediately.

Security Configuration: HTTPS Only

By default, web apps created in Azure accept plain HTTP traffic on port 80. Enabling the **HTTPS Only** setting activates an automated server-side HTTP 301 Permanent Redirect rule, routing all incoming unencrypted requests to encrypted HTTPS on port 443 with TLS certificates.

4.3.4 Azure SQL Database: Decoupled Storage and Hyperscale Tier

Traditional relational database management systems bind computing power (CPU and memory) tightly to physical disk arrays. Under heavy scaling, expanding storage requires database migration or extensive downtime.

Azure SQL Database Hyperscale revolutionizes relational storage by introducing a fully decoupled, cloud-native architecture:

- **Compute Tier:** Stateless compute nodes executing the SQL engine (query optimization, parsing, transaction coordinator). Compute nodes can scale up or down in seconds independently of data size. Read-only replicas can be added dynamically to scale read queries.
- **Page Servers:** Distributed secondary storage nodes that cache database pages in local solid-state drives (SSD) and serve read operations.
- **Log Service:** High-throughput, low-latency transaction log processing tier using Paxos consensus algorithms for deterministic durability.
- **Azure Storage Fabric:** Persistent long-term storage backed by Azure Blob Storage, allowing automatic storage expansion up to **100 TB** without application downtime or performance degradation.

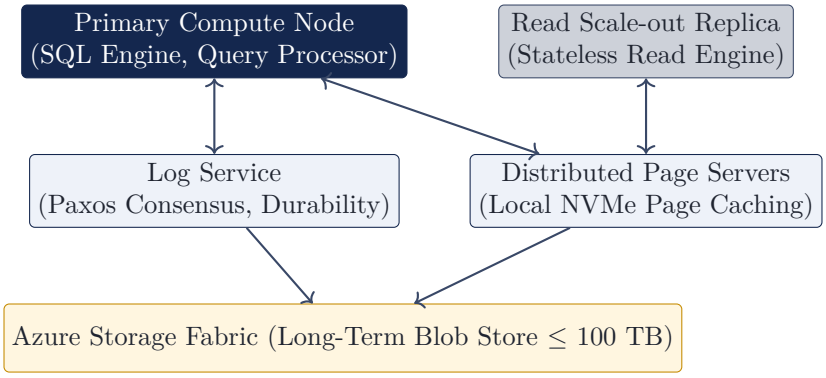


Figure 4.2: Azure SQL Database Hyperscale Decoupled Compute-Storage Architecture.

4.3.5 Hands-On Deployment: Python Web App via Azure CLI

The lecture demonstrates deploying a Python Flask application to Azure App Service using the Azure Cloud Shell:

```
# Step 1: Clone the Flask starter template
git clone https://github.com/Azure-Samples/python-docs-hello-world
cd python-docs-hello-world

# Step 2: Configure deployment credentials for Git/FTP
az webapp deployment user set --user-name devadmin --password "SecretPassword123!"

# Step 3: Create an Azure Resource Group
az group create --name CloudNPTEL-RG --location "West Europe"

# Step 4: Provision an App Service Plan (Free SKU)
az appservice plan create --name NPTEL-ServicePlan \
   --resource-group CloudNPTEL-RG --sku FREE --is-linux
```

```
# Step 5: Provision the Web App container with Python 3.9 runtime
az webapp create --name nptel-demo-web-2024 \
   --resource-group CloudNPTEL-RG \
   --plan NPTEL-ServicePlan --runtime "PYTHON:3.9"

# Step 6: Configure local Git deployment endpoint
az webapp deployment source config-local-git \
   --name nptel-demo-web-2024 --resource-group CloudNPTEL-RG

# Step 7: Push source code to the remote Azure Git repository
git remote add azure https://devadmin@nptel-demo-web-2024.scm.azurewebsites.net/nptel-demo-
    web-2024.git
git push azure master
```

4.4 Google Cloud Platform (GCP) Core Ecosystem

4.4.1 Google Global Infrastructure and Software-Defined Networking

Google Cloud Platform runs on the identical physical infrastructure that powers Google Search, YouTube, and Gmail. Key differentiators highlighted in the curriculum include:

- **Global Private Fiber Network:** Traffic between GCP regions travels across Google's private transoceanic fiber backbone rather than the public Internet, dramatically reducing latency, jitter, and packet loss.
- **Andromeda SDN:** Google's custom software-defined network virtualization stack delivering high packet processing rates and massive bandwidth.
- **Jupiter Data Center Fabric:** Non-blocking bisection switching fabric delivering petabits-per-second aggregate bandwidth within data center clusters.

4.4.2 Taxonomy of GCP Compute Services

GCP provides a spectrum of compute abstractions spanning from bare hardware control to fully managed serverless execution:

Compute Service	Model	Control Level	Core Technical Characteristic
Compute Engine (GCE)	IaaS	Full OS / Kernel	Raw virtual machines; custom CPU/RAM configurations; local SSD scratch disks; live migration support.
Google Kubernetes Engine (GKE)	Managed PaaS / CaaS	Container Cluster	Enterprise managed Kubernetes; automated node repair, rolling upgrades, and horizontal pod autoscaling.
Cloud Run	Serverless CaaS	Container Image	Fully managed serverless container platform; scales down to zero instances; billed per 100 ms of CPU execution.
App Engine (GAE)	PaaS	Application Code	Turnkey application hosting; automatic traffic splitting, built-in load balancing, and automated zero-management scaling.
Cloud Functions	FaaS	Single Function	Event-driven code snippets executed in response to Cloud Storage, Pub/-Sub, or HTTP webhook triggers.

4.4.3 Google App Engine (GAE): Standard vs Flexible Environments

Google App Engine provides two architectural runtimes:

- **Standard Environment:** Applications run in secure sandboxed language runtimes (Python, Java, Go, PHP, Node.js). It provides instantaneous scale-up in response to sudden traffic spikes and scales down to **zero instances** when traffic ceases, minimizing operational costs. However, it restricts background threads, filesystem writes, and custom OS packages.

- **Flexible Environment:** Applications execute inside Docker containers on managed Compute Engine virtual machines. It allows custom runtimes, background threads, and arbitrary C-extensions, but scaling takes several minutes and cannot scale to zero instances.

Configuration and Deployment Mechanics

An App Engine application is configured via `app.yaml`:

```
# Minimal app.yaml for Python Flask on App Engine Standard
runtime: python39
instance_class: F1

handlers:
  - url: /static
    static_dir: static
  - url: /.*
    script: auto
```

Deployment and traffic orchestration commands:

```
# Start local development server watching for file changes
dev_appserver.py app.yaml

# Deploy application to GCP App Engine
gcloud app deploy app.yaml --project gcp-nptel-cloud

# View running web application in default browser
gcloud app browse

# Split traffic between version v1 and version v2 (A/B testing)
gcloud app services set-traffic default --splits v1=0.8,v2=0.2
```

4.4.4 GCP Storage and Database Services

- **Cloud Storage (GCS):** Unified object storage offering four storage classes (Standard, Nearline, Coldline, Archive) accessible via HTTP REST APIs. It supports static website hosting by uploading HTML/CSS assets to a globally unique bucket, setting public read permissions (`allUsers:R`), and configuring default index (`index.html`) and 404 error pages.
- **Cloud SQL:** Fully managed relational database service supporting MySQL, PostgreSQL, and Microsoft SQL Server. It handles automated backups, read replicas, and vertical hardware scaling.
- **Cloud Spanner:** A globally distributed, enterprise relational database service that uniquely combines SQL semantics, horizontal scaling across continents, and **external consistency** (the strongest consistency guarantee in distributed systems) using synchronized hardware atomic clocks and GPS receivers (**TrueTime API**).
- **Firestore / Datastore:** Serverless, horizontally scalable NoSQL document database designed for mobile and web backends, supporting offline synchronization and ACID transactions.
- **BigQuery:** Serverless, highly scalable, petabyte-scale cloud data warehouse designed for fast SQL queries on massive datasets using Google's columnar storage format (Capacitor) and distributed execution engine (Dremel).

4.5 Cross-Cloud Platform Comparative Matrix

The following comparative table synthesizes equivalent service abstractions across the three leading public hyperscalers and the private OpenStack framework:

Functional Category	OpenStack	Microsoft Azure	Google Cloud (GCP)	Amazon W
Compute (VMs)	Nova	Virtual Machines	Compute Engine (GCE)	Elastic Co
Containers (K8s)	Magnum	Azure Kubernetes (AKS)	Google Kubernetes (GKE)	Elastic Ku
Serverless PaaS	Solum	App Service	App Engine (GAE)	Elastic Be
Serverless FaaS	Qinling	Azure Functions	Cloud Functions	AWS Lam
Block Storage	Cinder	Azure Managed Disks	Persistent Disk	Elastic Bl
Object Storage	Swift	Azure Blob Storage	Cloud Storage (GCS)	Simple St
Virtual Networking	Neutron	Azure Virtual Network	Virtual Private Cloud (VPC)	Virtual Pr
Identity & IAM	Keystone	Azure Active Directory	Cloud IAM	AWS IAM
Relational SQL	Trove	Azure SQL Database	Cloud SQL / Spanner	Amazon R
Data Warehouse	—	Azure Synapse Analytics	BigQuery	Amazon R
Message Bus	Zaqar	Azure Service Bus	Cloud Pub/Sub	Amazon S

4.6 Comprehensive Solved Assignment Problems (Week 4)

Solved Example: Azure SQL Database Hyperscale Storage Decoupling

Given: An enterprise architect requires an Azure SQL Database tier where storage can scale dynamically up to 100 TB without application downtime or performance degradation during scaling operations.
Formula: Compute-Storage Decoupling in Hyperscale:

$$\text{Storage Capacity} \leq 100 \text{ TB}, \quad \text{Scale Time} = \mathcal{O}(1) \text{ w.r.t Data Volume} \tag{4.2}$$

Step-by-step substitution:

1. In standard relational architectures (Basic, Standard, General Purpose), compute and storage nodes are co-located. Resizing storage requires data movement or disk replication, causing downtime.
2. **Hyperscale** introduces a multi-tiered architecture with stateless compute nodes, distributed page servers, and an Azure Blob Storage fabric.
3. Storage allocates 40 GB increments automatically as data grows, scaling seamlessly up to 100 TB.

Final Answer: **Hyperscale**.

Solved Example: Availability Zone Power Outage Isolation

Given: A company deploys two Virtual Machines across different Availability Zones within the same Azure Region. A total power failure occurs in the data center hosting Zone 1.

Formula: Fault Domain Independence:

$$\text{State}(\text{VM}_2) = \text{Running}, \quad \text{since Power}(\text{AZ}_2) \cap \text{Power}(\text{AZ}_1) = \emptyset \tag{4.3}$$

Step-by-step substitution:

1. Availability Zones are physically discrete data centers within an Azure Region.
2. Each zone has independent physical utility substations, backup diesel generators, cooling towers, and networking uplinks.
3. A power outage in Zone 1 disrupts resources inside Zone 1 only. Resources in Zone 2 operate without interruption.

Final Answer: **The second VM continues running because Availability Zones are physically isolated from each other**.

Solved Example: App Service Deployment Slot Limits

Given: Identification of the Azure App Service pricing plan that supports the highest number of staging deployment slots for CI/CD pipelines.
Step-by-step substitution:

1. **Free / Shared:** 0 deployment slots.
2. **Basic:** 0 deployment slots.
3. **Standard:** Supports up to 5 deployment slots.
4. **Premium (P1v3 and higher):** Supports up to 20 deployment slots.

Final Answer: **Premium (P1v3 and higher)**.

Solved Example: Enforcing Encrypted Web Traffic on App Service

Given: Users can access an Azure App Service application via unencrypted HTTP (`http://`). The administrator must enforce end-to-end encryption.
Formula: HTTP 301 Permanent Redirect:

$$\text{Request}(\text{http://...:80}) \xrightarrow{\text{Redirect}} \text{Response}(\text{https://...:443}) \tag{4.4}$$

Step-by-step substitution:

1. Enabling **HTTPS Only** in the App Service configuration menu automatically configures the frontend reverse proxy to redirect all plain HTTP requests to HTTPS, securing all communications.

Final Answer: **HTTPS Only**.

Solved Example: Globally Scalable Relational Database in GCP

Given: Selection of a Google Cloud database service that provides relational SQL querying, global horizontal scaling, and strict transactional consistency.
Step-by-step substitution:

1. **Cloud SQL:** Managed relational DB, but scales primarily vertically (single-region).
2. **Firestore:** NoSQL document store, not relational SQL.
3. **Memorystore:** In-memory Redis/Memcached cache.
4. **Cloud Spanner:** Combines relational structure, SQL support, ACID transactions, and horizontal scaling across global regions with TrueTime synchronization.

Final Answer: **Cloud Spanner**.

Solved Example: OpenStack Persistent Volume Service

Given: A virtual machine requires a 100 GB persistent block storage disk that persists after VM termination.
Step-by-step substitution:

1. **Nova:** Manages instance compute execution.
2. **Swift:** Provides object storage (REST API), not attachable block devices.
3. **Keystone:** Provides identity and access tokens.
4. **Cinder:** Manages block storage volumes. Cinder volumes have independent lifecycles from Nova compute instances.

Final Answer: **Cinder**.

Solved Example: GKE vs Cloud Run Architectural Distinction

Given: Distinguishing Google Kubernetes Engine (GKE) from Google Cloud Run.
Step-by-step substitution:

1. **GKE** provides managed Kubernetes clusters where users configure nodes, control plane settings, and cluster networking.
2. **Cloud Run** is a fully managed serverless container platform where developers deploy container images without managing Kubernetes clusters, nodes, or control planes.

Final Answer: **GKE manages Kubernetes clusters, while Cloud Run executes containers without requiring cluster management**.

Solved Example: GCP Static Website Hosting Configuration

Given: Configuring Google Cloud Storage for static website hosting with `index.html` and `404.html`.
Step-by-step substitution:

1. The user creates a storage bucket named to match the domain.
2. In the bucket configuration, the user sets the **Main Page (Index)** suffix and **404 (Not Found)** error document.
3. The bucket and objects are shared with public read access (`allUsers`).

Final Answer: **Specify the Main Page and 404 (Not Found) Page in the website configuration dialog**.

Solved Example: App Service Plan Resource Sharing

Given: Determining the impact of hosting multiple Azure Web Apps inside the same App Service Plan.
Step-by-step substitution:

1. An App Service Plan defines the physical VM compute capacity (CPU, RAM, Region).
2. All web applications assigned to that plan share the underlying virtual machine instances, reducing hosting costs.

Final Answer: **The web apps share the compute resources defined by the single App Service Plan, optimizing overall cost**.

Solved Example: OpenStack Virtual Networking Service

Given: Identification of the OpenStack component responsible for managing tenant virtual networks, routers, subnets, and floating IP allocation.
Step-by-step substitution:

1. **Neutron** is OpenStack's software-defined networking (SDN) component, managing virtual switches, routers, firewalls, and IP address management.

Final Answer: **Neutron**.

Service Level Agreements, Cloud Economics, and Green Computing

5.1 Introduction and Week Overview

Week 5 investigates the operational, contractual, and environmental dimensions of cloud infrastructure. The core pillars covered in this module include:

1. **Service Level Agreements (SLAs) and Availability Modeling:** Contractual metrics, Service Level Objectives (SLOs), uptime and downtime mathematics, and tiered service credit penalty formulations.
2. **Cloud Economics and Utility Pricing:** Total Cost of Ownership (TCO), peak-to-average demand analysis, utility premium thresholds ($U < P/A$), and penalty integrals under dynamic workloads.
3. **Big Data Processing with Hadoop and HDFS:** Distributed file storage architectures, input split calculations, NameNode metadata management, and MapReduce execution mechanics (Partitioner vs Combiner).
4. **Green Cloud Computing and Energy Efficiency:** Data center power consumption breakdown, dynamic server energy behavior, Power Usage Effectiveness (PUE), Data Center Infrastructure Efficiency (DCiE), and Dynamic Voltage and Frequency Scaling (DVFS).

5.2 Service Level Agreements (SLAs) in Cloud Computing

5.2.1 Contractual Foundations and Definitions

Definition: Service Level Agreement (SLA)

A **Service Level Agreement (SLA)** is a legally binding formal contract negotiated between a Cloud Service Provider (CSP) and a Cloud Consumer. It codifies the minimum expected quality of service, availability guarantees, performance baselines, measurement methodologies, and financial liabilities (service credits/penalties) incurred when guarantees are breached.

An SLA is composed of discrete, quantifiable criteria known as **Service Level Objectives (SLOs)**:

$$\text{SLO}_i = \langle \text{Service Metric}, \text{Comparison Operator}, \text{Target Value}, \text{Measurement Window} \rangle \quad (5.1)$$

Typical SLO parameters include:

- **Availability (Uptime):** Percentage of time the service responds to valid HTTP requests with non-error status codes.
- **Latency / Response Time:** Round-trip processing delay for API calls (e.g., 99th percentile latency < 150 ms).
- **Throughput:** Number of discrete transactions or input/output operations per second (IOPS).
- **Data Durability:** Probability of preventing permanent data loss over a one-year period (e.g., AWS S3 guarantees 99.999999999% "eleven nines" durability).

5.2.2 Availability Mathematics and "Nines" Notation

Service availability A is mathematically defined as the ratio of scheduled operational uptime to total scheduled duration:

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} = \frac{T_{\text{scheduled}} - T_{\text{downtime}}}{T_{\text{scheduled}}} \times 100\% \tag{5.2}$$

where MTBF is the **Mean Time Between Failures** and MTTR is the **Mean Time To Repair**.

Availability Level	Percentage	Allowable Downtime / Year	Allowable Downtime / Month (30 Days)
Two Nines	99.0%	87.6 hours (3.65 days)	7.20 hours
Three Nines	99.9%	8.76 hours	43.8 minutes
Four Nines	99.99%	52.56 minutes	4.38 minutes
Five Nines	99.999%	5.26 minutes	26.3 seconds

5.2.3 Scheduled Operational Hours Downtime Formulation

In enterprise contexts, customer applications frequently run on partial daily schedules rather than continuously. When an SLA availability guarantee A_{target} applies to a service operating h hours per day across D calendar days:

$$T_{\text{scheduled}} = h \times D \tag{5.3}$$

$$T_{\text{allowable downtime}} = (1 - A_{\text{target}}) \times T_{\text{scheduled}} \tag{5.4}$$

Solved Example: SLA Outage Evaluation (Lecture Tutorial Problem 1)

Given: A cloud provider guarantees 99% service availability. A third-party client application runs for $h = 12$ hours/day over a $D = 30$-day billing month. An audit reveals a total outage of $T_{\text{actual}} = 10.75$ hours. Determine if the provider violated the SLA.
Formula:

$$T_{\text{allowable}} = (1 - 0.99) \times (12 \times 30) = 0.01 \times 360 = 3.6 \text{ hours} \tag{5.5}$$

Step-by-step substitution:

1. Compute total scheduled operational time: $T_{\text{scheduled}} = 12 \times 30 = 360$ hours.

2. Compute allowable outage duration: $0.01 \times 360 = 3.6$ hours.

3. Compare actual outage against threshold: $T_{\text{actual}} = 10.75$ hours > 3.6 hours.

4. Measured availability achieved:

$$A_{\text{actual}} = \frac{360 - 10.75}{360} \times 100\% = \frac{349.25}{360} \times 100\% \approx 97.01\% \tag{5.6}$$

Final Answer: The SLA guarantee was severely violated ($97.01\% < 99.0\%$). Actual downtime exceeded allowable downtime by 7.15 hours.

5.2.4 Service Credit Penalty Models

CSPs do not pay cash damages for SLA breaches; instead, they issue **Service Credits** that discount future billing cycles according to a tiered schedule:

Solved Example: Comprehensive SLA Credit Settlement (Lecture Tutorial Problem 2)

Given: A company contracts a cloud service from Provider P for a $D = 30$-day period with:

- Target availability: 99.95%.
- Maximum operational hours: $h = 12$ hours/day.
- Base daily billing rate: \$50/day (Total nominal fee: $30 \times \$50 = \$1500$).
- Tiered Service Credit policy:
  - Uptime $< 99.95\% \implies$ 10% credit discount.
  - Uptime $< 99.00\% \implies$ 25% credit discount.
- Recorded outages across 5 separate incidents: 5 h, 30 min (0.5 h), 1 h 30 min (1.5 h), 15 min (0.25 h), and 2 h 25 min (2.4167 h).

Formula:

$$T_{\text{outage}} = \sum_{k=1}^{5} t_k, \quad A = \frac{T_{\text{scheduled}} - T_{\text{outage}}}{T_{\text{scheduled}}} \times 100\% \tag{5.7}$$

Step-by-step substitution:

1. Total scheduled service duration:

$$T_{\text{scheduled}} = 12 \times 30 = 360 \text{ hours} \tag{5.8}$$

2. Sum of all discrete outage durations:

$$T_{\text{outage}} = 5.0 + 0.5 + 1.5 + 0.25 + 2.4167 = 9.6667 \text{ hours} \quad (9 \text{ h } 40 \text{ min}) \tag{5.9}$$

3. Calculate realized monthly uptime percentage:

$$A = \frac{360 - 9.6667}{360} \times 100\% = \frac{350.3333}{360} \times 100\% \approx 97.315\% \tag{5.10}$$

4. Evaluate penalty tier: Since $97.315\% < 99.00\%$, the violation triggers the higher **25% Service Credit**.

5. Compute effective billing settlement:

$$\text{Nominal Cost} = 30 \times \$50 = \$1500 \tag{5.11}$$

$$\text{Service Credit Awarded} = 0.25 \times \$1500 = \$375 \tag{5.12}$$

$$\text{Effective Payable Cost} = \$1500 - \$375 = \$1125 \tag{5.13}$$

Final Answer: Effective amount payable is **$1,125.00** (with $375 awarded as SLA penalty credits).

5.3 Cloud Economics and Utility Pricing

5.3.1 Economic Drivers of Cloud Adoption

Traditional IT infrastructure demands that enterprises size on-premises data centers to satisfy **Peak Demand** (P). Because physical server procurement requires long lead times, hardware sits idle during periods of average demand (A), generating waste.

Cloud computing transforms capital expenses (**CapEx**) into operational expenses (**OpEx**) through:

- **Common Infrastructure:** Standardized resource pools yielding statistical multiplexing gains.
- **Location Independence:** Aggregating demand across global time zones to smooth aggregate utilization curves.
- **Elasticity and Pay-As-You-Go:** Provisioning and releasing resources instantaneously to mirror real-time load.

5.3.2 Formal Utility Premium Break-Even Analysis

Let:

- $D(t)$: Instantaneous resource demand at time t over time interval $[0, T]$.
- $P = \max_{0 \le t \le T} D(t)$: Peak resource demand.
- $A = \frac{1}{T} \int_0^T D(t)\, dt$: Average resource demand.
- B: Unit cost per unit time of owned (in-house baseline) infrastructure.
- C: Unit cost per unit time of utility cloud infrastructure.
- $U = \frac{C}{B}$: **Utility Premium** (the pricing multiple charged by the cloud provider over bare raw hardware ownership).

Key Result: Cloud vs In-House Cost Break-Even Theorem

Total in-house baseline cost (B_T) must provision for peak demand P:

$$B_T = P \times B \times T \tag{5.14}$$

Total cloud cost (C_T) meters exact demand over time:

$$C_T = \int_0^T C \cdot D(t)\, dt = C \int_0^T D(t)\, dt = (U \cdot B) \cdot (A \cdot T) = A \cdot U \cdot B \cdot T \tag{5.15}$$

Cloud computing is economically superior to owning physical hardware if and only if:

$$C_T < B_T \iff A \cdot U \cdot B \cdot T < P \cdot B \cdot T \iff U < \frac{P}{A} \tag{5.16}$$

Conclusion: Cloud computing is cheaper than owning private hardware whenever the utility premium U is strictly less than the **Peak-to-Average demand ratio** (P/A).

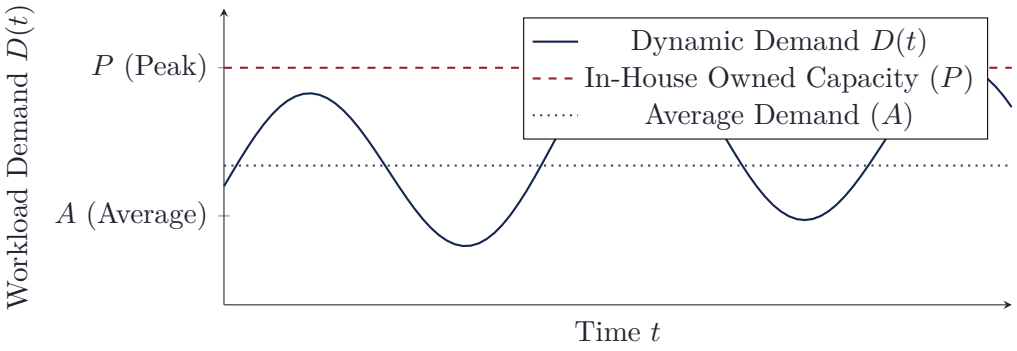


Figure 5.1: Dynamic Demand vs In-House Static Over-Provisioning.

5.3.3 Workload Penalty Integrals for Static Provisioning

When an organization owns fixed resources $R(t)$, any divergence between available capacity $R(t)$ and instantaneous workload demand $D(t)$ incurs financial and operational penalties:

$$\text{Penalty Cost} \propto \int_0^T |D(t) - R(t)|\, dt \tag{5.17}$$

- If $R(t) > D(t)$: The organization wastes capital paying for idle hardware, power, cooling, and software licensing.
- If $R(t) < D(t)$: The system suffers performance degradation, latency spikes, dropped transactions, and lost customer revenue.

Under exponential workload growth ($D(t) = e^t$), periodic static procurement with interval t_p produces provisioning $R(t) = e^{t-t_p}$. The shortfall increases exponentially:

$$\text{Deficit}(t) = D(t) - R(t) = e^t - e^{t-t_p} = e^t(1 - e^{-t_p}) \tag{5.18}$$

demonstrating that fixed provisioning cycles can never keep pace with exponential demand growth.

5.4 Big Data Architecture: Hadoop and HDFS

5.4.1 Hadoop Distributed File System (HDFS) Architecture

HDFS is designed to store multi-terabyte and petabyte files reliably across large clusters of commodity hardware. It adopts a master/worker topology:

- **NameNode (Master):** Maintains the directory tree and file metadata in system RAM. File metadata maps filenames to a sequence of block IDs. Modifications are recorded durably in the `EditLog` on disk and consolidated periodically with the `FsImage` checkpoint file via the **Secondary NameNode**.
- **DataNodes (Workers):** Store actual data blocks as discrete files on local ext4/xfs filesystems. They handle block read/write operations from clients and emit heartbeat signals every 3 seconds to confirm node health.

5.4.2 Block Size and Input Split Mathematics

Unlike traditional filesystems with 4 KB disk clusters, HDFS employs very large block sizes (typically 64 MB, 128 MB, or 256 MB) to amortize disk seek times against sequential transfer rates.

When an input file of size S is loaded into an HDFS cluster configured with block size B:

$$N_{\text{blocks}} = \left\lceil \frac{S}{B} \right\rceil \tag{5.19}$$

In standard MapReduce configurations, Hadoop assigns exactly one **Mapper task** per logical **InputSplit** (which aligns 1:1 with HDFS physical blocks).

Solved Example: HDFS Mapper Task Allocation

Given: An HDFS cluster operates with block size $B = 128$ MB. A client uploads an input file of size $S = 700$ MB to execute a MapReduce job. Assuming default mapping of one map task per HDFS block, calculate the number of mapper tasks created.
Formula:

$$N_{\text{mappers}} = \left\lceil \frac{S}{B} \right\rceil \tag{5.20}$$

Step-by-step substitution:

1. Decompose 700 MB across 128 MB block allocations:

Block 1:	128 MB	Block 4:	128 MB	
Block 2:	128 MB	Block 5:	128 MB	
Block 3:	128 MB	Block 6:	60 MB	(Partial Block)

2. Total allocated blocks: $5 \times 128 \text{ MB} + 60 \text{ MB} = 700 \text{ MB} \implies 6$ blocks.

3. Each block corresponds to an input split: $N_{\text{mappers}} = 6$.

Final Answer: Exactly **6 mapper tasks** are spawned.

5.4.3 MapReduce Execution Pipeline: Partitioner vs Combiner

The lecture highlights two critical intermediate optimization components in MapReduce:

- **Combiner (Mini-Reducer):** An *optional* optimization component executed on the map worker node prior to network shuffle. It performs local pre-aggregation on intermediate key-value pairs (e.g., computing sub-totals in a word count job), drastically decreasing the volume of intermediate data transmitted across the cluster network. The execution of a Combiner is never guaranteed; Hadoop may run it zero, one, or multiple times depending on memory pressure.
- **Partitioner:** A *mandatory* component that determines which physical Reducer instance receives each intermediate key. By default, Hadoop utilizes the **HashPartitioner**:

$$\text{Reducer ID} = (\text{key.hashCode() \& Integer.MAX_VALUE}) \pmod{N_{\text{reducers}}} \tag{5.21}$$

  This guarantees that all intermediate values associated with the identical key arrive at the exact same Reducer node.

5.5 Green Cloud Computing and Server Energy Dynamics

5.5.1 Data Center Energy Footprint and Efficiency Metrics

Massive data centers consume hundreds of megawatts of electricity. Energy consumption is divided into two broad categories:

1. **IT Equipment Power (P_{IT}):** Compute servers, storage arrays, and network switches.
2. **Infrastructure Overhead Power (P_{overhead}):** Chiller plants, cooling towers, fans, uninterruptible power supplies (UPS), power distribution units (PDUs), and facility lighting.

Definition: Power Usage Effectiveness (PUE) and DCiE

The industry standard metric established by The Green Grid is **Power Usage Effectiveness (PUE)**:

$$\text{PUE} = \frac{\text{Total Facility Power}}{\text{IT Equipment Power}} = \frac{P_{\text{IT}} + P_{\text{overhead}}}{P_{\text{IT}}} \geq 1.0 \tag{5.22}$$

A PUE of 2.0 indicates that for every 1 Watt powering servers, another 1 Watt is consumed by cooling and power delivery. Modern hyperscale facilities (Google, Microsoft) achieve PUE values between 1.1 and 1.2.
The inverse metric is **Data Center Infrastructure Efficiency (DCiE)**:

$$\text{DCiE} = \frac{1}{\text{PUE}} \times 100\% \tag{5.23}$$

5.5.2 Server Power Dynamics and Core Utilization

A physical server's power consumption is not static; it scales with CPU utilization and active hardware components:

$$P(u) = P_{\text{idle}} + (P_{\text{peak}} - P_{\text{idle}}) \times u \tag{5.24}$$

where P_{idle} is the idle power draw (typically 50% to 60% of peak power P_{peak}), and $u \in [0, 1]$ represents CPU utilization.

Key Result: Impact of Dynamic Core Activation on Power Draw

When a multi-core server detects a workload spike and dynamically switches CPU cores from low-power C-states (sleep states) to active C0 execution states:

1. Each active core draws dynamic switching power: $P_{\text{dynamic}} = \alpha \cdot C_L \cdot V^2 \cdot f$ (where α is activity factor, C_L load capacitance, V operating voltage, and f clock frequency).
2. Static leakage current power increases proportionally with core temperature.
3. **Net Effect:** Total server power consumption **increases** as additional CPU cores become active, even if workload execution time per core drops.

5.6 Comprehensive Solved Assignment Problems (Week 5)

Solved Example: SLA Downtime Allowance Calculation

Given: A cloud provider commits to a 99% uptime SLA. The client's application runs 12 hours/day for 30 days. Find maximum allowable downtime.
Step-by-step substitution:

$$T_{\text{scheduled}} = 12 \text{ h/day} \times 30 \text{ days} = 360 \text{ hours} \tag{5.25}$$

$$T_{\text{allowed downtime}} = (1 - 0.99) \times 360 = 0.01 \times 360 = 3.6 \text{ hours} \tag{5.26}$$

Final Answer: **3.6 hours**.

Solved Example: Multi-Core Server Power Consumption

Given: A cloud server dynamically activates additional CPU cores to handle a sudden surge in user requests.
Step-by-step substitution:

1. Activating dormant processor cores draws additional dynamic switching power and leakage current.
2. While techniques like DVFS reduce voltage at lower frequencies, adding active cores increases total aggregate wattage drawn by the system.

Final Answer: **Power consumption increases as additional CPU cores become active**.

Solved Example: Mitigating Upfront Capital Expenditure

Given: Which cloud characteristic directly eliminates massive upfront hardware purchases?
Step-by-step substitution:

1. **Pay-as-you-go pricing** converts Capital Expenditure (CapEx) into Operational Expenditure (OpEx), allowing organizations to pay strictly for consumed compute time and storage without purchasing servers upfront.

Final Answer: **Pay-as-you-go pricing**.

Solved Example: HDFS Block Input Splits and Map Tasks

Given: Block size $B = 128$ MB, input file $S = 700$ MB.
Step-by-step substitution:

$$N = \left\lceil \frac{700}{128} \right\rceil = \lceil 5.46875 \rceil = 6 \tag{5.27}$$

Final Answer: **6**.

Solved Example: Primary Virtualization Management Entity

Given: Identify the virtualization layer component responsible for creating, running, and managing virtual machines.
Step-by-step substitution:

1. The **Hypervisor** (or Virtual Machine Monitor, VMM) abstracts physical hardware resources and mediates guest virtual machine execution.

Final Answer: **Hypervisor**.

Solved Example: MapReduce Key-to-Reducer Assignment

Given: Identify the MapReduce subsystem that routes each intermediate key to its designated Reducer.
Step-by-step substitution:

1. The **Partitioner** calculates the destination reducer index for each key (typically via hash modulo operations).

Final Answer: **Partitioner**.

Solved Example: Optional MapReduce Optimization Phase

Given: Identify the MapReduce component that is optional and may execute zero, one, or multiple times.
Step-by-step substitution:

1. The **Combiner** operates as a semi-reducer on local map output. Hadoop does not guarantee its execution; therefore, MapReduce job logic must remain algebraically valid even when the Combiner is skipped entirely.

Final Answer: **Combiner**.

Solved Example: Data Center Energy Consumption Contributors

Given: Which data center subsystems contribute directly to overall energy consumption?
Step-by-step substitution:

1. IT server equipment (processors, memory, disks).
2. Air conditioning chillers, cooling towers, and ventilation fans.

3. Power distribution units (PDUs) and uninterruptible power supply (UPS) transformers.

Final Answer: **All of the above**.

Solved Example: Server Consolidation Impact on Green Computing

Given: How does server consolidation via live VM migration promote green cloud computing?
Step-by-step substitution:

1. Packing virtual machines onto fewer physical hosts allows underutilized hosts to be powered down into deep sleep states, eliminating idle power waste.

Final Answer: **Reduces total energy consumption by powering down idle physical servers**.

Solved Example: HDFS Data Integrity and Fault Tolerance

Given: How does HDFS achieve fault tolerance against DataNode drive crashes without relying on RAID hardware?
Step-by-step substitution:

1. HDFS replicates each data block across multiple physically independent DataNodes (default replication factor $R = 3$) with rack awareness.

Final Answer: **Block Replication across multiple nodes**.

Cloud Security, Privacy, and Trust Management

6.1 Introduction and Week Overview

Week 6 establishes the theoretical and operational foundations of security in multi-tenant cloud environments. The curriculum systematically progresses through four integrated modules:

1. **Foundational Security Primitives and the CIA Triad:** Mathematical definitions of Confidentiality, Integrity, and Availability; formal models of security policy versus enforcement mechanisms.
2. **Attack Taxonomy and Attack Lifecycles:** The four classical structural attacks (Interruption, Interception, Modification, Fabrication); Passive vs Active attacks; reconnaissance, scanning, and exploitation phases.
3. **Identity, Authentication, and Authorization:** Deep distinction between Authentication (AuthN) and Authorization (AuthZ); federated protocols (SAML 2.0 and OAuth 2.0); Role-Based Access Control (RBAC).
4. **Virtualization Vulnerabilities and Gartner Cloud Risks:** Cross-VM side-channel attacks, cache timing analysis, hypervisor escape, and Gartner's seven cardinal cloud security risks.

6.2 Foundational Security Primitives: The CIA Triad

Security in distributed systems rests upon three core pillars known as the **CIA Triad**:

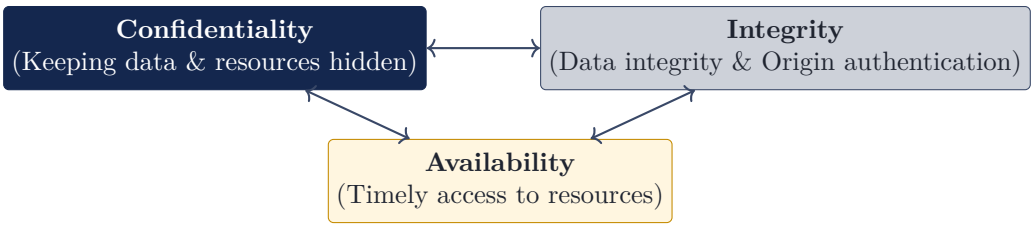


Figure 6.1: The Classical CIA Triad in Cloud Computing.

- **Confidentiality:** Preventing the unauthorized disclosure of sensitive information. Cryptographic controls (such as AES-256 for data at rest and TLS 1.3 for data in transit) ensure confidentiality.
- **Integrity:** Guaranteeing that data has not been altered, modified, corrupted, or deleted by unauthorized parties or malfunctioning software. It encompasses:

  - *Data Integrity:* State verification ensuring data matches its original format (enforced via SHA-256 cryptographic hashes and HMACs).
  - *Origin Integrity (Authenticity):* Guaranteeing the genuine origin of the data or service identity.

- **Availability:** Ensuring authorized users have uninterrupted, timely access to computing resources, data stores, and network endpoints without unexpected denial of service.

Warning: Integrity Failure Despite Perfect Encryption

A common misconception is that encryption automatically guarantees data integrity. Encryption scrambles data to preserve **Confidentiality**. If an administrator or erroneous application software bug alters database entries (e.g., corrupting financial ledger balances), the data remains confidential (still encrypted) but its **Integrity** is completely destroyed!

6.3 Structural Attack Taxonomy

Prof. Ghosh classifies security violations into four canonical architectural attack patterns based on the source (S), destination (D), and interceptor (I):

Attack Type	Violated Property	Structural Mechanism	Concrete Real-World Example
Interruption	Availability	Attacker destroys or severs the communication path between S and D.	Distributed Denial of Service (DDoS), physical fiber cut, RAM starvation.
Interception	Confidentiality	Attacker secretly monitors or taps the link, reading packets without altering them.	Packet sniffing via Wireshark, optical fiber tapping, eavesdropping.
Modification	Integrity	Attacker intercepts legitimate packets from S, alters payload bits, and forwards them to D.	Man-in-the-Middle (MitM) payload alteration, SQL injection data corruption.
Fabrication	Authenticity	Attacker injects spurious, counterfeit messages into the system claiming to originate from S.	DNS spoofing, phishing portals, fake VPN login gateways, ARP poisoning.

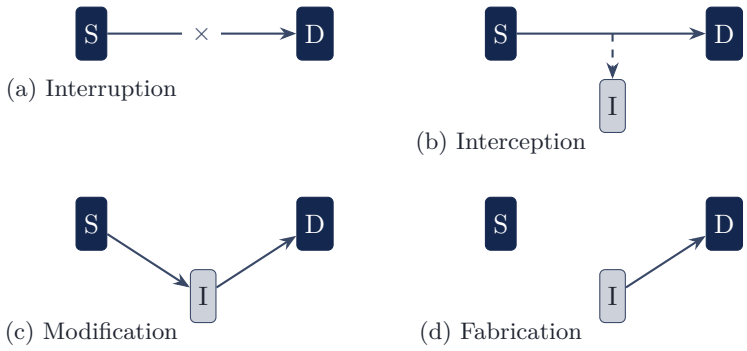


Figure 6.2: Four Fundamental Classes of Structural Security Attacks.

6.3.1 Passive vs Active Attacks

- **Passive Attacks:** The attacker monitors, inspects, or analyzes network transmissions without injecting, modifying, or delaying data packets.
  - Examples: Eavesdropping, network packet sniffing, traffic volume/timing analysis.
  - Operational Implication: Extremely difficult to detect because the system continues functioning normally. Primary defense is proactive **prevention** via strong end-to-end encryption.
- **Active Attacks:** The attacker actively alters the data stream, creates false streams, disrupts server operation, or modifies system states.
  - Examples: Session hijacking, replay attacks, man-in-the-middle tampering, masquerading, Denial of Service (DoS).
  - Operational Implication: Can be detected via integrity checks, sequence numbers, and anomaly monitoring; focus is on **detection and rapid automated recovery**.

6.3.2 The Multi-Phase Attack Lifecycle

A sophisticated adversary typically executes attacks in structured operational phases:

1. **Reconnaissance (Information Gathering):** Collecting target intelligence without actively exploiting systems.
   - *Passive Reconnaissance:* Scraping public databases, social media, WHOIS records, and DNS records.
   - *Active Reconnaissance:* DNS zone transfers, ping sweeps, and network service mapping.
2. **Scanning and Probing:** Using port scanners (e.g., Nmap) and vulnerability scanners to discover listening network services, operating system versions, and known unpatched CVE vulnerabilities.
3. **Gaining Access (Exploitation):** Executing targeted exploit payloads (such as buffer overflow attacks, SQL injections, or remote code execution) to bypass security boundaries and achieve administrative privileges.
4. **Maintaining Access and Persistence:** Installing stealth rootkits, trojans, or backdoor SSH keys to retain control across server reboots.
5. **Covering Tracks:** Altering or deleting system log files (`/var/log/auth.log`), resetting file modification timestamps, and disabling intrusion detection alerts.

6.4 Identity Management: Authentication vs Authorization

A central security theme emphasized in Week 6 is the crisp formal distinction between Authentication and Authorization:

Dimension	Authentication (AuthN)	Authorization (AuthZ)
Core Question	"Who are you?" (Verification of Identity)	"What actions are you permitted to execute?"
Mechanism	Verification of digital credentials (passwords, cryptographic tokens, biometric signatures, MFA).	Evaluation of permissions, Access Control Lists (ACLs), or Role-Based policies (RBAC).
Timing	Occurs **first** at system ingress before session establishment.	Evaluated **continuously** whenever an authenticated subject attempts an operation.
Protocol Standards	Kerberos, OpenID Connect (OIDC), SAML 2.0 Identity Assertion, FIDO2 / WebAuthn.	OAuth 2.0 (access tokens and scopes), XACML, AWS IAM Policy documents.
Failure Result	HTTP 401 Unauthorized (Unauthenticated identity).	HTTP 403 Forbidden (Identity verified, but lacks sufficient permissions).

Definition: Security Assertion Markup Language (SAML 2.0) vs OAuth 2.0

SAML 2.0: An XML-based federated protocol designed primarily for cross-domain Web Single Sign-On (**Authentication**). An Identity Provider (IdP) signs an XML assertion containing identity attributes and transmits it to a Service Provider (SP) via the user's browser.
OAuth 2.0: A lightweight, JSON-based framework designed for delegated **Authorization**. It issues short-lived Access Tokens (such as JSON Web Tokens, JWTs) granting a third-party client scoped access to a user's protected resources on a Resource Server without revealing user credentials.

6.5 Virtualization Security and Multi-Tenancy Exploits

6.5.1 Hypervisor Isolation and Virtual Machine Escape

In a public cloud, multiple untrusted virtual machines co-exist on the same physical bare-metal host. The hypervisor (Virtual Machine Monitor, VMM) is responsible for enforcing strict isolation:

- **VM Escape:** The most catastrophic virtualization exploit, where code running inside a guest operating system breaks out of the virtualized sandbox and interacts directly with the host hypervisor or host OS kernel. Once escaped, the attacker acquires total control over all co-located tenant VMs on that physical server.

6.5.2 Cross-VM Side-Channel Attacks (Ristenpart et al.)

In their seminal 2009 research paper (*Hey, You, Get Off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds*), Ristenpart et al. demonstrated that physical resource sharing in public clouds (specifically Amazon EC2) introduces subtle side-channel vulnerabilities:

1. **Co-Residence Verification:** An attacker probes cloud IP ranges to identify the target victim's network address. By exploiting internal network topologies (comparing Dom0 IP hops and round-trip ping times ≤ 1 ms), the attacker verifies that their own VM resides on the identical physical hardware as the victim.
2. **Shared Hardware Contention:** Even when CPU registers and memory pages are strictly isolated by hardware MMU page tables, physical cores share hardware caches:
   - L1 / L2 caches (per core) and unified **L3 Last Level Cache (LLC)**.
   - Memory buses, CPU branch target buffers (BTB), and hyperthreading pipelines.
3. **Cache-Timing Side Channels (Prime + Probe):**
   - *Prime Phase:* The attacker fills specific cache sets with their own data lines.
   - *Idle Window:* The attacker waits while the victim VM executes cryptographic operations (such as modular exponentiation during RSA signing).
   - *Probe Phase:* The attacker re-reads their own cached memory addresses, timing the access latency. Memory addresses that were evicted by the victim take longer to reload from main RAM.
   - *Key Extraction:* By observing which cache sets experienced evictions, the attacker reconstructs internal CPU access patterns, leaking private cryptographic keys without ever violating memory permissions!

6.6 Gartner's Seven Cardinal Cloud Security Risks

The lecture highlights the influential risk taxonomy published by Gartner for enterprise cloud computing adopters:

1. **Privileged User Access:** Cloud providers maintain system administrators with root access to physical hosts and virtualization hypervisors. Organizations must demand strict administrative auditing, background vetting, and cryptographic envelope encryption.
2. **Regulatory Compliance:** Enterprises remain legally liable for data compliance (HIPAA, PCI-DSS, ISO 27001) even when data is hosted externally. Providers must submit to external third-party compliance audits.
3. **Data Location and Sovereignty:** Cloud consumers often cannot specify the exact physical storage medium or geographic jurisdiction where data is stored. Trans-border data movement may violate national data residency laws (e.g., GDPR in the European Union or local data sovereignty mandates).
4. **Data Segregation / Isolation:** In shared multi-tenant databases and SAN volumes, data from competing corporations resides on adjacent disk sectors. Multi-tenant encryption schemes must guarantee complete logical segregation.

5. **Recovery and Business Continuity:** In the event of catastrophic physical outages or ransomware events at the provider's facility, providers must have verified Recovery Point Objectives (RPO) and Recovery Time Objectives (RTO).
6. **Investigative Support (E-Discovery):** Conducting forensic investigations or digital discovery in a multi-tenant cloud is exceptionally difficult without violating the privacy rights of other co-located tenants.
7. **Long-Term Viability:** What happens to an organization's mission-critical data if the cloud provider goes bankrupt, gets acquired by a hostile competitor, or abruptly terminates service? Data portability and non-proprietary exit strategies are essential.

6.7 Comprehensive Solved Assignment Problems (Week 6)

Solved Example: Database Bug and Security Objective Failure

Given: A database stores encrypted customer records. An administrator erroneously alters account balances due to an unhandled application bug. Identify the security objective that failed.
Step-by-step substitution:

1. Encryption preserves **Confidentiality** (unauthorized parties cannot read the records).
2. Availability is intact because users can still query the system.
3. Erroneous or unauthorized modification of data directly compromises **Integrity**.

Final Answer: **Integrity only**.

Solved Example: Fake VPN Portal Exploitation

Given: An employee visits a fake VPN portal that visually imitates the legitimate corporate login page. Before any credentials are typed, which security property has failed?
Step-by-step substitution:

1. No confidential password has been disclosed yet.
2. The fraudulent portal is masquerading as the genuine corporate gateway (a fabrication attack).
3. The security property that is broken is **Authenticity** (the claim of genuine identity).

Final Answer: **Authenticity**.

Solved Example: Classification of Active vs Passive Attacks

Given: Identify which attack is NOT considered a passive attack among: Packet sniffing, Traffic analysis, Session hijacking, Network monitoring.
Step-by-step substitution:

1. Packet sniffing, traffic analysis, and network monitoring merely inspect transmission without altering packets (Passive).
2. **Session hijacking** actively injects malicious packets, alters sequence numbers, or ter-

minates the user's connection to seize control of the session (Active).

Final Answer: **Session hijacking**.

Solved Example: Inability to Modify Transmitted Payloads

Given: Which attack cannot directly modify the contents of transmitted data?
Step-by-step substitution:

1. Replay attacks inject recorded packets; MitM attacks alter bits in flight; Masquerading sends forged data.
2. **Eavesdropping** is purely passive listening and cannot modify data contents.

Final Answer: **Eavesdropping**.

Solved Example: Denial of Service (DoS) Characteristics

Given: What best describes a Denial-of-Service attack?
Step-by-step substitution:

1. An adversary floods servers or networks with overwhelming request volumes, consuming compute/bandwidth resources so legitimate users cannot access the service.

Final Answer: **An attacker floods a server with excessive requests, making the service unavailable to legitimate users**.

Solved Example: Reconnaissance vs Exploitation Phase

Given: Identify the activity least likely to be considered part of the reconnaissance phase.
Step-by-step substitution:

1. DNS enumeration, port scanning, and searching public websites are reconnaissance activities.
2. **Exploiting a buffer overflow** to gain root access is part of the active *Exploitation* phase, not reconnaissance.

Final Answer: **Exploiting a buffer overflow vulnerability to gain administrator privileges**.

Solved Example: Authentication vs Authorization Statements

Given: Evaluate:

- Statement I: Authentication determines whether a user is allowed to access a resource.
- Statement II: Authorization determines what actions an authenticated user is permitted to perform.

Step-by-step substitution:

1. Statement I is FALSE: Authentication verifies *identity* (who the user is), not whether they are allowed to access a specific resource.

2. Statement II is TRUE: Authorization evaluates permissions to determine permitted actions.

Final Answer: **Statement I is False, and Statement II is True**.

Solved Example: Data Location and Sovereignty Concerns

Given: A multinational corporation stores EU citizen records in a US-based cloud data center and faces legal scrutiny under GDPR. Which Gartner cloud risk applies?
Step-by-step substitution:

1. European privacy mandates require personal citizen data to reside within compliant jurisdictions.

2. Moving data across geographic borders directly triggers **Data Location and Sovereignty** risk.

Final Answer: **Data Location**.

Solved Example: Multi-Tenant Cache-Timing Exploits

Given: How does an attacker extract private cryptographic keys in a cross-VM side-channel attack?
Step-by-step substitution:

1. By establishing co-residence on the same physical CPU socket and monitoring access latencies to shared L3 CPU cache lines (Prime+Probe technique).

Final Answer: **By measuring CPU cache access latency on shared physical hardware**.

Solved Example: Hypervisor Sandbox Escape Threat

Given: What is the security impact of a successful VM Escape vulnerability?
Step-by-step substitution:

1. The attacker breaks through guest OS virtualization barriers to execute arbitrary code with host hypervisor privileges, compromising all co-located tenant VMs.

Final Answer: **The attacker gains direct access to the host hypervisor, compromising all tenant VMs on the physical host**.

Mobile Cloud Computing, Cloudlets, and Geospatial Cloud

7.1 Introduction and Week Overview

Week 7 explores the convergence of cloud computing with mobile handheld devices and geographic information systems. The curriculum addresses computational scarcity at the network edge through three fundamental themes:

1. **Mobile Cloud Computing (MCC):** Architectural models augmenting battery-constrained, CPU-limited mobile devices with remote cloud resources; runtime partitioning and computation offloading.
2. **Mathematical Formulation of Energy-Aware Offloading:** Derivation of the Kumar and Lu energy saving conditions, transmission delays, idle listening power, and numerical break-even analysis.
3. **Cloudlets and Low-Latency Edge Computing:** Three-tier hierarchical architecture (Mobile $\rightarrow$ Cloudlet $\rightarrow$ Distant Cloud), transient virtual machine synthesis, and real-time augmented reality constraints.
4. **Geospatial Cloud Computing:** Spatial Data Infrastructure (SDI), Open Geospatial Consortium (OGC) web services, massive spatial data storage, and distributed spatial indexing.

7.2 Mobile Cloud Computing (MCC) Foundations

7.2.1 Motivation and Intrinsic Hardware Constraints

Despite dramatic improvements in mobile hardware, smartphones and IoT edge devices remain fundamentally constrained compared to server infrastructure:

- **Battery Energy Capacity:** Electrochemical battery energy density grows at less than 5% to 8% annually, whereas mobile application computational demands (computer vision, speech synthesis, 3D gaming) grow exponentially.
- **Thermal Dissipation Limits:** Passive cooling restricts sustained high-frequency CPU/GPU execution to avoid thermal throttling.
- **Network Variability:** Wireless connections (4G/5G, Wi-Fi) suffer from variable bandwidth, signal fading, packet loss, and handoff delays.

Definition: Mobile Cloud Computing (MCC)

Mobile Cloud Computing (MCC) integrates cloud computing into mobile environments to overcome hardware limitations. Computation-intensive and data-heavy tasks are dynamically extracted from the mobile device and processed on remote cloud infrastructure, delivering results back across wireless networks.

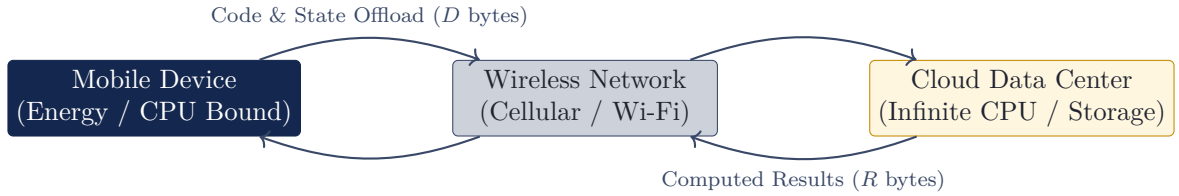


Figure 7.1: Classical Mobile Cloud Computation Offloading Architecture.

7.2.2 Workload Suitability for Offloading

Not all mobile computational tasks are candidates for cloud offloading:

- **Highly Suitable Tasks:** High computational complexity paired with small data exchange (High Compute-to-Data Ratio).
  - Complex mathematical simulations, video encoding/transcoding, optical character recognition (OCR), deep learning inference, natural language processing.
- **Unsuitable Tasks:**
  - *Hard Real-Time Tasks:* Tasks requiring deterministic sub-millisecond response times (e.g., autonomous vehicle collision avoidance, sensor stabilization). Network latency jitter makes remote cloud offloading hazardous.
  - *Data-Heavy Tasks with Minimal Compute:* Tasks where wireless data transmission energy exceeds local computation energy (e.g., streaming uncompressed sensor readings without local processing).
  - *Hardware-Coupled Tasks:* Code requiring direct physical hardware access (accelerometer reads, camera frame grabbing, display rendering).

7.3 Mathematical Formulation of Computation Offloading

7.3.1 System Model and Parameters

Following the formal analytical model established by Kumar and Lu (IEEE Computer):

- C: Total computational complexity of the workload (number of CPU instructions).
- M: Execution speed of the mobile device processor (instructions per second).
- F: Cloud acceleration factor ($F \gg 1$), such that the cloud executes at speed $F \cdot M$ instructions per second.
- D: Volume of application data and program state transferred over the wireless uplink (bytes).

- B: Usable wireless transmission bandwidth (bytes per second).
- P_c: Power consumed by the mobile device CPU during active local computation (Watts).
- P_{tr}: Power consumed by the mobile wireless transceiver during active transmission (Watts).
- P_i: Power consumed by the mobile device in the idle state while awaiting cloud responses (Watts).

7.3.2 Energy Consumption Derivations

Local Execution Energy (E_{local})

The time required to compute the workload locally on the mobile device is:

$$t_{\text{local}} = \frac{C}{M} \tag{7.1}$$

The total electrical energy consumed by local processing is:

$$E_{\text{local}} = P_c \times t_{\text{local}} = P_c \left(\frac{C}{M}\right) \tag{7.2}$$

Cloud Offloading Energy (E_{cloud})

When offloading, the mobile device incurs energy in two sequential stages:

1. **Transmission Stage:** Transmitting D bytes across the network at bandwidth B:

$$t_{\text{tx}} = \frac{D}{B}, \quad E_{\text{tx}} = P_{\text{tr}} \times t_{\text{tx}} = P_{\text{tr}} \left(\frac{D}{B}\right) \tag{7.3}$$

2. **Idle Wait Stage:** Awaiting computation completion while the cloud processes the workload at speed $F \cdot M$:

$$t_{\text{cloud}} = \frac{C}{F \cdot M}, \quad E_{\text{idle}} = P_i \times t_{\text{cloud}} = P_i \left(\frac{C}{F \cdot M}\right) \tag{7.4}$$

Summing the transmission and idle listening energies (assuming negligible result return payload):

$$E_{\text{cloud}} = E_{\text{tx}} + E_{\text{idle}} = P_{\text{tr}} \left(\frac{D}{B}\right) + P_i \left(\frac{C}{F \cdot M}\right) \tag{7.5}$$

Key Result: Energy Conservation Offloading Threshold Theorem

Offloading saves energy if and only if $E_{\text{cloud}} < E_{\text{local}}$, yielding net savings $\Delta E > 0$:

$$\Delta E = E_{\text{local}} - E_{\text{cloud}} = P_c \left(\frac{C}{M}\right) - \left[P_{\text{tr}} \left(\frac{D}{B}\right) + P_i \left(\frac{C}{F \cdot M}\right)\right] > 0 \tag{7.6}$$

Rearranging to isolate the transmission time $t_{\text{tx}} = \frac{D}{B}$:

$$P_{\text{tr}} \left(\frac{D}{B}\right) < \frac{C}{M} \left(P_c - \frac{P_i}{F}\right) \tag{7.7}$$

$$\frac{D}{B} < \frac{C}{M} \left(\frac{P_c - P_i/F}{P_{\text{tr}}}\right) \tag{7.8}$$

Physical Meaning: Offloading is energy-efficient if the time to transmit the input data (D/B) is strictly less than the local compute duration (C/M) scaled by the relative power differential ratio $\frac{P_c - P_i/F}{P_{\text{tr}}}$.

Solved Example: Offloading Energy Savings Calculation (Lecture Numerical Example)

Given: A mobile device executes a computer vision task with parameters:

- Workload complexity: $C = 1.2 \times 10^9$ instructions.
- Mobile compute speed: $M = 6 \times 10^8$ inst/s (600 MIPS).
- Cloud speedup multiplier: $F = 10$ ($F \cdot M = 6 \times 10^9$ inst/s).
- Data to transmit: $D = 3$ MB.
- Network throughput: $B = 2$ MB/s.
- Power coefficients: $P_c = 1.8$ W, $P_{\text{tr}} = 1.4$ W, $P_i = 0.2$ W.

Formula:

$$\Delta E = P_c \left(\frac{C}{M}\right) - P_{\text{tr}} \left(\frac{D}{B}\right) - P_i \left(\frac{C}{F \cdot M}\right) \tag{7.9}$$

Step-by-step substitution:

1. **Local Execution Evaluation:**

$$t_{\text{local}} = \frac{1.2 \times 10^9}{6 \times 10^8} = 2.0 \text{ seconds} \tag{7.10}$$

$$E_{\text{local}} = 1.8 \text{ W} \times 2.0 \text{ s} = 3.60 \text{ Joules} \tag{7.11}$$

2. **Cloud Offloading Evaluation:**

$$t_{\text{tx}} = \frac{3 \text{ MB}}{2 \text{ MB/s}} = 1.5 \text{ seconds} \tag{7.12}$$

$$E_{\text{tx}} = 1.4 \text{ W} \times 1.5 \text{ s} = 2.10 \text{ Joules} \tag{7.13}$$

$$t_{\text{cloud}} = \frac{2.0 \text{ s}}{10} = 0.2 \text{ seconds} \tag{7.14}$$

$$E_{\text{idle}} = 0.2 \text{ W} \times 0.2 \text{ s} = 0.04 \text{ Joules} \tag{7.15}$$

$$E_{\text{cloud}} = 2.10 + 0.04 = 2.14 \text{ Joules} \tag{7.16}$$

3. **Net Energy Differential:**

$$\Delta E = E_{\text{local}} - E_{\text{cloud}} = 3.60 \text{ J} - 2.14 \text{ J} = +1.46 \text{ Joules} \tag{7.17}$$

Final Answer: Offloading is highly beneficial, saving **1.46 Joules** of battery energy (40.56% reduction in power consumption).

7.4 Cloudlets and Mobile Edge Architecture

7.4.1 The WAN Latency Bottleneck

Connecting a mobile device across cellular base stations and multi-hop public Internet routing to distant hyperscale data centers introduces 80 ms to 200 ms of round-trip latency. This latency violates the strict requirements of cognitive mobile applications (real-time language translation, augmented reality overlays, robotic teleoperation) which demand end-to-end response times under 20 ms.

7.4.2 Three-Tier Cloudlet Architecture

Pioneered by Mahadev Satyanarayanan (Carnegie Mellon University), a **Cloudlet** represents a decentralized computing paradigm:

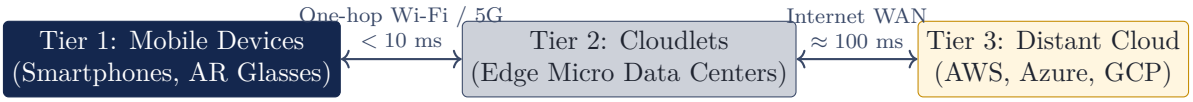


Figure 7.2: Three-Tier Hierarchy: Mobile Device → Cloudlet → Cloud.

Definition: Cloudlet Definition and Characteristics

A **Cloudlet** is a trusted, resource-rich computer or cluster of computers that is well-connected to the Internet and available for use by nearby mobile devices via a high-speed, single-hop wireless local area network (LAN).

- **Zero Administrative Footprint:** Self-managing appliance functioning like a Wi-Fi access point ("data center in a box").
- **Transient State:** Instantiates temporary virtual machine execution contexts for mobile users and flushes state upon user departure.
- **Dynamic VM Synthesis:** The mobile device transmits a compact, compressed VM overlay (delta) containing application binaries to the cloudlet. The cloudlet rapidly synthesizes the overlay onto a base VM image within seconds.

7.5 Geospatial Cloud Computing

7.5.1 Foundations of Geographic Information Systems (GIS)

A Geographic Information System (GIS) captures, stores, analyzes, and visualizes spatially referenced data:

- **Vector Data:** Discrete geometric features represented as Points (e.g., GPS tracking coordinates), Lines (e.g., road networks, rivers), and Polygons (e.g., administrative boundaries, parcels).
- **Raster Data:** Continuous gridded pixel arrays representing continuous phenomena (e.g., satellite multispectral imagery, digital elevation models, thermal heat maps).

7.5.2 Spatial Data Infrastructure (SDI) and OGC Web Standards

The Open Geospatial Consortium (OGC) establishes open web service specifications for interoperable geospatial cloud platforms:

OGC Standard	Full Name	Functional Payload and Operation
WMS	Web Map Service	Renders georeferenced spatial layers into dynamic raster map images (JPEG, PNG) for client display.
WFS	Web Feature Service	Queries and returns raw geometric vector features and attribute tables encoded in GML (Geography Markup Language) or GeoJSON.
WCS	Web Coverage Service	Delivers raw multidimensional raster coverage data with original sensor values (satellite imagery, DEM grids) for analytical computation.
WPS	Web Processing Service	Standardizes inputs and outputs for executing spatial analysis algorithms (e.g., polygon intersection, buffer generation, viewshed analysis) in the cloud.

7.5.3 Cloud Scaling for Big Spatial Data

Geospatial data generation has exploded due to commercial satellite constellations, Earth observation platforms, and ubiquitous mobile smartphone GPS logs. Storing and querying terabytes of spatial geometries in the cloud requires specialized architectures:

- **Spatial Partitioning and Indexing:** Traditional B-trees fail on multi-dimensional spatial coordinates. Geospatial clouds implement:
  - *R-Trees / R*-Trees:* Bounding spatial objects within hierarchical Minimum Bounding Boxes (MBR).
  - *Quadtrees:* Recursive 2D quadrant partitioning of spatial bounding domains.
  - *GeoHash:* Space-filling Z-order curves that encode 2D latitude-longitude coordinates into 1D alphanumeric strings, allowing spatial proximity range queries inside standard NoSQL Key-Value stores (e.g., Apache HBase, AWS DynamoDB).
- **Distributed Spatial MapReduce:** Splitting spatial datasets across Hadoop clusters where Mapper tasks process spatial tiles in parallel, and Reducers execute spatial joins (e.g., finding all GPS points intersecting a polygonal boundary).

7.6 Comprehensive Solved Assignment Problems (Week 7)

Solved Example: Unsuitable Workload for Cloud Offloading

Given: Identify the application least suitable for computation offloading to a remote cloud in MCC.
Step-by-step substitution:

1. Image rendering, ML inference, and video transcoding tolerate seconds of delay and benefit from cloud GPUs.

2. **Emergency collision avoidance** demands deterministic sub-10 ms response. Network latency, packet jitter, and cellular dropouts make remote cloud offloading dangerous.

Final Answer: **Emergency collision avoidance requiring millisecond response**.

Solved Example: Dynamic Resource Scaling Property

Given: An application automatically provisions additional resources during demand surges and releases them when load drops.
Step-by-step substitution:

1. Automated scaling up/down in proportion to dynamic workload demand is the definition of **Rapid Elasticity**.

Final Answer: **Rapid elasticity**.

Solved Example: Dynamic Offloading Optimization Objectives

Given: Dynamic computation offloading algorithms primarily balance which combination of metrics?
Step-by-step substitution:

1. Offloading decisions evaluate execution delay, mobile battery power consumption, and network bandwidth utilization.

Final Answer: **Execution time, energy consumption, and resource utilization**.

Solved Example: Low-Latency Infrastructure for Mobile AR

Given: An augmented reality app requires object recognition with end-to-end response time under 20 ms.
Step-by-step substitution:

1. Distant clouds introduce $\geq$ 100 ms WAN latency.
2. A nearby **Cloudlet** reached via single-hop Wi-Fi provides low-latency compute capability within the required 20 ms window.

Final Answer: **A nearby cloudlet connected through a high-speed local network**.

Solved Example: Healthcare Cloud Security Requirement

Given: Which security requirement is paramount to prevent unauthorized access to cloud-hosted medical records?
Step-by-step substitution:

1. Preventing unauthorized disclosure of confidential medical records is the core objective of **Data Confidentiality**.

Final Answer: **Data confidentiality**.

Solved Example: Primary Problem Solved by Fog/Edge Computing

Given: What fundamental limitation of traditional centralized clouds is resolved by fog computing?
Step-by-step substitution:

1. Transmitting data to geographically distant hyperscale facilities incurs significant network transit delay. Fog computing places compute near devices to eliminate this **high network latency**.

Final Answer: **High network latency caused by distant cloud data centers**.

Solved Example: Fog Nodes Characteristics Statement Analysis

Given: Identify the FALSE statement regarding fog nodes.
Step-by-step substitution:

1. Fog nodes filter and aggregate sensor data (True).
2. Fog nodes are positioned close to IoT devices (True).
3. Fog nodes *permanently store all enterprise data* (FALSE). Fog nodes have limited storage capacity and flush transient data; permanent long-term historical storage resides in the centralized cloud.

Final Answer: **They permanently store all enterprise data**.

Solved Example: OGC Service for Vector Geometries

Given: Which Open Geospatial Consortium service delivers raw vector geographic features (points, lines, polygons)?
Step-by-step substitution:

1. WMS returns rendered map images.
2. **Web Feature Service (WFS)** delivers raw vector spatial geometry features and attributes.

Final Answer: **Web Feature Service (WFS)**.

Solved Example: Offloading Energy Break-Even Condition

Given: State the algebraic condition under which offloading saves mobile battery energy.
Step-by-step substitution:

1. From Kumar and Lu's model, $\Delta E = E_{\text{local}} - E_{\text{cloud}} > 0$.
2. This directly requires:

$$\frac{D}{B} < \frac{C}{M}\left(\frac{P_c - P_i/F}{P_{\text{tr}}}\right) \tag{7.18}$$

Final Answer: $\frac{D}{B} < \frac{C}{M}\left(\frac{P_c - P_i/F}{P_{\mathbf{tr}}}\right)$.

Solved Example: Spatial Indexing Primitives in Cloud GIS

Given: Which spatial indexing mechanism uses recursive 2D space partitioning to index geospatial datasets?
Step-by-step substitution:

1. A **Quadtree** recursively subdivides a two-dimensional spatial region into four quadrants (nodes) to index coordinates efficiently.

Final Answer: **Quadtree**.

Containers, Docker Virtualization, and Sensor-Cloud Infrastructure

8.1 Introduction and Week Overview

Week 8 transitions from heavy hypervisor-based hardware virtualization into lightweight operating-system-level virtualization (Containers and Docker), followed by the architecture of Sensor-Cloud computing. The key modules examined include:

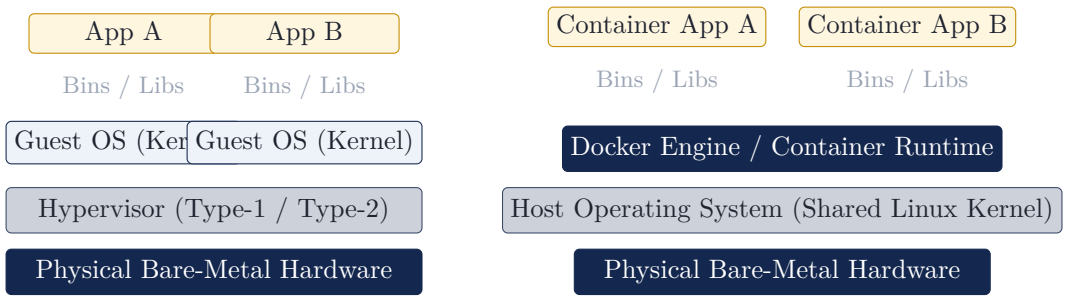
1. **Containers vs Virtual Machines:** Kernel-sharing mechanics, isolation primitives, architectural tradeoffs, and performance benchmarking.
2. **Docker Ecosystem and Architecture:** The Docker engine, copy-on-write image layering (UnionFS/OverlayFS), Dockerfile build directives, and container lifecycle CLI commands.
3. **Container Orchestration Primitives:** High-level role of Kubernetes and distributed configuration state management via `etcd`.
4. **Sensor-Cloud Framework:** Bridging Wireless Sensor Networks (WSNs) with cloud elasticity; virtual sensors, multi-tenant provisioning topologies, and layered middleware architectures.

8.2 Containers vs Virtual Machines: Architectural Divergence

8.2.1 Structural Comparison

Traditional virtualization relies on a hypervisor (Type-1 or Type-2) to emulate physical hardware (CPUs, chipsets, network interface cards, memory controllers, BIOS). Each guest Virtual Machine must run a complete, independent guest operating system including its own kernel, device drivers, and system services.

In contrast, **Operating System-Level Virtualization (Containerization)** relies on kernel-level isolation features within a single host operating system kernel to partition user space into isolated execution environments known as **Containers**.



(a) Virtual Machines (Hardware Virtualization) **(b) Containers (OS-Level Virtualization)**

Figure 8.1: Architectural Comparison: Virtual Machines vs Application Containers.

8.2.2 Quantitative Tradeoff Analysis

Metric / Dimension	Virtual Machines (VMs)	Containers (Docker)
Isolation Level	High (Hardware-enforced by MMU, VT-x, and separate kernel ring 0).	Medium-High (Process-level isolation enforced by host kernel namespaces and cgroups).
Kernel Instances	Each VM executes its own discrete OS kernel.	All containers share the single host OS kernel.
Startup Latency	Minutes (30 s to 180 s) due to full virtual BIOS and OS boot.	Milliseconds to seconds (< 1 s) as processes spawn directly.
Disk Footprint	Gigabytes (2 GB to 40 GB) per instance (entire OS image).	Megabytes (10 MB to 500 MB) containing only app dependencies.
CPU/RAM Overhead	Significant (10% to 25%) dedicated to running guest OS daemons.	Minimal ($< 1\%$), near bare-metal execution performance.
Density	Tens of VMs per physical bare-metal host.	Hundreds to thousands of containers per physical host.
Portability	Heavy image formats (VMDK, VHD, QCOW2) tied to hypervisor formats.	Standard OCI image formats portable across any Linux distribution.

8.3 The Docker Ecosystem and Architecture

8.3.1 Docker Engine Components

Docker is an open-source platform that automates the deployment, scaling, and management of applications inside portable container environments. The Docker architecture consists of:

- **Docker Daemon (`dockerd`):** The persistent background service running on the host that builds images, manages container states, attaches virtual networks, and coordinates storage volumes.
- **Docker Client (`docker`):** The command-line interface (CLI) tool through which users interact with `dockerd` via REST API endpoints over UNIX domain sockets (`/var/run/docker.sock`) or TCP.

- **Docker Images:** Read-only, immutable templates composed of stacked union filesystem layers.
- **Docker Containers:** Runnable instances of a Docker image. When a container starts, a thin, writable copy-on-write (CoW) layer is mounted on top of the immutable image layers.
- **Docker Registry (Docker Hub):** Centralized repositories for publishing, distributing, and pulling container images.

8.3.2 Copy-on-Write (CoW) and Storage Layering

Docker utilizes union filesystems (such as **OverlayFS** or **AUFS**) to stack directories and present them as a single cohesive unified filesystem:

- When multiple containers instantiate the identical base image (`ubuntu:22.04`), they all share the identical read-only physical disk blocks in memory.
- If a container modifies an existing file, the underlying file is copied up from the read-only layer into the container's top-level writable layer before modification (*Copy-on-Write*). This ensures zero mutation of base images.

8.3.3 Dockerfile Syntax and Construction

A **Dockerfile** is a declarative text manifest specifying the sequential instructions required to assemble a Docker image:

```
# Base image layer
FROM python:3.9-slim

# Set execution working directory
WORKDIR /app

# Copy dependency manifest and install packages
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source code into image layer
COPY . .

# Expose internal listening port
EXPOSE 5000

# Specify default container entry command
CMD ["python", "app.py"]
```

8.3.4 Complete Docker Command Lifecycle

Docker Command	Operational Execution Description
`docker build -t myapp:1.0 .`	Reads the local Dockerfile, executes instructions sequentially, caches intermediate layers, and tags the resulting image.
`docker run -d -p 8080:5000 myapp:1.0`	Instantiates a container in detached mode (`-d`), binding host port 8080 to container port 5000 (`-p`).
`docker ps -a`	Lists all containers (both actively running and terminated/exited).
`docker stop <container_id>`	Sends SIGTERM followed by SIGKILL to gracefully terminate container processes.
`docker rm <container_id>`	Deletes a stopped container instance and purges its writable layer.
`docker rmi <image_id>`	Permanently deletes an unreferenced container image from the local storage cache.
`docker network ls`	Displays all configured virtual bridge, overlay, and host networks.
`docker exec -it <id> /bin/bash`	Spawns an interactive bash shell session inside an actively running container.

8.4 Container Orchestration: Kubernetes and etcd

When applications scale to thousands of containers spanning hundreds of bare-metal servers, manual container management becomes impossible. **Kubernetes** automates container scheduling, automated rollouts, service discovery, load balancing, and self-healing.

Definition: Role of `etcd` in Kubernetes Clusters

`etcd` is a strongly consistent, distributed, and highly available key-value datastore based on the **Raft consensus algorithm**.
In Kubernetes, `etcd` serves as the ultimate single source of truth for the entire cluster. It persistently stores:

- Cluster configuration states, namespaces, and node registrations.
- Pod definitions, ReplicaSets, and Deployments.
- Secrets, ConfigMaps, and cluster metadata.

The Kubernetes control plane components (API Server, Scheduler, Controller Manager) are stateless; they coordinate state transitions strictly by reading from and writing to `etcd`.

8.5 Sensor-Cloud Infrastructure

8.5.1 Limitations of Traditional Wireless Sensor Networks (WSNs)

Traditional Wireless Sensor Networks (WSNs) consist of small sensor motes deployed to monitor physical environments (temperature, moisture, seismic activity). However, conventional WSNs exhibit severe structural limitations:

- **Isolated Application Silos:** A sensor network deployed for soil moisture monitoring is typically accessible only to a single proprietary backend application.
- **Resource Exhaustion:** Sensor nodes possess limited battery life, micro-controllers with few kilobytes of RAM, and low-power radios.
- **No Multi-Tenancy:** Multiple users with differing analytical queries cannot dynamically share the same physical sensor hardware simultaneously.

8.5.2 The Sensor-Cloud Paradigm

Definition: Sensor-Cloud Definition

Sensor-Cloud is an infrastructure that virtualizes physical Wireless Sensor Networks into dynamically manageable, reconfigurable, and multi-tenant **Virtual Sensors** provisioned on-demand over the cloud.

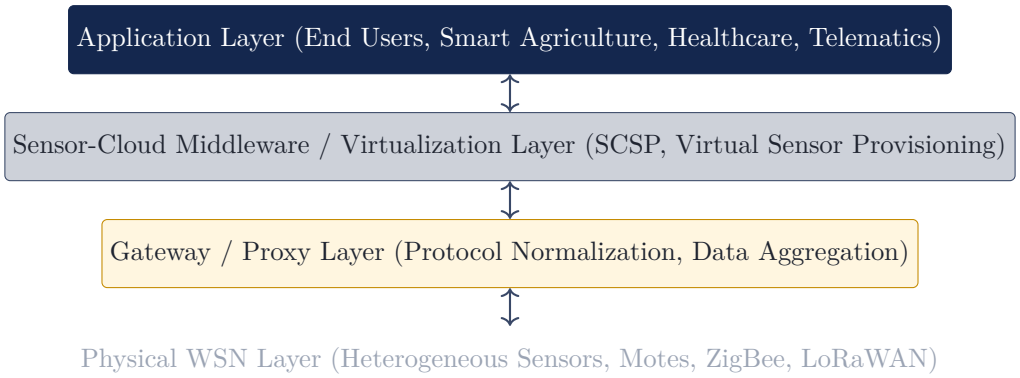


Figure 8.2: Layered Architecture of Sensor-Cloud Computing.

8.5.3 Stakeholders and Economic Model

Sensor-Cloud establishes a three-party multi-tenant marketplace:

1. **Physical Sensor Owners:** Entities that procure and deploy physical sensor hardware in the field. They register their WSNs with the Sensor-Cloud provider to monetize idle sensing capacity.
2. **Sensor-Cloud Service Provider (SCSP):** The cloud entity that maintains the virtualization middleware, registers physical sensors, provisions virtual sensors, negotiates SLAs, and handles metering and billing.
3. **End Users:** Consumers or enterprises who provision virtual sensors on-demand for specific temporal durations and geographic boundaries without purchasing physical hardware.

8.5.4 Virtual Sensor Configurations and Mapping Topologies

A **Virtual Sensor** is a software abstraction representing a sensing unit provisioned in the cloud. It decouples the user application from the underlying physical sensor hardware. The virtualization engine supports three primary configurations:

- **One-to-One Mapping** $(1:1)$**:** A single virtual sensor directly represents a single physical sensor mote.
- **Many-to-One Mapping** $(N:1)$**:** Multiple physical sensor motes are aggregated into a single virtual sensor. For example, averaging temperature readings from 20 distributed agricultural sensor nodes across a field into a single virtual temperature feed.
- **One-to-Many Mapping** $(1:N)$**:** A single high-capability physical sensor is dynamically partitioned into multiple virtual sensors serving different end users simultaneously with distinct sampling frequencies and delivery thresholds.
- **Spatio-Temporal Virtual Sensors:** Virtual sensors synthesized dynamically to cover an arbitrary geographical polygon $\mathcal{R}$ over a designated time window $[t_1, t_2]$, collecting and synthesizing telemetry from all physical sensors falling inside $\mathcal{R}$.

8.6 Comprehensive Solved Assignment Problems (Week 8)

Solved Example: Docker Image Deletion Command

Given: Which command deletes a Docker image that is no longer required?
Step-by-step substitution:

1. `docker rm` removes stopped *containers*.
2. `docker rmi` (**r**e**m**ove **i**mage) purges container *images*.

Final Answer: `docker rmi`.

Solved Example: Listing Docker Virtual Networks

Given: Identify the command used to display all available Docker networks on the host.
Step-by-step substitution:

1. Docker CLI manages network namespaces via the `network` command group.
2. The command `docker network ls` lists all bridge, host, and overlay networks.

Final Answer: `docker network ls`.

Solved Example: VM vs Container Kernel Isolation Comparison

Given: Which is the most accurate comparison between VMs and containers?
Step-by-step substitution:

1. Virtual Machines virtualize physical hardware, executing complete guest operating systems with independent kernels.
2. Containers share the underlying host OS kernel, providing process isolation via Linux

kernel namespaces and cgroups.

Final Answer: **Containers generally share the host kernel, while VMs virtualize hardware and run separate guest OS kernels**.

Solved Example: Primary Function of etcd in Kubernetes

Given: What is the fundamental responsibility of `etcd` in a Kubernetes cluster?
Step-by-step substitution:

1. `etcd` is a distributed key-value store used to persist cluster configuration data, metadata, and state information across all master control plane components.

Final Answer: **Storing Kubernetes cluster state and configuration data**.

Solved Example: Hypervisor Type Classification

Given: Identify an example of a Type 2 (hosted) hypervisor among: VMware ESXi, Xen, Oracle VirtualBox, Microsoft Hyper-V Server.
Step-by-step substitution:

1. VMware ESXi, Xen, and Hyper-V Server are Type-1 bare-metal hypervisors.
2. **Oracle VirtualBox** runs as an application process on top of a standard host operating system (Windows, Linux, macOS), making it a Type 2 hypervisor.

Final Answer: **Oracle VirtualBox**.

Solved Example: Energy Reduction via VM Consolidation

Given: In a data center of 100 servers, 40 are heavily loaded while 60 operate at very low utilization. What is the most effective energy saving strategy?
Step-by-step substitution:

1. Idle servers consume 50% to 60% of peak power.
2. Consolidating VMs from underutilized servers onto fewer hosts via live migration allows the empty physical machines to be powered down into sleep states, maximizing energy savings.

Final Answer: **Consolidate VMs onto fewer servers and switch idle servers to a low-power state**.

Solved Example: Role of Middleware in Sensor-Cloud

Given: Why is a middleware layer required between Wireless Sensor Networks (WSNs) and cloud applications?
Step-by-step substitution:

1. Physical sensor motes use diverse, proprietary communication protocols and data formats.
2. The middleware abstracts heterogeneous physical sensors, enabling cloud-accessible virtual sensors with standardized APIs.

Final Answer: **To abstract heterogeneous sensors and provide cloud-accessible sensor resources**.

Solved Example: Virtual Sensor Provisioning Topologies

Given: Can multiple physical sensors be aggregated to form a single virtual sensor?
Step-by-step substitution:

1. Yes, Sensor-Cloud supports Many-to-One ($N:1$) mapping, allowing data from multiple geographic sensors to be aggregated into a single virtual sensor feed.

Final Answer: **Yes, Many-to-One mapping is supported**.

Solved Example: Docker Container Isolation Primitives

Given: Which Linux kernel primitives provide resource metering and process isolation for Docker containers?
Step-by-step substitution:

1. **Namespaces** partition system resources (process IDs, network interfaces, mount points).
2. **Control Groups (cgroups)** meter and limit hardware consumption (CPU shares, memory quotas, I/O bandwidth).

Final Answer: **Namespaces and cgroups**.

Solved Example: Economic Benefit of Sensor-Cloud to Sensor Owners

Given: How do physical sensor owners benefit economically in the Sensor-Cloud model?
Step-by-step substitution:

1. Sensor owners register their physical WSNs with the Sensor-Cloud Service Provider (SCSP) and earn recurring revenue from multi-tenant subscriptions without building custom software applications.

Final Answer: **Monetizing deployed physical sensor capacity through multi-tenant usage fees**.

Hierarchical Edge-Fog-Cloud Computing and Cloud Federation

9.1 Introduction and Week Overview

Week 9 explores the continuum between centralized hyperscale data centers and decentralized edge computing. Centralized clouds struggle to process the massive volume, velocity, and variety of data generated by billions of Internet of Things (IoT) devices with low latency. This module covers:

1. **The Edge-Fog-Cloud Continuum:** Cisco's Fog Computing architectural paradigm, multi-tier hierarchical delegation, and latency-sensitive control loops.
2. **Empirical Performance Modeling with iFogSim:** Network bandwidth conservation, execution cost reduction, latency bounds, and healthcare case studies.
3. **Resource Management and Service Placement:** Formal taxonomies of resource management (Data Flow, Control, Tenancy), vertical vs horizontal scalability, and multi-objective task placement.
4. **Cloud Federation and Inter-Cloud Governance:** Overcoming vendor lock-in through Broker Architectures, Peer-to-Peer (P2P) peering, and cross-provider SLA enforcement.

9.2 The Edge-Fog-Cloud Computing Hierarchy

9.2.1 The Need for Decentralization

While hyperscale cloud computing provides elastic computing capacity and virtually infinite storage, routing all raw IoT telemetry to centralized data centers introduces critical bottlenecks:

- **Excessive WAN Latency:** Multi-hop packet routing across public Internet backbones introduces unpredictable delays (100 ms to 300 ms), violating real-time cyber-physical feedback loops.
- **Core Network Congestion:** Streaming millions of high-definition video feeds or high-frequency vibrational sensor telemetry overloads Internet Service Provider (ISP) backbones.
- **Data Privacy and Sovereignty:** Raw audio, video, and biometric streams transmitted across national borders pose regulatory and privacy risks.

9.2.2 Architectural Division of Responsibilities

Introduced by Cisco Systems and formalized in IEEE/ACM distributed systems research, the **Edge–Fog–Cloud** hierarchy divides responsibilities across three distinct tiers:

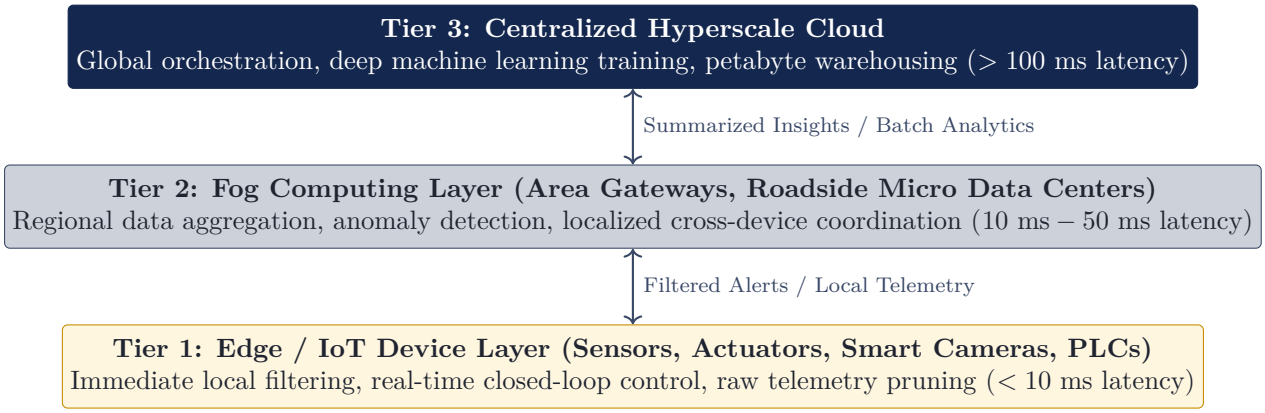


Figure 9.1: Three-Tier Edge-Fog-Cloud Hierarchical Architecture.

System Dimension	Edge Layer	Fog Layer	Cloud Layer
Response Latency	Ultra-low (1 ms – 10 ms)	Low (10 ms – 50 ms)	Moderate to High (100 ms – 500 ms)
Data Scope	Transient, sensor-local	Regional, facility-wide	Global, enterprise-wide
Storage Duration	Seconds (transient buffer)	Hours to days	Years (permanent historical archival)
Compute Hardware	Microcontrollers, ASICs, Raspberry Pi	Micro-servers, edge gateways, high-end routers	Multi-socket server blades, GPU clusters
Data Volume Flow	Ingests 100% of raw telemetry	Filters/aggregates to ≈ 10% of raw data	Receives < 1% (compact statistical summaries)
Network Uplink	Local Wi-Fi, BLE, ZigBee	Regional Ethernet, 5G NR, Fiber	Transoceanic Internet WAN fiber

9.2.3 Empirical Simulation Analysis: The iFogSim Framework

The lecture features research co-authored by Prof. Soumya K. Ghosh and Prof. Rajkumar Buyya utilizing **iFogSim**, an open-source Java simulation toolkit for modeling resource management in IoT, Edge, and Fog environments:

Healthcare Case Study: Smart Ambient Assisted Living

A patient monitoring application measures ECG, blood pressure, and pulse telemetry:

- **Configuration A (Cloud-Only):** Every raw biometric pulse packet is forwarded directly to AWS cloud servers.
- **Configuration B (Cloud-Fog-Edge):** Wearable edge sensors execute lightweight feature extraction. An Area Gateway (Fog Node) runs automated arrhythmia detection algorithms locally. Only abnormal readings trigger an alert that contacts the cloud confirmatory module.

Performance Findings

1. **Network Bandwidth Consumption:** In the Fog configuration, network usage decreases by over 90% because raw sinusoidal signals are evaluated locally; only verified anomaly events traverse the wide-area network.
2. **Operational Execution Cost:** Cloud compute hours are strictly pay-per-use. Processing raw data on customer-owned edge gateways slashes recurring monthly cloud billing fees.
3. **Control Loop Latency:** In the Cloud-only model, closed-loop actuation delays fluctuated between 1500 ms and 4000 ms due to network congestion. In the Fog architecture, control loop latency remained deterministic and fixed below 50 ms.

9.3 Resource Management and Scalability Models

9.3.1 Resource Management Taxonomies

Following the ACM survey by Hong and Varghese highlighted in the lecture, resource management in Fog/Edge computing is partitioned along three axes:

- **Data Flow:** Movement of workloads and telemetry. Workloads may flow upstream (from sensor devices to edge/cloud) or downstream (offloading pre-trained AI weights from cloud to edge inferencing engines).
- **Control Hierarchy:**
  - *Centralized Control:* A master coordinator governs scheduling across all edge devices.
  - *Distributed / Autonomous Control:* Edge and fog nodes negotiate tasks peer-to-peer using localized game theory or consensus algorithms.
- **Tenancy:** Single-tenant dedicated hardware vs multi-tenant virtualized containers running concurrent multi-user tasks.

9.3.2 Vertical vs Horizontal Scalability

When a computing node experiences resource exhaustion:

Definition: Vertical Scalability (Scale-Up) vs Horizontal Scalability (Scale-Out)

Vertical Scalability (Scale-Up): Increasing the physical or virtual capacity of an existing node by assigning additional CPU cores, expanding system RAM, or upgrading disk I/O throughput within the same machine.
Horizontal Scalability (Scale-Out): Adding new independent nodes or servers into the computing pool and distributing incoming workload requests across the expanded cluster via a load balancer.

9.4 Cloud Federation and Inter-Cloud Governance

9.4.1 Motivation for Cloud Federation

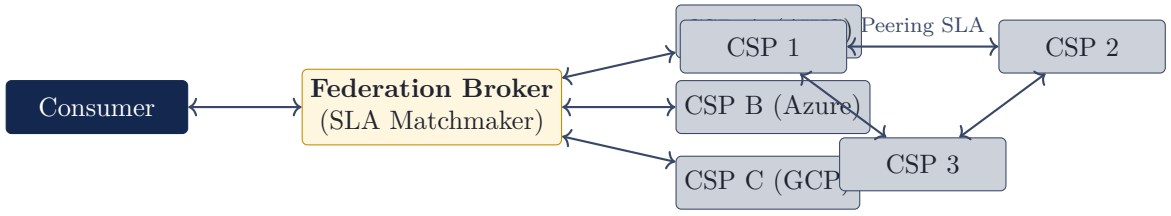
Relying on a single Cloud Service Provider (CSP) exposes enterprises to catastrophic risks:

- **Vendor Lock-In:** Proprietary APIs, incompatible storage schemas, and exorbitant egress bandwidth fees trap enterprise applications.
- **Geographic Blackouts:** Outages in a single provider's regional data center can incapacitate global business operations.
- **Capacity Exhaustion:** Sudden global demand spikes can surpass a single provider's available unallocated capacity.

Definition: Cloud Federation

Cloud Federation is the seamless aggregation and dynamic coordination of computing, storage, and networking resources from multiple autonomous, heterogeneous Cloud Service Providers to satisfy consumer requirements under a unified SLA.

9.4.2 Federation Architectural Topologies



(a) Broker-Based Federation **(b) Peer-to-Peer (P2P) Federation**

Figure 9.2: Architectural Models of Cloud Federation.

1. **Broker Architecture (Centralized Federation):** An independent entity (the Cloud Federation Broker) acts as a neutral intermediary between cloud consumers and multiple CSPs:
   - Discovers available capacity and catalogs service pricing across diverse providers.
   - Evaluates consumer QoS and latency constraints, selecting the optimal provider.
   - Negotiates multi-provider contractual SLAs and orchestrates cross-cloud billing.
2. **Peer-to-Peer (P2P) Federation:** Cloud service providers negotiate direct peering agreements without a central broker. When Provider A experiences localized capacity exhaustion, it transparently subcontracts surplus workload to Provider B under pre-agreed bilateral cross-charging tariffs.

9.5 Comprehensive Solved Assignment Problems (Week 9)

Solved Example: Multi-Tier Placement for Industrial Anomaly Detection

Given: A factory requires machine-level anomaly detection within milliseconds and plant-wide data aggregation before sending summaries to the cloud. What is the optimal placement?

Step-by-step substitution:

1. Millisecond anomaly detection on individual machines requires processing at the **Edge**.

2. Plant-wide aggregation across multiple machines requires intermediate coordination at **Fog** nodes.

3. Final long-term historical reporting is handled in the **Cloud**.

Final Answer: **Machine-level processing at the edge and plant-level processing at fog nodes**.

Solved Example: Smart City Multi-Tier Video Analytics Architecture

Given: Object detection occurs near cameras, regional congestion patterns are analyzed at roadside micro-datacenters, and long-term city trends are processed in the cloud. Which architecture represents this system?
Step-by-step substitution:

1. Processing near cameras → Edge.

2. Roadside micro-datacenters → Fog.

3. Long-term trends in the central facility → Cloud.

4. The complete multi-tiered paradigm is **Edge–Fog–Cloud**.

Final Answer: **Edge–Fog–Cloud architecture**.

Solved Example: Raw Data Volume Transmitted to Central Infrastructure

Given: Rank the architectural strategies by raw data volume transmitted toward the central cloud, from lowest to highest.
Step-by-step substitution:

1. **Edge processing** filters and prunes telemetry immediately at the sensor, transmitting the least volume.

2. **Fog processing** aggregates data across a local area, transmitting moderate volume.

3. **Cloud-only processing** streams all raw uncompressed data over the WAN, transmitting the highest volume.

Final Answer: **Edge processing → Fog processing → Cloud-only processing**.

Solved Example: Scaling Mechanism Classification

Given: A fog node running low on memory is provisioned additional RAM and CPU cores on that same physical machine. Which scalability approach is this?
Step-by-step substitution:

1. Upgrading the CPU and RAM capacity of an existing machine without adding new nodes is **Vertical Scalability** (Scale-Up).

Final Answer: **Vertical scalability**.

Solved Example: Division of Responsibilities in Edge-Fog-Cloud

Given: Which statement best describes the division of responsibilities in a hierarchical Edge-Fog-Cloud system?
Step-by-step substitution:

1. Edge executes immediate local actions.
2. Fog coordinates regional resources and aggregates telemetry.
3. Cloud performs global optimization and long-term strategic analytics.

Final Answer: **Edge handles immediate local actions, Fog coordinates regional resources, and Cloud performs global orchestration**.

Solved Example: Split Architecture for Video Analytics

Given: A video analytics application requires first-stage inference within 10 ms of the camera, while heavy model retraining requires massive computing power.
Step-by-step substitution:

1. Sub-10 ms inference must run at the Edge/Fog close to the camera.
2. Heavy model training runs in the centralized Cloud.

Final Answer: **First-stage inference on an edge/fog node and model training in the cloud**.

Solved Example: Federation Broker Responsibilities

Given: What is the primary role of a Cloud Federation Broker?
Step-by-step substitution:

1. The Federation Broker mediates between consumers and multiple CSPs, discovering available resources, matching SLA terms, and routing workloads across cloud providers.

Final Answer: **Discovers, negotiates, and coordinates services across multiple independent cloud providers**.

Solved Example: Limitation of Edge Devices in Heavy Analytics

Given: Why can't edge devices replace centralized cloud data centers for large-scale historical model training?
Step-by-step substitution:

1. Edge devices have constrained CPU, GPU, memory, and battery capacity, making them incapable of processing multi-terabyte datasets required for model training.

Final Answer: **Edge devices possess limited computational power, memory, and storage capacity**.

Solved Example: Peer-to-Peer vs Broker Federation

Given: In a Peer-to-Peer (P2P) cloud federation model, how are resources negotiated?
Step-by-step substitution:

1. Participating cloud providers establish direct, bilateral peering agreements to share surplus capacity without relying on a centralized intermediary broker.

Final Answer: **Directly between participating cloud providers through bilateral peering agreements**.

Solved Example: Bandwidth Conservation via Fog Preprocessing

Given: How does fog preprocessing conserve WAN network bandwidth?
Step-by-step substitution:

1. By filtering, aggregating, and compressing raw sensor streams locally, preventing redundant telemetry from being transmitted across the core Internet backbone.

Final Answer: **By filtering and aggregating telemetry locally before wide-area transmission**.

Virtual Machine Migration, Container Primitives, and Cloud Orchestration

10.1 Introduction and Week Overview

Week 10 concludes the technical core of the NPTEL Cloud Computing curriculum by investigating dynamic runtime relocation of compute instances and the deep operating-system primitives governing modern container engines. The curriculum covers:

1. **Virtual Machine Migration Taxonomy:** Motivations (load balancing, hardware maintenance, energy consolidation, fault tolerance), what to migrate (CPU state, memory pages, network connections, storage), and Cold vs Live migration.
2. **Live Migration Paradigms:**
   - *Pre-Copy Iterative Migration:* Push phase, iterative dirty-page tracking, mathematical convergence criteria ($D < R$), and stop-and-copy downtime.
   - *Post-Copy Migration:* Rapid CPU context handover, target-first execution, and on-demand network page faulting (Pull phase).
3. **Linux Kernel Container Primitives:** The deep mechanics of Linux Namespaces (process isolation) versus Control Groups / cgroups (resource quotas and metering).
4. **Docker Storage Persistence:** Ephemeral writable layers vs Docker Named Volumes.

10.2 Virtual Machine Migration Fundamentals

10.2.1 Motivation and Operational Drivers

In a production cloud data center, virtual machines cannot remain permanently pinned to a single physical blade. Dynamic instance relocation is required for:

- **Dynamic Load Balancing:** Relieving resource congestion when a physical server experiences CPU, RAM, or I/O saturation.
- **Zero-Downtime Hardware Maintenance:** Evacuating all active tenant VMs from a physical server prior to hardware upgrades, firmware flashes, or operating system patches.
- **Green Energy Consolidation:** Packing lightly loaded VMs onto fewer physical servers during off-peak hours and powering off redundant hosts into low-power ACPI sleep states.
- **Proactive Fault Tolerance:** Migrating VMs away from hardware nodes exhibiting escalating memory ECC error rates or thermal warnings before fatal crashes occur.

10.2.2 The Migration State Tuple

A virtual machine is fully encapsulated as a state tuple:

$$\mathcal{S}_{\text{VM}} = \langle \mathcal{C}_{\text{CPU}}, \mathcal{M}_{\text{RAM}}, \mathcal{D}_{\text{Storage}}, \mathcal{N}_{\text{Network}}, \mathcal{I}_{\text{I/O}} \rangle \tag{10.1}$$

- **CPU Context ($\mathcal{C}_{\text{CPU}}$):** Register states, Program Counter (PC), translation lookaside buffer (TLB) state, and virtual interrupt controllers. Size: Kilobytes; transfer time: milliseconds.
- **Main Memory ($\mathcal{M}_{\text{RAM}}$):** The entire physical memory space allocated to the guest OS (2 GB to 512 GB). Migrating active memory while guest processes continuously modify memory pages is the central algorithmic challenge of live migration.
- **Persistent Storage ($\mathcal{D}_{\text{Storage}}$):** In enterprise clouds, storage disks reside on shared network-attached storage (NAS, SAN, or Ceph). The virtual disk is accessible from both source and destination physical hosts, avoiding the need to transfer multi-terabyte disk images across the migration link.
- **Network State ($\mathcal{N}_{\text{Network}}$):** The VM must retain its IP address and open TCP connections. Upon migration completion to a new switch port, the target hypervisor emits a **Gratuitous ARP (GARP)** broadcast, updating network switch forwarding tables (CAM tables) without severing established client sessions.

10.3 Cold vs Live Migration Taxonomy

Dimension	Cold (Offline / Non-Live) Migration	Live (Hot / Online) Migration
VM State During Transfer	Powered off, halted, or suspended.	Actively running and processing user requests.
Service Downtime	Long ($T_{\text{downtime}} = T_{\text{transfer}}$). Equal to total migration duration.	Near-zero ($T_{\text{downtime}} \ll T_{\text{transfer}}$), typically $50\text{ ms} - 500\text{ ms}$.
Implementation Complexity	Trivial (copy memory/disk snapshot file and boot on target).	High (requires continuous dirty-page tracking or on-demand page pulling).
Application Impact	Unacceptable for latency-sensitive, real-time, or transactional services.	Undetectable by end users and active TCP sessions.

10.3.1 Cold Migration Transfer Duration Formulation

In cold migration, the entire memory footprint V_m must be transferred over a network connection with usable sustained throughput R:

$$T_{\text{transfer}} = \frac{V_m}{R} \tag{10.2}$$

Solved Example: Cold VM Migration Duration Calculation

Given: A virtual machine has memory $V_m = 32$ GB. The migration link provides a sustained throughput $R = 16$ MB/s. Calculate the transfer duration for a non-live migration. (Use 1 GB $= 2^{30}$ bytes, 1 MB $= 2^{20}$ bytes).

Formula:

$$T = \frac{V_m}{R} \tag{10.3}$$

Step-by-step substitution:

1. Convert gigabytes to megabytes:

$$V_m = 32 \text{ GB} = 32 \times 1024 \text{ MB} = 32,768 \text{ MB} \tag{10.4}$$

2. Compute duration in seconds:

$$T = \frac{32,768 \text{ MB}}{16 \text{ MB/s}} = 2048 \text{ seconds} \tag{10.5}$$

3. Convert to minutes:

$$T_{\text{minutes}} = \frac{2048}{60} \approx 34.133 \text{ minutes} \quad (\approx 34 \text{ minutes}) \tag{10.6}$$

Final Answer: The transfer duration is approximately **34 minutes** (2048 seconds).

10.4 Live Migration Paradigms: Pre-Copy vs Post-Copy

Live migration decouples total migration time from service downtime through two primary algorithmic strategies:

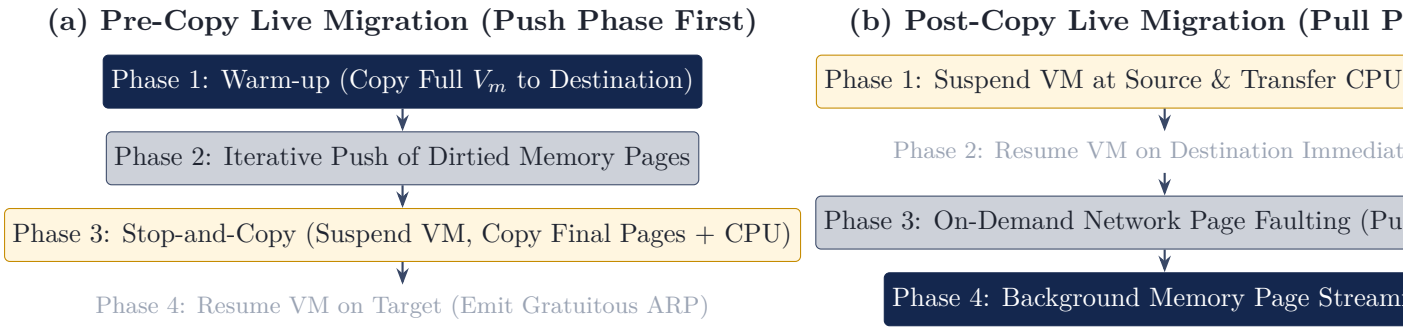


Figure 10.1: Algorithmic Comparison: Pre-Copy vs Post-Copy Live Migration.

10.4.1 Pre-Copy Iterative Live Migration

Employed by Xen, KVM, and VMware ESXi, Pre-Copy migration relies on a **Push** model:

1. **Pre-Copy Initialization:** The source VM continues executing normally while the hypervisor copies the entire memory space (V_m) to the target host.

2. **Iterative Pre-Copying:** As the guest OS executes, it modifies memory pages (creating "dirty pages"). The hypervisor uses shadow page tables or hardware dirty-bit tracking to log modified pages and re-transmits them in successive iterations.

3. **Mathematical Convergence Criterion:** Let D denote the memory dirtying rate (MB/s) and R the network transmission rate (MB/s). The data remaining to be sent in iteration $k+1$ is:

$$V_{k+1} = V_k \times \left(\frac{D}{R}\right) = V_m \cdot r^{k+1}, \quad \text{where } r = \frac{D}{R} \tag{10.7}$$

Key Result: Pre-Copy Convergence Condition

The iterative series converges to zero if and only if the dirtying rate is strictly less than the network transmission rate:

$$r = \frac{D}{R} < 1 \iff D < R \tag{10.8}$$

If the VM executes a write-intensive workload where $D \geq R$, memory is dirtied faster than the network can transmit it. The iterative process will **never converge**, dramatically increasing migration duration and network bandwidth consumption!

4. **Stop-and-Copy Phase:** When remaining dirty pages drop below a threshold (or a maximum round limit is reached), the source VM is suspended. The remaining dirty pages and CPU execution state are transmitted.

5. **Resume Phase:** The VM is resumed on the target host.

10.4.2 Post-Copy Live Migration

Post-Copy reverses the sequence:

1. The VM is suspended immediately on the source host.

2. Its minimal processor state ($\mathcal{C}_{\text{CPU}}$) is transferred to the destination.

3. The VM is **resumed on the destination immediately**!

4. When the VM accesses a memory page not yet present on the target, a network page fault traps execution, pulling the required page from the source host on demand (**Pull Phase**).

5. Remaining pages are pushed in the background.

6. **Key Advantage:** Every memory page is transmitted across the network **at most once**, guaranteeing that total data transferred equals exactly V_m, minimizing energy consumption and eliminating convergence failure.

10.5 Linux Kernel Container Primitives

Modern container runtimes (Docker, containerd, CRI-O) do not use hypervisors; they construct isolated user-space sandboxes using two fundamental Linux kernel subsystems:



Figure 10.2: The Two Pillars of Linux Container Virtualization.

10.5.1 Linux Namespaces: Isolation of System Views

Namespaces wrap global system resources in an abstraction that makes it appear to processes within the namespace that they possess their own isolated instance of the resource:

Namespace	Flag	Isolated System Resource
PID	`CLONE_NEWPID`	Process IDs. A container process sees itself as PID 1 (init), isolated from host PIDs.
NET	`CLONE_NEWNET`	Network devices, IP routing tables, port bindings, firewall rules (iptables).
MNT	`CLONE_NEWNS`	Filesystem mount points. Isolates container rootfs from the host root directory.
IPC	`CLONE_NEWIPC`	System V Inter-Process Communication and POSIX message queues.
UTS	`CLONE_NEWUTS`	Hostname and NIS domain name.
USER	`CLONE_NEWUSER`	User and Group ID mappings. Allows root inside a container to map to an unprivileged user outside.
CGROUP	`CLONE_NEWCGROUP`	Cgroup hierarchy visibility.

10.5.2 Control Groups (cgroups): Resource Allocation and Quotas

While namespaces prevent a container from seeing other processes, they do not restrict resource consumption. Without cgroups, a malicious container could execute an infinite loop or memory leak, starving the host. **Control Groups (cgroups)** meter, prioritize, and enforce hard limits on:

- **CPU Shares and Quotas (`cpu.cfs_quota_us`):** Enforcing strict CPU bandwidth allocations.
- **Memory Limits (`memory.limit_in_bytes`):** Enforcing hard memory ceilings; triggers Out-Of-Memory (OOM) killer if exceeded.
- **Block I/O (`blkio`):** Throttling disk read/write bandwidth and IOPS.
- **Device Access (`devices`):** Restricting access to physical character and block devices.

10.5.3 Docker Storage Persistence: Named Volumes

Docker container writable layers are ephemeral; deleting a container purges all files written to its top layer. To persist data (e.g., database files):

- **Docker Named Volumes:** Persistent storage directories managed by Docker located on the host filesystem (`/var/lib/docker/volumes/`).

- **Lifecycle Decoupling:** If a database container is stopped or deleted, data residing inside an attached Docker named volume **remains completely intact and unaffected**.

10.6 Comprehensive Solved Assignment Problems (Week 10)

Solved Example: Condition Increasing Live Migration Duration

Given: During live VM migration, which condition is most likely to increase the total migration time?
Step-by-step substitution:

1. In pre-copy live migration, the hypervisor iteratively copies dirty pages.
2. If the guest application modifies memory rapidly (**high rate of memory-page modification**, $D \geq R$), each iteration generates massive volumes of dirty pages, preventing rapid convergence and increasing total migration time.

Final Answer: **High rate of memory-page modification by the VM**.

Solved Example: Post-Copy Live Migration Mechanics

Given: Which live VM migration technique starts the VM on the destination first and then fetches memory pages from the source on demand as they are accessed?
Step-by-step substitution:

1. **Post-Copy Migration** suspends the VM, transfers the execution context to resume on the target immediately, and pulls memory pages across the network on-demand when page faults occur.

Final Answer: **Post-copy migration**.

Solved Example: Minimizing Data and Energy in Pre-Copy Migration

Given: Which technique is most likely to reduce the total data transferred and energy consumed during pre-copy live migration?
Step-by-step substitution:

1. Compressing memory pages before network transmission reduces the total number of bytes transferred over the link, saving network bandwidth and transceiver energy.

Final Answer: **Compressing memory pages before transmission**.

Solved Example: Resource Management to Prevent Fog Overload

Given: What resource management strategy is most important to prevent overload when many edge devices send telemetry to nearby fog nodes?
Step-by-step substitution:

1. Implementing dynamic load balancing and cooperative workload shedding across neighboring fog nodes distributes load evenly and avoids localized saturation.

Final Answer: **Dynamic load balancing and workload offloading among fog nodes**.

Solved Example: Kubernetes Pod Placement Component

Given: Which Kubernetes component assigns newly created Pods to appropriate worker nodes based on resource constraints?
Step-by-step substitution:

1. The `kube-scheduler` filters and scores candidate nodes based on CPU/RAM availability, taints, tolerations, and affinity rules to select the optimal host for each Pod.

Final Answer: `kube-scheduler`.

Solved Example: Docker Build Output Artifact

Given: In the workflow: `Dockerfile` → `docker build` → `_____` → `docker run`, which entity is produced?
Step-by-step substitution:

1. The `docker build` command compiles the declarative instructions of a Dockerfile into an immutable, multi-layered **Docker image**.

Final Answer: **Docker image**.

Solved Example: Roles of Linux Namespaces and cgroups

Given: Which statement correctly describes the division of responsibilities between Linux namespaces and cgroups?
Step-by-step substitution:

1. **Namespaces** isolate system views (processes, networks, mounts).
2. **Control groups (cgroups)** meter, prioritize, and limit hardware resource consumption (CPU, memory, disk I/O).

Final Answer: **Namespaces provide isolation of processes and system resources, while cgroups regulate resource usage**.

Solved Example: Cold Migration Duration Computation

Given: Memory size $V_m = 32$ GB, usable network bandwidth $R = 16$ MB/s. Find approximate transfer duration.
Step-by-step substitution:

$$V_m = 32 \times 1024 \text{ MB} = 32,768 \text{ MB} \tag{10.9}$$

$$T = \frac{32,768 \text{ MB}}{16 \text{ MB/s}} = 2048 \text{ s} \approx 34.13 \text{ minutes} \tag{10.10}$$

Final Answer: **About 34 minutes**.

Solved Example: Docker Named Volume Data Persistence

Given: A database container is accidentally deleted, but its data files were stored in a Docker named volume. What happens to the data?
Step-by-step substitution:

1. Docker named volumes exist independently of container lifecycles on the host filesystem.
2. Deleting the container has zero impact on the volume; the data remains completely intact and can be mounted to a new container.

Final Answer: **The data persists safely within the named volume on the host**.

Solved Example: Core Comparison Between Containers and VMs

Given: Which statement about containers and virtual machines is correct?
Step-by-step substitution:

1. Virtual Machines virtualize physical hardware and execute separate guest OS kernels.
2. Containers share the host operating system kernel and isolate applications at the process level.

Final Answer: **Containers share the host operating system kernel, whereas virtual machines run separate guest operating systems**.

Emerging Paradigms: Dew Computing, Serverless Architecture, and Sustainable Cloud

11.1 Introduction and Week Overview

Week 11 investigates modern evolutionary paradigms that transcend traditional centralized server infrastructure. As distributed computing advances toward the extreme network edge and operational costs skyrocket, the curriculum addresses three cutting-edge domains:

1. **Dew Computing (DC):** Bringing cloud microservice capabilities directly to end devices to eliminate absolute internet dependency, balancing independence and cloud collaboration.
2. **Serverless Computing and Function-as-a-Service (FaaS):** Fine-grained, event-driven execution abstractions that completely mask server administration, scale to zero, and bill strictly for resource runtime.
3. **Sustainable and Green Cloud Computing:** Architectural frameworks, power-regulation strategies, and economic models designed to curb the escalating multi-terawatt energy consumption and carbon footprints of hyperscale Cloud Data Centers (CDCs).

11.2 Dew Computing Foundations

11.2.1 Motivation: Overcoming the Internet Dependency Bottleneck

While Cloud Computing provides virtually infinite computation and storage, it exhibits a fundamental vulnerability: **mandatory continuous Internet connectivity**. If network access is severed due to infrastructure failure, remote geography, or catastrophic events, traditional cloud services become entirely inaccessible.

Definition: Dew Computing (DC)

Dew Computing is an information technology paradigm where on-premises personal computing devices (smartphones, PCs, edge gateways) host lightweight, autonomous microservices that operate independently of an active Internet connection while collaborating with the centralized cloud whenever network connectivity is available.

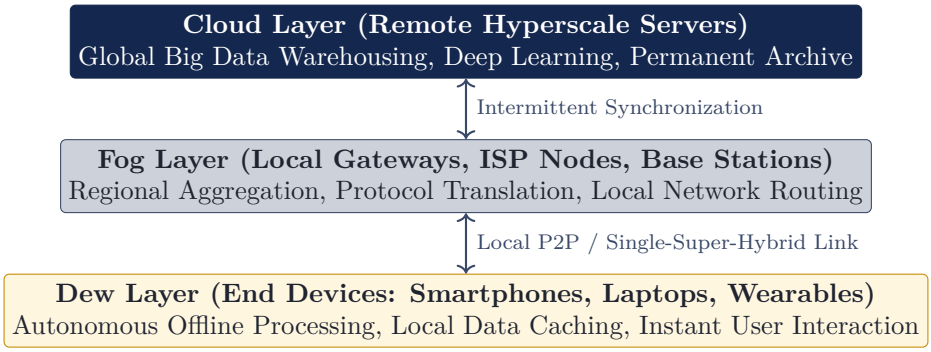


Figure 11.1: The Meteorological Hierarchy: Cloud (Sky) → Fog (Near Ground) → Dew (On the Ground).

11.2.2 Core Principles: Independence and Collaboration

Introduced by Yingwei Wang, the operational philosophy of Dew Computing rests upon two dual pillars:

- **Independence:** The local device must provide full or graceful core service functionality entirely offline without depending on continuous Internet or remote server availability.
- **Collaboration:** When network connectivity is established, local applications must automatically discover cloud endpoints, synchronize delta state modifications, and reconcile conflicts seamlessly.

A canonical everyday realization of Dew Computing is **Dropbox** or **Google Drive Desktop**: users read, edit, create, and delete files locally inside an offline directory without network access (independence). The moment an Internet connection is detected, background synchronization daemons propagate file deltas to the remote cloud server (collaboration).

11.2.3 Dew Service Architecture Taxonomy

Analogous to the classical SPI model of cloud computing, Dew Computing introduces a granular service taxonomy:

Dew Service Model	Definition	Exemplary Realization
Software-In-Dew (SiD)	Software configuration, license ownership, and code metadata are mastered in the cloud, while local runtime binaries execute independently offline on the end device.	Local office productivity suites with cloud license sync; offline mobile apps.
Platform-In-Dew (PiD)	Development runtime environments, APIs, and microframeworks exist on the local device, enabling local compilation and execution with cloud registry synchronization.	Local Docker container runtimes, offline Python/Node development toolchains.
Infrastructure-As-Dew (IaD)	Hardware capabilities (CPU, RAM, local storage) of diverse end devices are virtualized and pooled into local ad-hoc processing clusters.	P2P distributed file storage across office PCs; local Bluetooth mesh processing.
Database-In-Dew (DbID)	A lightweight local database engine (e.g., SQLite, Realm) processes queries locally with submillisecond response and synchronizes transactional deltas to cloud databases.	Mobile banking offline transaction caching; offline relational records.
Website-In-Dew (WiD)	A complete local web server instance runs directly on the client machine, rendering pages locally from cached templates and syncing dynamic content on reconnect.	Progressive Web Apps (PWAs) with Service Workers caching dynamic offline assets.
Storage-In-Dew (StID)	File data resides permanently in local non-volatile storage while background threads synchronize snapshots to remote cloud buckets.	Dropbox local file sync; OneDrive Smart Files offline mirrors.

11.3 Serverless Computing and Function-as-a-Service (FaaS)

11.3.1 The Paradigm Shift from Servers to Applications

In classical IaaS, developers rent virtual machines and must manage operating system patches, kernel configurations, firewall rules, and scaling thresholds. In PaaS, while OS management is abstracted, developers still configure server instances, manage runtime pools, and pay for idle compute capacity.

Serverless Computing elevates abstraction to the highest level: the cloud customer writes only modular, stateless business logic functions. The cloud provider assumes 100% of operational responsibilities—including provisioning hardware, container lifecycle management, automated horizontal scaling, high availability, and fault tolerance.

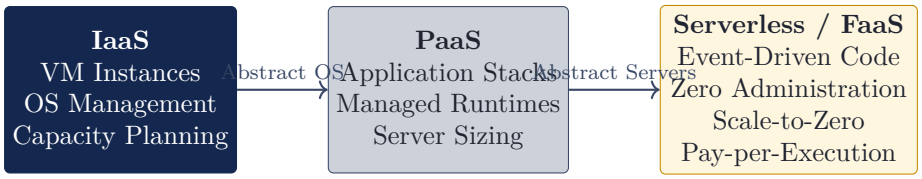


Figure 11.2: Evolutionary Trajectory of Cloud Abstraction Layers.

11.3.2 Architectural Characteristics of FaaS

- **Fine-Grained Granularity:** Applications are partitioned into discrete, single-purpose functions (e.g., resizing an image, validating an authentication token, parsing an incoming webhook).
- **Event-Driven Activation:** Functions execute strictly in response to designated asynchronous or synchronous triggers:
  - HTTP REST requests via API Gateways.
  - Object storage notifications (e.g., file uploaded to AWS S3 bucket).
  - Message broker streams (e.g., Apache Kafka, AWS Kinesis, RabbitMQ).
  - Cron-like scheduled timers.
- **Stateless Execution:** Functions retain no persistent internal state across invocations. Any required session state must be committed to external storage (e.g., Amazon DynamoDB, Redis, S3).
- **Automatic Scale-to-Zero and Scale-to-Infinity:** If zero requests arrive, zero instances run, and the customer incurs **$0.00 cost**. If 10,000 concurrent requests arrive simultaneously, the cloud platform automatically spins up 10,000 isolated execution environments in parallel.

11.3.3 The Serverless Cold Start Challenge

When an incoming event triggers a serverless function that has no pre-warmed idle container ready in memory:

1. The FaaS control plane selects a physical host.
2. It downloads the container image or code package.
3. It instantiates an isolated sandbox (using microVMs such as AWS Firecracker).

4. It initializes the language runtime (JVM, Python, Node.js) and executes global initialization code.

This sequence introduces a **Cold Start Latency** (ranging from 100 ms for Node/Python to $>$ 2 seconds for Java/JVM). Subsequent invocations reuse the warm container, executing with minimal latency ($<$ 10 ms) until the container is recycled after an idle timeout window (typically 5 to 15 minutes).

11.3.4 Serverless Economic Billing Formulation

Serverless billing abandons hourly instance leasing in favor of exact sub-second consumption metering:

$$\text{Total FaaS Cost} = N_{\text{req}} \times C_{\text{req}} + \sum_{i=1}^{N_{\text{req}}} (M_i \times T_i) \times C_{\text{GB-sec}} \tag{11.1}$$

Where:

- N_{req}: Total number of function invocations.
- C_{req}: Cost per million invocations (standardly \$0.20 per 10^6 calls).
- M_i: Memory allocated to the function (in GB, e.g., 512 MB = 0.5 GB).
- T_i: Execution duration of invocation i rounded to the nearest millisecond.
- $C_{\text{GB-sec}}$: Unit cost per Gigabyte-second (standardly $\approx$ \$0.0000166667/GB $\cdot$ s).

11.4 Sustainable and Green Cloud Computing

11.4.1 The Data Center Energy Dilemma

Hyperscale Cloud Data Centers (CDCs) have become the most power-dense industrial facilities on earth. Driven by machine learning training, real-time video streaming, and global cloud workflows:

- Global data center electricity consumption exceeded 200 TWh in 2020 and is projected to reach **8,000 TeraWatt-hours (TWh) by 2030** (representing nearly 10% to 15% of global electrical generation).
- Substantial electricity is generated from fossil fuel grids, translating directly into millions of metric tons of atmospheric CO_2 emissions.

11.4.2 Energy Breakdown Within a Cloud Data Center

Empirical measurements across modern enterprise data centers reveal that electricity consumption is distributed across five primary subsystems:

Subsystem	Typical Energy %	Primary Operational Drivers
Processors (CPUs / GPUs)	45%	Active computation, transistor switching, leakage current.
Cooling	20%	Chilled water plants, air handlers, CRAH units, fan walls.
Main Memory (RAM)	15%	Continuous capacitive DRAM refresh cycles.
Storage Subsystems	10%	Mechanical HDD spindle spinning, SSD flash controllers.
Networking Equipment	10%	Spine-leaf switches, optical transceivers, routers.

11.4.3 Energy Efficiency Metrics: PUE and DCiE

To benchmark environmental performance, the Green Grid consortium defined standardized metrics:

$$\text{Power Usage Effectiveness (PUE)} = \frac{\text{Total Facility Energy Consumption}}{\text{IT Equipment Energy Consumption}} = \frac{P_{\text{IT}} + P_{\text{Cooling}} + P_{\text{Lighting}} + P_{\text{Losses}}}{P_{\text{IT}}} \tag{11.2}$$

An ideal theoretical data center with zero cooling or power conversion overhead has PUE = 1.0. Historical enterprise facilities operated at PUE ≈ 2.0, whereas state-of-the-art hyperscale facilities (Google, Microsoft) achieve PUE $\approx 1.10 - 1.15$ using advanced evaporative cooling, AI-controlled temperature set-points, and direct-to-chip liquid cooling.

The reciprocal metric is **Data Center Infrastructure Efficiency (DCiE)**:

$$\text{DCiE} = \frac{1}{\text{PUE}} = \frac{\text{IT Equipment Energy Consumption}}{\text{Total Facility Energy Consumption}} \times 100\% \tag{11.3}$$

A PUE of 1.25 corresponds to DCiE = 80%.

11.4.4 Power Management and Conservation Approaches

Following the conceptual framework by Rajkumar Buyya and Sukhpal Singh Gill:

- **Dynamic Voltage and Frequency Scaling (DVFS):** CMOS CPU dynamic power scales cubically with clock frequency ($P \propto V^2 f \approx f^3$). Throttling clock frequency during light loads yields substantial energy savings. *Tradeoff:* Lower frequencies prolong execution times, risking SLA response time violations.
- **Dynamic VM Consolidation:** Migrating VMs from lightly loaded servers onto a compact set of physical blades and switching empty hosts into low-power ACPI sleep states (e.g., C6/S3 states consuming < 10 W instead of 150 W idle power).
- **Thermal-Aware Scheduling:** Directing computationally heavy workloads to server racks located in cooler physical aisles or during nighttime hours to decrease refrigeration workload.
- **Renewable Energy Microgrids:** Powering data centers directly with co-located solar arrays, wind farms, and battery energy storage systems (BESS).

11.5 Comprehensive Solved NPTEL-Style Questions (Week 11)

Solved Example: Dew Computing Core Tenets (NPTEL Assessment Problem)

Given: What are the two foundational operational principles that define the Dew Computing paradigm, and how do they interact?
Step-by-step substitution:

1. **Independence:** The client application executes autonomously on local device hardware without requiring an active or continuous Internet connection.
2. **Collaboration:** The application detects cloud connectivity and synchronizes data updates, configurations, and state modifications with centralized cloud servers.

Final Answer: **Independence and Collaboration**.

Solved Example: Dew Service Model Classification

Given: A mobile banking app allows a field loan officer to record customer transactions in a local SQLite database in offline villages. When returning to the branch office, the app automatically syncs transactional records to the bank's central Oracle database in the cloud. Which Dew service model is exemplified?
Step-by-step substitution:

1. The core mechanism is a local structured database engine handling queries locally and synchronizing transactional deltas with a cloud database.
2. By taxonomy definition, this matches Database-In-Dew (DbID).

Final Answer: **Database-In-Dew (DbID)**.

Solved Example: Serverless FaaS Economic Cost Calculation

Given: A microservice deployed on AWS Lambda executes $10,000,000$ invocations in a month. Each invocation runs for an average duration of 200 ms and is configured with 512 MB (0.5 GB) of RAM. Given pricing:

- Invocation charge: \$0.20 per $1,000,000$ requests.
- Compute charge: \$0.0000166667 per GB $\cdot$ second.

Calculate the total monthly serverless execution cost.
Step-by-step substitution:

1. **Invocation Request Cost:**

$$\text{Cost}_{\text{req}} = \frac{10,000,000}{1,000,000} \times \$0.20 = 10 \times \$0.20 = \$2.00 \tag{11.4}$$

2. **Compute Duration in GB-Seconds:**

$$\text{Execution Time per Request} = 200 \text{ ms} = 0.20 \text{ seconds} \tag{11.5}$$

$$\text{GB-seconds per Request} = 0.5 \text{ GB} \times 0.20 \text{ s} = 0.10 \text{ GB} \cdot \text{s} \tag{11.6}$$

$$\text{Total GB-seconds} = 10,000,000 \times 0.10 \text{ GB} \cdot \text{s} = 1,000,000 \text{ GB} \cdot \text{s} \tag{11.7}$$

3. **Compute Charge:**

$$\text{Cost}_{\text{compute}} = 1,000,000 \times \$0.0000166667 = \$16.67 \tag{11.8}$$

4. **Total Monthly Settlement:**

$$\text{Total Cost} = \$2.00 + \$16.67 = \$18.67 \tag{11.9}$$

Final Answer: Total monthly serverless cost is **$18.67**.

Solved Example: Serverless Cold Start Optimization Mechanism

Given: Which of the following strategies is most effective for mitigating cold-start latency in production FaaS architectures?
Step-by-step substitution:

1. Cold starts occur when the provider must instantiate a new runtime container sandbox from scratch.
2. Provisioned Concurrency / Pre-warming keeps a predetermined number of initialized container sandboxes permanently resident in memory, eliminating runtime initialization delay for latency-critical traffic.

Final Answer: **Provisioned Concurrency (Pre-warming container instances)**.

Solved Example: Data Center PUE and DCiE Formulation

Given: A cloud data center consumes a total of 25 MegaWatts (MW) of electrical power. Measurements indicate that internal IT equipment (servers, storage arrays, switches) consumes 16 MW, while cooling, lighting, and power distribution account for the remaining 9 MW. Find the Power Usage Effectiveness (PUE) and Data Center Infrastructure Efficiency (DCiE).
Step-by-step substitution:

1. **Compute PUE:**

$$\text{PUE} = \frac{\text{Total Power}}{\text{IT Power}} = \frac{25 \text{ MW}}{16 \text{ MW}} = 1.5625 \tag{11.10}$$

2. **Compute DCiE:**

$$\text{DCiE} = \frac{\text{IT Power}}{\text{Total Power}} \times 100\% = \frac{16}{25} \times 100\% = 64.0\% \tag{11.11}$$

$$\text{Verification: DCiE} = \frac{1}{\text{PUE}} = \frac{1}{1.5625} = 0.64 = 64\% \tag{11.12}$$

Final Answer: **PUE** $= 1.5625$, **DCiE** $= 64.0\%$.

Solved Example: Component Energy Distribution in Hyperscale Data Centers

Given: According to empirical data center surveys highlighted in NPTEL Lecture 54, which subsystem consumes the single largest fraction of electrical energy inside a cloud data center?
Step-by-step substitution:

1. Empirical energy breakdown: Processors (CPUs/GPUs) account for $\approx 45\%$, Cooling for $\approx 20\%$, Memory for $\approx 15\%$, Storage for $\approx 10\%$, and Networking for $\approx 10\%$.

Final Answer: **Processors (CPUs / GPUs) at approximately 45% of total energy**.

Solved Example: DVFS Power vs Latency Tradeoff

Given: A server CPU operates at voltage $V = 1.2$ V and frequency $f = 3.0$ GHz, drawing dynamic power $P_1 = 100$ W. If the hypervisor lowers the frequency to $f' = 2.0$ GHz and reduces supply voltage to $V' = 1.0$ V, compute the new dynamic power dissipation P_2 assuming $P \propto V^2 f$.
Step-by-step substitution:

1. Using the CMOS power formula:

$$\frac{P_2}{P_1} = \frac{(V')^2 \cdot f'}{V^2 \cdot f} = \frac{(1.0)^2 \times 2.0}{(1.2)^2 \times 3.0} = \frac{1.0 \times 2.0}{1.44 \times 3.0} = \frac{2.0}{4.32} \approx 0.463 \tag{11.13}$$

2. New dynamic power:

$$P_2 = 100 \text{ W} \times 0.463 = 46.3 \text{ Watts} \tag{11.14}$$

3. **Observation:** Energy dissipation drops by 53.7%, but task execution duration increases by 50% ($\frac{3.0}{2.0} = 1.5$), which can violate tight SLA deadlines.

Final Answer: New dynamic power is **46.3 Watts** (53.7% reduction).

Solved Example: State Management in Serverless Function Architecture

Given: Why are serverless functions strictly mandated to be stateless by cloud providers?
Step-by-step substitution:

1. Statelessness allows the provider to dynamically spawn, destroy, or route requests to any function instance across arbitrary physical nodes without synchronizing internal memory states.

2. State persistence must be delegated to scalable shared external data stores (e.g., DynamoDB, Redis, S3).

Final Answer: **To enable arbitrary horizontal scalability, rapid instantiation, and zero-downtime routing without cross-instance synchronization overhead**.

Solved Example: Website-In-Dew (WiD) Operational Mechanism

Given: In the Dew Computing framework, how does Website-In-Dew (WiD) serve web applications during complete network disconnection?
Step-by-step substitution:

1. WiD deploys a lightweight local web server or Progressive Web App Service Worker on the user's client machine.

2. Static assets and dynamic data templates cached locally are rendered without querying the remote web server. Delta updates sync automatically upon reconnecting.

Final Answer: **A local web service instance renders cached web content locally on the client machine, maintaining usability offline**.

Solved Example: Dynamic VM Consolidation Energy Savings

Given: A data center contains 200 servers. Each idle active server consumes 160 W, while an active server running at 80% load consumes 280 W. A sleeping server in ACPI C6 state consumes 10 W. Currently, 100 lightly loaded servers run at 20% load (consuming 190 W each). By live migration, these workloads are consolidated onto 25 servers running at 80% load, and the remaining 75 servers are placed into C6 sleep mode. Calculate the total power saved.

Step-by-step substitution:

1. **Initial Power Consumption:**

$$P_{\text{initial}} = 100 \times 190 \text{ W} = 19,000 \text{ Watts} \tag{11.15}$$

2. **Post-Consolidation Power Consumption:**

$$P_{\text{active}} = 25 \times 280 \text{ W} = 7,000 \text{ Watts} \tag{11.16}$$

$$P_{\text{sleep}} = 75 \times 10 \text{ W} = 750 \text{ Watts} \tag{11.17}$$

$$P_{\text{final}} = 7,000 + 750 = 7,750 \text{ Watts} \tag{11.18}$$

3. **Net Power Savings:**

$$\Delta P = 19,000 \text{ W} - 7,750 \text{ W} = 11,250 \text{ Watts} = 11.25 \text{ kW} \tag{11.19}$$

$$\text{Percentage Reduction} = \frac{11,250}{19,000} \times 100\% = 59.21\% \tag{11.20}$$

Final Answer: Total power saved is **11,250 Watts (11.25 kW)**, representing a **59.21% reduction**.

Advanced Paradigms and Case Studies: 5G Era, Cyber-Physical Systems, Spatial Cloud, and IoHT

12.1 Introduction and Week Overview

Week 12 explores the confluence of cloud, edge, and modern telecommunication systems across complex real-world cyber-physical environments. The curriculum culminates with four interconnected pillars:

1. **Cloud Computing in the 5G Era:** How fifth-generation cellular networks, network softwarization, and multi-access edge computing (MEC) eliminate the communication latency bottleneck.
2. **Cyber-Physical Systems (CPS) and Cloud Integration:** Bridging computational cyber cores with physical dynamics via closed-loop feedback, formalizing the NIST Cyber-Physical Cloud Computing (CPCC) architecture.
3. **Case Study I — Spatial Cloud Computing and Trajectory Cloud (Traj-Cloud):** Managing spatiotemporal big data, spatial indexing, distributed map-matching, and mobility trace mining over cloud infrastructure.
4. **Case Study II — Internet of Health Things (IoHT):** A complete four-tier hierarchical Cloud-Fog-Edge-IoT healthcare architecture evaluated empirically via iFogSim, physical embedded hardware prototypes, and Dew computing resilience.

12.2 Cloud Computing in the 5G Era

12.2.1 Generational Evolution of Mobile Networks

Mobile telecommunications have evolved across distinct architectural epochs:

Generation	Epoch	Core Capability & Technology	Latency Benchmark
1G	1980s	Analog voice transmission (AMPS, TACS, NMT)	$\sim$ 1000 ms
2G	Early 1990s	Digital voice, SMS, circuit-switched data (GSM, CDMA)	$\sim$ 300–500 ms
3G	Early 2000s	Mobile packet data, early mobile web (UMTS, CDMA2000)	$\sim$ 100–150 ms
4G LTE	2010s	All-IP mobile broadband, video streaming, LTE-Advanced	$\sim$ 30–50 ms
5G	2020s+	Multi-Gbps peak rates, ultra-low latency, massive IoT, MEC	$<$ 1–5 ms

12.2.2 The Three Functional Pillars of 5G

The International Telecommunication Union (ITU-R) under IMT-2020 standardizes 5G capabilities across three distinct service categories:

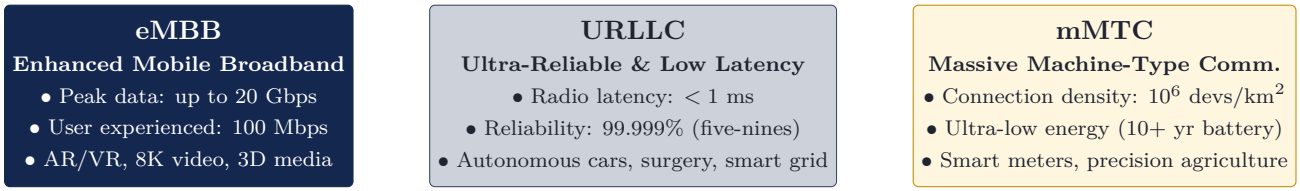


Figure 12.1: The ITU IMT-2020 5G Service Triangle.

12.2.3 5G and Cloud Synergy: Softwarization and Edge Convergence

The symbiosis between 5G and Cloud Computing operates in two directions:

1. **Cloudification of 5G (Network Softwarization):** 5G replaces proprietary telecommunication hardware with software running on commodity cloud data center servers via:
   - **Software-Defined Networking (SDN):** Decoupling the network control plane (centralized decision-making) from the data forwarding plane.
   - **Network Functions Virtualization (NFV):** Packaging firewalls, routers, EPC (Evolved Packet Core), and Baseband Units (BBUs) as virtualized network functions (VNFs) running on hypervisors or Kubernetes clusters.
   - **Network Slicing:** Creating multiple isolated virtual networks on a single shared physical infrastructure, each customized to meet specific QoS guarantees (e.g., an eMBB slice for video streaming alongside a URLLC slice for robotic surgery).
2. **Cloud Evolution Driven by 5G (Multi-Access Edge Computing — MEC):** Standardized by ETSI, MEC deploys cloud compute and storage resources directly at cellular Radio Access Network (RAN) base stations (gNodeB). By executing computation one hop away from User Equipment (UE), round-trip propagation delays drop from 50–100 ms (remote cloud) to under 5 ms (edge).

12.2.4 Mobile Cloud Computing (MCC) vs. MEC

Dimension	Mobile Cloud Computing (MCC)	Multi-Access Edge Computing (MEC)
Server Location	Centralized hyperscale cloud data centers across the wide-area Internet.	At local cellular base stations (gNodeB), aggregation points, or ISP gateways.
Latency	Moderate to high (50–150 ms) due to multi-hop WAN routing.	Ultra-low ($<$ 5 ms) via direct one-hop wireless access.
Context Awareness	Low; cloud has no direct insight into real-time radio signal or user cell location.	High; access to real-time radio channel quality, cell congestion, and user velocity.
Bandwidth Pressure	Strains backhaul and core telecommunication networks.	Keeps high-volume traffic local, drastically reducing core network backhaul usage.

12.3 Cyber-Physical Systems (CPS) and Cloud Integration

12.3.1 Foundations of Cyber-Physical Systems

Definition: Cyber-Physical System (CPS)

Coined in 2006 by Helen Gill at the National Science Foundation (NSF), a **Cyber-Physical System (CPS)** is an orchestration of computational algorithms and physical components. Embedded computers monitor and control physical processes via feedback loops, where physical processes affect computational logic and vice versa.

CPS is characterized by the **intersection**, rather than the union, of the cyber world and the physical world:

$$\text{CPS} = \text{Cyber} \cap \text{Physical} = \{\text{Computation}\} \times \{\text{Dynamics}\} \times \{\text{Communication}\} \times \{\text{Control}\} \quad (12.1)$$

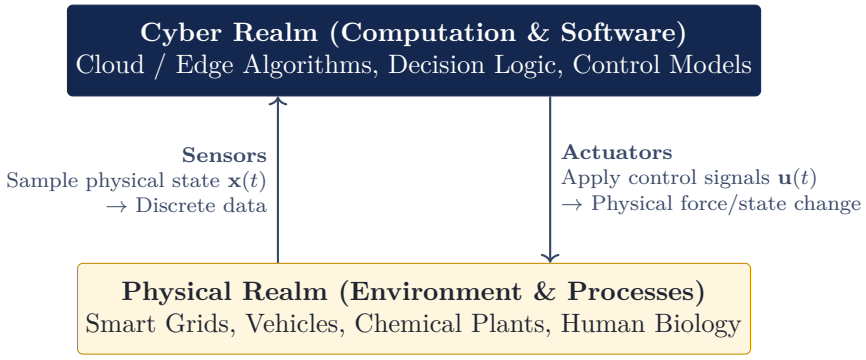


Figure 12.2: Closed-Loop Feedback Architecture in Cyber-Physical Systems.

12.3.2 CPS vs. Traditional Embedded Systems and IoT

- **Embedded Systems:** Focus on standalone computation embedded within a host device (e.g., microwave microcontroller), rarely networked or aware of broad physical dynamics.

- **Internet of Things (IoT):** Focuses on uniquely identifiable objects connected to the Internet primarily for data aggregation, monitoring, and telemetry collection.
- **Cyber-Physical Systems (CPS):** Mandates tight, continuous, closed-loop bidirectional feedback control with hard temporal deadlines, stability guarantees, and safety-critical constraints.

12.3.3 Cyber-Physical Cloud Computing (CPCC)

In August 2013, the National Institute of Standards and Technology (NIST) published a landmark report establishing the **Cyber-Physical Cloud Computing (CPCC)** architectural framework:

Definition: NIST CPCC Definition

Cyber-Physical Cloud Computing (CPCC) is a system environment that can rapidly build, modify, and provision cyber-physical systems composed of a set of cloud computing-based sensor, processing, control, and data services.

Key Benefits of CPCC

1. **Resource Pooling and Scalability:** Dynamically aggregates heterogeneous sensor and actuator fleets across geographical boundaries.
2. **Modular Composition:** Engineers can compose complex cyber-physical applications using modular microservices (Sensing-as-a-Service, Actuation-as-a-Service, Control-as-a-Service).
3. **Cross-Domain Interoperability:** Translates disparate industrial and proprietary physical fieldbus protocols (Modbus, CAN bus, OPC-UA) into unified cloud APIs.
4. **Adaptive Scaled Analytics:** Executes heavy predictive simulations (digital twins, finite element analysis) in the cloud while dispatching real-time control directives to the edge.

12.4 Case Study I: Spatial Cloud Computing and Traj-Cloud

12.4.1 Geospatial Data Characteristics

Spatial (or geospatial) data describes objects, phenomena, or events located on or near the Earth's surface. Every spatial datum is formally modeled as a three-dimensional tuple:

$$\mathcal{D}_{\text{spatial}} = \langle \mathbf{L}, \mathbf{A}, \mathbf{T} \rangle \tag{12.2}$$

where:

- $\mathbf{L} = (\text{Latitude}, \text{Longitude}, \text{Altitude})$ represents the geographical coordinate geometry.
- $\mathbf{A} = \{a_1, a_2, \ldots, a_k\}$ represents non-spatial thematic attributes (e.g., road speed limit, patient ID, air pollution level).
- $\mathbf{T} = [t_{\text{start}}, t_{\text{end}}]$ represents the temporal validity lifespan.

12.4.2 Spatial Cloud Computing Principles

Definition: Spatial Cloud Computing

Spatial Cloud Computing is the cloud computing paradigm driven by geospatial sciences and optimized by spatiotemporal principles for enabling geospatial scientific discoveries within a distributed computing environment.

Key spatiotemporal principles leveraged to optimize cloud resources include:

- **Tobler's First Law of Geography:** "Everything is related to everything else, but near things are more related than distant things." Cloud data partitioning strategies cluster spatially proximate points onto the same physical server nodes to minimize inter-node network shuffles.
- **Spatial Indexing (Quadtrees and R-Trees):** Standard B-trees fail in multi-dimensional space. Spatial cloud databases utilize R-trees (bounding box hierarchies) or Geohash-based spatial keys to achieve $\mathcal{O}(\log N)$ range and nearest-neighbor query performance.

12.4.3 Traj-Cloud: Trajectory Cloud for Mobility Trace Mining

Developed by researchers at IIT Kharagpur (S. Ghosh and S. K. Ghosh, IEEE COMSNETS 2019), **Traj-Cloud** is an end-to-end cloud platform designed to analyze massive GPS mobility trajectories for intelligent transportation systems (ITS) and smart-city applications.

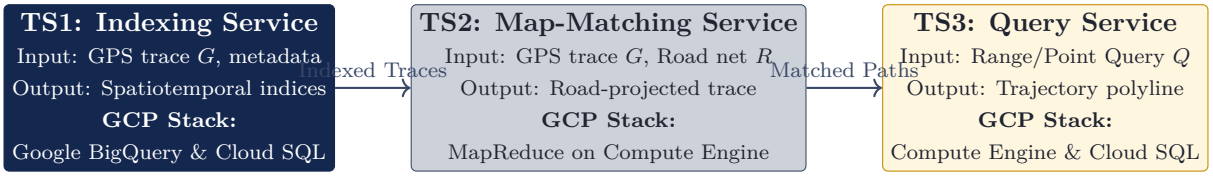


Figure 12.3: The Three Core Microservices of Traj-Cloud.

1. **TS1 — Trajectory Data Indexing Service:** Takes raw, noisy GPS coordinates, assigns spatiotemporal bounding boxes, and ingests them into analytical distributed column stores (Google BigQuery) and transactional indexes (Cloud SQL).
2. **TS2 — Trajectory Map-Matching Service:** Raw GPS signals exhibit sampling drift and urban canyon interference. TS2 projects raw coordinates onto the true topological road network graph R using distributed MapReduce algorithms on Google Compute Engine clusters to resolve ambiguities in near real-time.
3. **TS3 — Trajectory Query Service:** Executes high-throughput spatio-temporal range queries ("find all vehicles within 500 meters of junction J between 09:00 and 09:15") and trajectory similarity joins using Compute Engine worker pools.

12.5 Case Study II: Internet of Health Things (IoHT)

12.5.1 The Healthcare Paradigm Problem

Continuous patient health monitoring (e.g., cardiac arrhythmia, epileptic seizures, post-operative vitals) generates massive high-velocity data streams from body-worn sensors.

Transmitting continuous raw biometric streams directly to distant cloud data centers introduces fatal latency, consumes prohibitive cellular bandwidth, and suffers from total failure when Internet connectivity drops.

12.5.2 The Four-Tier Hierarchical Network Topology

To solve this, researchers at IIT Kharagpur designed a four-tier architecture:

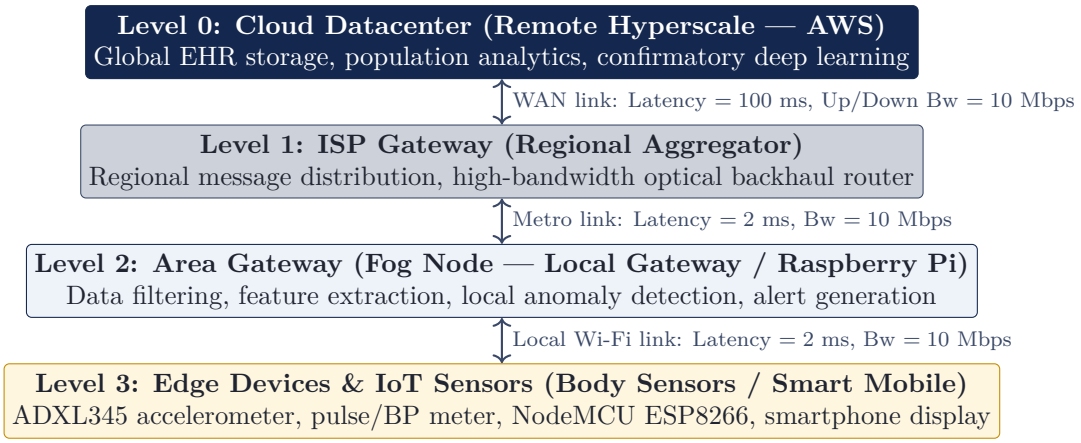


Figure 12.4: The 4-Tier Hierarchical Network Model for IoHT.

12.5.3 Empirical Simulation Parameters (iFogSim Model)

Based on the foundational iFogSim experiments presented in Lectures 59–60:

Device Tier	MIPS	RAM (MB)	Up Bw (kbps)	Down Bw (kbps)	Level	Cost / MIPS
Cloud	44,800	40,000	100	10,000	0	$0.01
ISP Gateway	2,800	4,000	10,000	10,000	1	$0.00 (Local)
Area Gateway	2,800	4,000	10,000	10,000	2	$0.00 (Local)
Mobile Edge	350	1,000	10,000	270	3	$0.00 (User)

Link Latency Distribution

- Body Sensor → Mobile Edge: 1 ms
- Mobile Edge → Area Gateway (Fog): 2 ms
- Area Gateway → ISP Gateway: 2 ms
- ISP Gateway → Cloud Datacenter: 100 ms
- Mobile Edge → Local Display: 1 ms

12.5.4 Application Module Placement: Fog Model vs. Cloud Model

The healthcare monitoring application consists of five logical modules:

Application Module	Placement in Fog-Based Model	Placement in Cloud-Based Model
Client Module	Mobile Device (Edge, Level 3)	Mobile Device (Edge, Level 3)
Data Filtering Module	Area Gateway (Fog, Level 2)	Cloud Datacenter (Level 0)
Data Processing Module	Area Gateway (Fog, Level 2)	Cloud Datacenter (Level 0)
Event Handler Module	Area Gateway (Fog, Level 2)	Cloud Datacenter (Level 0)
Confirmatory Module	Cloud Datacenter (Level 0)	Cloud Datacenter (Level 0)

12.5.5 Comparative Performance Evaluation Metrics

Simulations across expanding configurations (Config 1: 4 users, Config 2: 8 users, Config 3: 16 users, Config 4: 32 users, Config 5: 64 users) revealed definitive findings:

1. **Control Loop Latency:** In the Fog model, because the control loop (Sensor → Mobile → Area Gateway → Event Handler → Display) executes entirely within Levels 2 and 3, latency remains virtually **constant and minimal** (≈ 5–8 ms), independent of user scale. In the Cloud model, every packet travels to Level 0 and back, causing latency exceeding 210 ms, with severe queueing degradation as user count climbs.
2. **Network Bandwidth Usage:** In the Fog model, only the few positive anomaly events trigger the Confirmatory Module on the cloud, resulting in negligible WAN traffic. In the Cloud model, continuous raw biometric telemetry is pumped to the cloud, congesting wide-area links.
3. **Cost of Execution:** Fog devices (Area Gateways, Raspberry Pis) are customer-owned capital equipment incur zero per-second cloud billing fees. The Cloud model bills continuously for millions of MIPS, multiplying total operational cost.
4. **Energy Consumption:** Mobile device energy remains identical across both architectures because the mobile edge performs the same sensor acquisition load. However, cloud data center energy drops by orders of magnitude under the Fog model.

12.5.6 Hardware Prototype Implementation

The simulated IoHT architecture was physically built and validated using:

- **Biometric Sensor Node:** Customized Blood Pressure (BP) and Pulse meter outputting ASCII serial data at 9600 baud.
- **Movement Tracking Sensor:** ADXL345 3-axis digital accelerometer sampling acceleration at high frequency.
- **Edge Acquisition Node:** NodeMCU ESP8266 board (CP2102) with integrated Wi-Fi microcontroller.
- **Fog Gateway Node:** Raspberry Pi 3 (64-bit ARM quad-core, 1 GB RAM, on-board Wi-Fi and Bluetooth).
- **Cloud Tier:** Amazon Web Services (AWS) EC2 instances.

Signal Processing and Machine Learning Pipeline



Figure 12.5: Wearable Activity Classification Pipeline.

1. **Composite Magnitude Calculation:**

$$A[t] = \sqrt{x[t]^2 + y[t]^2 + z[t]^2} \tag{12.3}$$

2. **5-Point Moving Average Smoothing:** To eliminate transient electronic noise, each sampled signal point is replaced by the mean of itself, its two predecessors, and its two successors:

$$\bar{A}[t] = \frac{A[t-2] + A[t-1] + A[t] + A[t+1] + A[t+2]}{5} \tag{12.4}$$

3. **Feature Extraction:** Over sliding windows, six statistical features are extracted: Maximum Amplitude, Minimum Amplitude, Mean Amplitude, Standard Deviation in Amplitude, Energy in Time Domain ($\sum A[t]^2$), and Energy in Frequency Domain.

4. **Classification via k-Nearest Neighbors (KNN):** Features are normalized via min-max scaling ($Y = \frac{x - \min}{\max - \min}$). Optimal parameter $k = 3$ was established via 5-fold cross validation.

Demonstration Cardiac Risk Alert Logic

The edge node calculates calibrated risk indicators:

$$f = (\text{KNN Activity Class Value}) + 1 \tag{12.5}$$

$$p = \frac{\text{BPM}}{f}, \quad s = \frac{\text{Systolic BP}}{f}, \quad d = \frac{\text{Diastolic BP}}{f} \tag{12.6}$$

A severe emergency alarm triggers if:

$$(p \geq 170) \wedge (s \geq 180 \text{ mmHg}) \wedge (d \geq 120 \text{ mmHg}) \implies \text{HeartAttackAlarm} = \textbf{True} \tag{12.7}$$

12.5.7 Dew Resilience Extension in Healthcare

To prevent catastrophic patient monitoring blackouts when local fog routers or Internet backhauls fail:

- The smartphone or wearable edge node contains a **Dew Health Service Engine**.
- If Cloud or Fog connectivity is severed, the Dew engine immediately transitions to autonomous offline processing: vital signs are evaluated locally against emergency thresholds, acoustic patient alarms sound directly from the mobile handset, and time-stamped delta logs are stored in a local SQLite database (Database-in-Dew).
- The instant connectivity is restored, the Dew daemon reconciles and synchronizes all cached patient logs to the hospital cloud database.

Metric	Pure Cloud	Cloud-Fog-Edge	With Dew Computing
On-Premise Resource Utilization	Low	Sub-optimal	Optimal
Required Network Connectivity	Continuous Internet	Local Area Network	Fully Autonomous (Offline)
System Availability / Uptime	Low (Internet fragile)	High	Maximum (Fault-tolerant)
Bandwidth Demand	High (All data upstream)	Low (Alerts only)	Minimal (Delta sync)
Control Loop Latency	High (> 200 ms)	Ultra-low (< 10 ms)	Immediate (< 2 ms)

12.6 Comprehensive Solved NPTEL-Style Questions

Solved Example: 5G Service Pillar Categorization and QoS Matching

Given: Categorize the following real-world applications into their primary 5G service pillar (**eMBB**, **URLLC**, or **mMTC**):

(a) Autonomous vehicular collision avoidance system requiring < 1 ms air interface latency.

(b) Streaming ultra-high-definition 8K 360° virtual reality video to concert attendees.

(c) Smart agricultural soil moisture monitoring deploying 50,000 battery-powered sensors across farmland.

(d) Remote telesurgery robot operated by a specialist in a metropolitan hospital.

Step-by-step substitution:

1. **Autonomous collision avoidance:** Demands sub-millisecond deterministic latency and 99.999% transmission reliability to avert physical collisions $\implies$ **URLLC**.
2. 8**K** 360° **VR streaming:** Requires sustained multi-hundred Mbps throughput and wide radio channels, prioritizing high bandwidth over sub-millisecond delivery $\implies$ **eMBB**.
3. **Smart agricultural soil sensors:** Exhibits extreme connection density, very low individual data rates, and multi-year battery lifespans without human maintenance $\implies$ **mMTC**.
4. **Remote telesurgery robot:** Human tactile feedback and surgical tool precision require deterministic low latency (< 5 ms) and absolute reliability $\implies$ **URLLC**.

Final Answer: (a) **URLLC**, (b) **eMBB**, (c) **mMTC**, (d) **URLLC**.

Solved Example: MEC vs. Cloud Round-Trip Latency and Bandwidth Conservation

Given: A connected vehicle captures high-definition camera frames generating a data payload of $S = 2$ MB per event. The vehicle can offload image processing either to a local Multi-Access Edge Computing (MEC) server co-located at the base station or to a centralized Cloud data center:

- Uplink wireless radio bandwidth $R_{\text{radio}} = 100$ Mbps, one-hop radio latency $T_{\text{radio}} = 2$ ms.
- Core network backhaul bandwidth to Cloud $R_{\text{cloud}} = 20$ Mbps, wide-area propagation delay $T_{\text{wan}} = 45$ ms.
- Processing execution duration on MEC server: $T_{\text{proc, MEC}} = 30$ ms.
- Processing execution duration on Cloud server: $T_{\text{proc, Cloud}} = 10$ ms (faster compute).
- Result response packet is negligible in size (0 ms downlink transmission time).

Compute the total turnaround latency for both MEC offloading and Cloud offloading.
Step-by-step substitution:

1. Convert payload size to bits:

$$S = 2 \text{ MB} = 2 \times 8 = 16 \text{ Megabits (Mb)} \tag{12.8}$$

2. **Turnaround Latency for MEC:**

$$T_{\text{trans, radio}} = \frac{S}{R_{\text{radio}}} = \frac{16 \text{ Mb}}{100 \text{ Mbps}} = 0.160 \text{ s} = 160 \text{ ms} \tag{12.9}$$

$$T_{\text{total, MEC}} = T_{\text{radio}} + T_{\text{trans, radio}} + T_{\text{proc, MEC}} + T_{\text{radio}} \tag{12.10}$$

$$T_{\text{total, MEC}} = 2 \text{ ms} + 160 \text{ ms} + 30 \text{ ms} + 2 \text{ ms} = 194 \text{ ms} \tag{12.11}$$

3. **Turnaround Latency for Cloud:** The packet must traverse both the radio uplink and the wide-area network backhaul:

$$T_{\text{trans, cloud}} = \frac{S}{R_{\text{cloud}}} = \frac{16 \text{ Mb}}{20 \text{ Mbps}} = 0.800 \text{ s} = 800 \text{ ms} \tag{12.12}$$

$$T_{\text{total, Cloud}} = T_{\text{radio}} + T_{\text{trans, radio}} + T_{\text{wan}} + T_{\text{trans, cloud}} + T_{\text{proc, Cloud}} + T_{\text{wan}} + T_{\text{radio}} \tag{12.13}$$

$$T_{\text{total, Cloud}} = 2 + 160 + 45 + 800 + 10 + 45 + 2 = 1064 \text{ ms} \tag{12.14}$$

4. Even though the cloud CPU processes three times faster (10 ms vs 30 ms), the bottleneck in backhaul transmission and WAN propagation makes MEC over 5.4× faster (194 ms vs 1064 ms).

Final Answer: **MEC Latency: 194 ms**; **Cloud Latency: 1064 ms**.

Solved Example: Closed-Loop Cyber-Physical System Temporal Stability Bound

Given: A cyber-physical smart grid feedback control loop monitors frequency oscillations. The physical generator dynamics remain stable if the total closed-loop communication and computational delay τ satisfies the Nyquist stability margin:

$$\tau \leq \tau_{\max} = 25 \text{ ms} \tag{12.15}$$

The closed loop consists of:

- Phasor Measurement Unit (PMU) sensor sampling and A/D conversion time: $t_s = 3$ ms.
- Sensor-to-Controller transmission delay: t_{tx1}.
- Controller state estimation and optimal power flow computation time: $t_c = 8$ ms.
- Controller-to-Actuator transmission delay: t_{tx2}.
- Physical actuator response time: $t_a = 4$ ms.

Determine the maximum permissible bidirectional network delay ($t_{tx1} + t_{tx2}$) and state whether a centralized cloud with 30 ms WAN latency can maintain closed-loop stability.
Step-by-step substitution:

1. Write the closed-loop delay equation:

$$\tau = t_s + t_{tx1} + t_c + t_{tx2} + t_a \leq \tau_{\max} \tag{12.16}$$

2. Substitute the known internal hardware delays:

$$3 \text{ ms} + 8 \text{ ms} + 4 \text{ ms} + (t_{tx1} + t_{tx2}) \leq 25 \text{ ms} \tag{12.17}$$

$$15 \text{ ms} + (t_{tx1} + t_{tx2}) \leq 25 \text{ ms} \implies (t_{tx1} + t_{tx2}) \leq 10 \text{ ms} \tag{12.18}$$

3. If control algorithms are placed in a remote cloud where network transit alone takes 30 ms:

$$\tau_{\text{cloud}} = 15 \text{ ms} + 30 \text{ ms} = 45 \text{ ms} > 25 \text{ ms} \tag{12.19}$$

This violates the stability boundary, causing hazardous generator oscillation and grid tripping.

Final Answer: Maximum allowable network delay is **10 ms**. A centralized cloud (30 ms) **cannot maintain stability**; local edge/fog control is strictly required.

Solved Example: NIST Cyber-Physical Cloud Computing (CPCC) Framework

Given: According to the NIST 2013 Special Report on Cyber-Physical Cloud Computing, how does CPCC expand traditional cloud service abstractions (IaaS, PaaS, SaaS) to accommodate physical reality?
Step-by-step substitution:

1. Traditional cloud abstractions handle purely virtual abstractions: virtual hardware (IaaS), execution platforms (PaaS), and end-user software (SaaS).
2. CPCC introduces physical actuation and sensing abstractions:
   - **Sensing-as-a-Service (S2aaS):** Virtualizing real-world sensors (cameras, temperature probes, PMUs) into addressable cloud streams.
   - **Actuation-as-a-Service (AaaS):** Virtualizing physical actuators (valves, robotic arms, switches) with strict concurrency locking and safety protocols.
   - **Control-as-a-Service (CaaS):** Providing real-time closed-loop control algorithms hosted on deterministic edge-cloud nodes.

Final Answer: **CPCC abstracts physical hardware by introducing Sensing-as-a-Service, Actuation-as-a-Service, and Control-as-a-Service alongside traditional cloud models**.

Solved Example: Spatial Data Tuple and Quadtree Indexing Complexity

Given: A spatial cloud platform ingests $N = 1,000,000$ vehicle coordinates distributed across a metropolitan bounding box of 32 km $\times$ 32 km. The system constructs a spatial Point-Region (PR) Quadtree where each non-empty quadrant cell is recursively partitioned into 4 children until each leaf contains at most $B = 10$ points.

(a) Specify the formal three-part tuple defining each spatial vehicle datum.

(b) Compute the expected depth D of the resulting Quadtree assuming uniform spatial distribution.

Step-by-step substitution:

1. **Formal Tuple:** Each datum is represented as $\mathcal{D} = \langle \mathbf{L}, \mathbf{A}, \mathbf{T} \rangle$, where $\mathbf{L} = (\text{lat}, \text{lon})$ is

geographic coordinates, $\mathbf{A} = (\text{vehicle_id}, \text{speed}, \text{heading})$ are thematic attributes, and $\mathbf{T} = (\text{timestamp})$ is temporal validity.

2. **Quadtree Depth:** In a balanced Quadtree with fan-out 4, the number of leaf buckets required for N points with capacity B is:

$$N_{\text{leaves}} \approx \frac{N}{B} = \frac{1,000,000}{10} = 100,000 \tag{12.20}$$

Since each tree level quadruples the number of cells (4^D leaves at depth D):

$$4^D \geq 100,000 \implies D = \lceil \log_4(100,000) \rceil = \left\lceil \frac{\ln(100,000)}{\ln(4)} \right\rceil = \left\lceil \frac{11.5129}{1.3863} \right\rceil = \lceil 8.305 \rceil = 9 \tag{12.21}$$

3. Average point search complexity is $\mathcal{O}(D) = \mathcal{O}(\log_4 N) \approx 9$ node visits, compared to an exhaustive linear scan of 10^6 records.

Final Answer: (a) $\mathcal{D} = \langle \mathbf{L}, \mathbf{A}, \mathbf{T} \rangle$; (b) Expected tree depth is **9 levels**.

Solved Example: Traj-Cloud Map-Matching Service MapReduce Speedup

Given: A fleet of 10,000 smart taxis produces 1.8×10^7 raw GPS points per hour. An unoptimized single-node serial map-matching algorithm projects GPS coordinates onto a digital road network at a rate of 2,500 points per second. In Traj-Cloud, the TS2 service executes a distributed MapReduce job on a cluster of $M = 16$ Google Compute Engine instances, where parallel processing efficiency is $\eta = 85\%$ due to network shuffling overhead. Calculate the single-node execution time, the cluster execution time, and the effective speedup.

Step-by-step substitution:

1. **Single-Node Execution Time (T_1):**

$$T_1 = \frac{18,000,000 \text{ points}}{2,500 \text{ points/s}} = 7,200 \text{ seconds} = 2.0 \text{ hours} \tag{12.22}$$

Because $T_1 = 2.0$ hours > 1.0 hour, a single node cannot keep up with real-time streaming data.

2. **Cluster Processing Rate and Duration (T_M):**

$$R_{\text{cluster}} = M \times R_1 \times \eta = 16 \times 2,500 \times 0.85 = 34,000 \text{ points/s} \tag{12.23}$$

$$T_M = \frac{18,000,000}{34,000} \approx 529.41 \text{ seconds} \approx 8.82 \text{ minutes} \tag{12.24}$$

3. **Effective Speedup (S):**

$$S = \frac{T_1}{T_M} = \frac{7200 \text{ s}}{529.41 \text{ s}} \approx 13.6 \tag{12.25}$$

Theoretical linear speedup would be 16.0; with $\eta = 0.85$, $S = 16 \times 0.85 = 13.6$.

Final Answer: Single-node duration: **2.0 hours**; Cluster duration: **8.82 minutes**; Effective speedup: **13.6x**.

Solved Example: IoHT 4-Tier Network Latency Breakdown

Given: In the four-tier IoHT architecture (Sensors → Mobile → Area Gateway → ISP Gateway → Cloud), using the empirical link delays established in Lecture 59:

- $T_{\text{sensor}\to\text{mobile}} = 1$ ms
- $T_{\text{mobile}\to\text{areaGW}} = 2$ ms
- $T_{\text{areaGW}\to\text{ispGW}} = 2$ ms
- $T_{\text{ispGW}\to\text{cloud}} = 100$ ms
- $T_{\text{mobile}\to\text{display}} = 1$ ms

Compare the total control-loop response time (from sensor trigger to mobile alert display) for:

(a) The **Fog-based Model**, where filtering, processing, and event handling reside on the Area Gateway (assuming internal processing delay $T_{\text{proc, fog}} = 3$ ms).

(b) The **Cloud-based Model**, where data travels to the Cloud Datacenter for processing (assuming internal cloud compute delay $T_{\text{proc, cloud}} = 1$ ms).

Step-by-step substitution:

1. **Fog-Based Control Loop Path:** Sensor → Mobile → Area Gateway (Process) → Mobile → Display

$$T_{\text{fog}} = T_{\text{sensor}\to\text{mobile}} + T_{\text{mobile}\to\text{areaGW}} + T_{\text{proc, fog}} + T_{\text{areaGW}\to\text{mobile}} + T_{\text{mobile}\to\text{display}} \tag{12.26}$$

$$T_{\text{fog}} = 1 \text{ ms} + 2 \text{ ms} + 3 \text{ ms} + 2 \text{ ms} + 1 \text{ ms} = 9 \text{ ms} \tag{12.27}$$

2. **Cloud-Based Control Loop Path:** Sensor → Mobile → Area Gateway → ISP Gateway → Cloud (Process) → ISP Gateway → Area Gateway → Mobile → Display

$$T_{\text{up}} = 1 + 2 + 2 + 100 = 105 \text{ ms} \tag{12.28}$$

$$T_{\text{down}} = 100 + 2 + 2 + 1 = 105 \text{ ms} \tag{12.29}$$

$$T_{\text{cloud}} = T_{\text{up}} + T_{\text{proc, cloud}} + T_{\text{down}} = 105 \text{ ms} + 1 \text{ ms} + 105 \text{ ms} = 211 \text{ ms} \tag{12.30}$$

3. The Fog architecture delivers an emergency alert in 9 **ms**, representing a 23.4× latency reduction over the Cloud architecture (211 **ms**).

Final Answer: (a) Fog Model Latency: **9 ms**; (b) Cloud Model Latency: **211 ms**.

Solved Example: WAN Bandwidth and Cloud Cost Reduction in IoHT

Given: A hospital monitors $K = 500$ cardiac patients. Each wearable sensor transmits vital telemetry at a continuous rate of $r = 4$ KB/s (32 kbps).

- In the **Cloud Model**, all raw telemetry streams are transmitted continuously to AWS Cloud.
- In the **Fog Model**, Area Gateways filter and process vitals locally; only confirmed critical anomaly events are uploaded to the Cloud. Clinical statistics show critical

events occur on average for only 0.5% of monitoring time.

- AWS charges \$0.08 per GB of ingress/storage bandwidth and \$0.01 per million MIPS computed. (Each patient stream requires 10 MIPS of continuous processing).

Compute the monthly WAN data transferred (in GB) and compute cost under both models (assume 30 days/month).

Step-by-step substitution:

1. Total seconds in 30 days:

$$T_{\text{month}} = 30 \times 24 \times 3600 = 2,592,000 \text{ seconds} \tag{12.31}$$

2. **Cloud Model Data Volume:**

$$V_{\text{cloud}} = 500 \times 4 \text{ KB/s} \times 2,592,000 \text{ s} = 5,184,000,000 \text{ KB} \approx 5,184 \text{ GB} = 5.184 \text{ TB} \tag{12.32}$$

Bandwidth Cost: $5,184 \text{ GB} \times \$0.08/\text{GB} = \$414.72$. **Cloud Compute Load:**

$$\text{Total MIPS} = 500 \times 10 \text{ MIPS} \times 2,592,000 \text{ s} = 1.296 \times 10^{10} \text{ MIPS-seconds} \tag{12.33}$$

$$\text{Compute Cost} = \frac{1.296 \times 10^{10}}{10^6} \times \$0.01 = 12,960 \times \$0.01 = \$129.60 \tag{12.34}$$

$$\text{Total Cloud Cost} = \$414.72 + \$129.60 = \$544.32/\text{month} \tag{12.35}$$

3. **Fog Model Data Volume:** Only 0.5% (0.005) of data reaches Cloud:

$$V_{\text{fog}} = 5,184 \text{ GB} \times 0.005 = 25.92 \text{ GB} \tag{12.36}$$

Bandwidth Cost: $25.92 \text{ GB} \times \$0.08/\text{GB} = \$2.07$. **Compute Cost:** $\$129.60 \times 0.005 = \$0.65$.

$$\text{Total Fog Cost} = \$2.07 + \$0.65 = \$2.72/\text{month} \tag{12.37}$$

4. **Savings:** The Fog model saves 5,158 GB (99.5%) of WAN bandwidth and cuts monthly cloud operational cost from \$544.32 to \$2.72 (99.5% savings).

Final Answer: Cloud Model: **5,184 GB** (**\$544.32**); Fog Model: **25.92 GB** (**\$2.72**).

Solved Example: ADXL345 Accelerometer 5-Point Smoothing Calculation

Given: An ADXL345 wearable accelerometer samples raw tripartite acceleration components (x, y, z) in units of g (9.81 m/s^2). Five consecutive samples produce the following 3-axis vectors:

$$t_1 : (0.60, 0.80, 0.00)$$
$$t_2 : (0.00, 1.00, 0.00)$$
$$t_3 : (0.80, 0.00, 0.60)$$
$$t_4 : (0.50, 0.50, \sqrt{0.50})$$
$$t_5 : (0.00, 0.00, 1.00)$$

(a) Compute the composite acceleration magnitude $A[t]$ for each timestamp.

(b) Apply the 5-point moving average smoothing filter to evaluate the smoothed output

$\bar{A}[3]$ at timestamp $t = 3$.

Step-by-step substitution:

1. Formula: $A[t] = \sqrt{x^2 + y^2 + z^2}$.
   - $A[1] = \sqrt{0.60^2 + 0.80^2 + 0.00^2} = \sqrt{0.36 + 0.64} = \sqrt{1.00} = 1.00$ g
   - $A[2] = \sqrt{0.00^2 + 1.00^2 + 0.00^2} = \sqrt{1.00} = 1.00$ g
   - $A[3] = \sqrt{0.80^2 + 0.00^2 + 0.60^2} = \sqrt{0.64 + 0.36} = \sqrt{1.00} = 1.00$ g
   - $A[4] = \sqrt{0.50^2 + 0.50^2 + (\sqrt{0.50})^2} = \sqrt{0.25 + 0.25 + 0.50} = \sqrt{1.00} = 1.00$ g
   - $A[5] = \sqrt{0.00^2 + 0.00^2 + 1.00^2} = \sqrt{1.00} = 1.00$ g
2. Evaluate 5-point moving average for $t = 3$:

$$\bar{A}[3] = \frac{A[1] + A[2] + A[3] + A[4] + A[5]}{5} = \frac{1.00 + 1.00 + 1.00 + 1.00 + 1.00}{5} = \frac{5.00}{5} = 1.00 \text{ g} \tag{12.38}$$

3. Notice that while individual directional vector axes oscillated widely due to body movement, the smoothed composite magnitude accurately captures the constant 1.00 g gravitational baseline.

Final Answer: (a) $A[1] = A[2] = A[3] = A[4] = A[5] = 1.00$ g; (b) $\bar{A}[3] = \mathbf{1.00}$ g.

Solved Example: Activity Classification via K-Nearest Neighbor (KNN)

Given: In the IoHT activity recognition subsystem, an unknown sample U with normalized feature vector $(x_1, x_2) = (0.50, 0.40)$ is classified among four labeled training instances:

- $P_1 = (0.45, 0.35)$ — Class: **Resting** ($f = 1$)
- $P_2 = (0.55, 0.45)$ — Class: **Resting** ($f = 1$)
- $P_3 = (0.60, 0.60)$ — Class: **Running** ($f = 3$)
- $P_4 = (0.90, 0.80)$ — Class: **Running** ($f = 3$)

Using Euclidean distance and $k = 3$ nearest neighbors, classify sample U and compute the resulting cardiac scaling factor $f_{\text{scale}} = f + 1$.

Step-by-step substitution:

1. Calculate squared Euclidean distances $d^2(U, P_i) = (x_{U1} - x_{i1})^2 + (x_{U2} - x_{i2})^2$:

$$d^2(U, P_1) = (0.50-0.45)^2+(0.40-0.35)^2 = 0.05^2+0.05^2 = 0.0025+0.0025 = 0.0050 \implies d \approx 0.0707 \tag{12.39}$$

$$d^2(U, P_2) = (0.50-0.55)^2+(0.40-0.45)^2 = (-0.05)^2+(-0.05)^2 = 0.0050 \implies d \approx 0.0707 \tag{12.40}$$

$$d^2(U, P_3) = (0.50-0.60)^2+(0.40-0.60)^2 = (-0.10)^2+(-0.20)^2 = 0.0100+0.0400 = 0.0500 \implies d \approx 0.2236 \tag{12.41}$$

$$d^2(U, P_4) = (0.50-0.90)^2+(0.40-0.80)^2 = (-0.40)^2+(-0.40)^2 = 0.1600+0.1600 = 0.3200 \implies d \approx 0.5657 \tag{12.42}$$

2. The $k = 3$ closest points are P_1 ($d = 0.0707$), P_2 ($d = 0.0707$), and P_3 ($d = 0.2236$).

3. Tally majority votes: $P_1 \rightarrow$ Resting, $P_2 \rightarrow$ Resting, $P_3 \rightarrow$ Running.
4. The majority class (2 out of 3 votes) is **Resting** ($f = 1$).
5. Cardiac scaling factor:
$$f_{\text{scale}} = f + 1 = 1 + 1 = 2 \tag{12.43}$$

Final Answer: Predicted Class: **Resting** ($f = 1$); Scaling Factor: $f_{\text{scale}} = \mathbf{2}$.

Solved Example: Dew Resilience and Offline Fault Tolerance in IoHT

Given: Explain why adding a Dew Computing layer to a Cloud-Fog-Edge healthcare architecture achieves higher availability than a pure Cloud-Fog system during wide-area network partitions.
Step-by-step substitution:

1. In a standard Cloud-Fog architecture, if the local Wi-Fi router fails or the patient walks out of range of the Area Gateway, the wearable sensor is disconnected from processing services.
2. With **Dew Computing**, the user's mobile device hosts a full standalone microservice stack (SiD, DbID) that executes locally on the handset without any network connectivity:
   - Critical emergency thresholds (pulse, arrhythmia) are evaluated directly in mobile memory.
   - Alarms trigger locally on the patient's phone speaker, alerting caregivers nearby immediately.
   - Delta vitals are persisted locally in SQLite (DbID) and synchronized to the cloud hospital database automatically once network connectivity is re-established.

Final Answer: **Dew Computing guarantees zero-downtime offline autonomous monitoring, local alerting, and automatic delta reconciliation upon network reconnection**.

Solved Example: Comprehensive Comparison of Cloud, Fog, and Dew Computing

Given: Synthesize the key architectural trade-offs between Cloud Computing, Fog Computing, and Dew Computing across the five standard evaluation dimensions shown below.
Step-by-step substitution:

1. **Compute Location:** Cloud runs in hyperscale remote CDCs; Fog runs on local intermediate nodes (gateways, edge routers); Dew runs directly on user end devices (smartphones, PCs).
2. **Latency:** Cloud has high WAN latency (50–200 ms); Fog provides low LAN latency (5–15 ms); Dew provides immediate sub-millisecond local response (< 1 ms).
3. **Internet Dependency:** Cloud requires continuous Internet; Fog requires local network connectivity; Dew operates completely offline with intermittent cloud synchronization.
4. **Hardware Resource Scale:** Cloud provides virtually unlimited compute/storage; Fog

provides moderate multi-core gateway compute; Dew is constrained to local handset/PC hardware.

5. **Primary Failure Mode:** Cloud fails when WAN/ISP link drops; Fog fails when local gateway crashes; Dew provides graceful local degradation with zero network dependency.

Final Answer: **Dew provides maximum offline resilience and minimum latency, Fog provides local area aggregation and low-latency control, while Cloud provides global multi-petabyte analytics and archival storage**.